VNIVERSITAT DE VALÈNCIA

Grado en Ingeniería Informática




VNIVERSITAT DE VALÈNCIA

Trabajo Fin de Grado

# Análisis de vulnerabilidades web con OWASP ZAP y propuesta de buenas prácticas para el desarrollo de aplicaciones seguras

**Autor:** Alberto Moreno Alcácer

**Tutor:** Ramón Vicente Cirilo Gimeno

**Junio 2026**



VNIVERSITAT DE VALÈNCIA



Escola Tècnica Superior d'Enginyeria ETSE-UV

Trabajo Fin de Grado

# Análisis de vulnerabilidades web con OWASP ZAP y propuesta de buenas prácticas para el desarrollo de aplicaciones seguras


**Autor: Alberto Moreno Alcácer**

**Tutor: Ramón Vicente Cirilo Gimeno**

**Declaración de autoría:**

Yo, Alberto Moreno Alcácer, declaro la autoría del Trabajo Fin de Grado titulado "Análisis de vulnerabilidades web con OWASP ZAP y propuesta de buenas prácticas para el desarrollo de aplicaciones seguras" y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 23 de junio de 2026

Fdo: Alberto Moreno Alcácer

**Resumen:**

Este Trabajo de Fin de Grado aborda la seguridad de aplicaciones web desde un enfoque teórico y práctico. En primer lugar, se revisan marcos y guías relacionados con el desarrollo seguro, como NIST SSDF, OWASP Top 10, OWASP ASVS y OWASP ZAP, con el objetivo de situar el análisis dentro de referencias utilizadas en el ámbito de la ciberseguridad.
A partir de esta base, se diseña e implementa una aplicación web vulnerable desarrollada con Flask y SQLite, ejecutada en un entorno local macOS ARM. La aplicación incorpora de forma deliberada varias vulnerabilidades habituales en aplicaciones web: inyección SQL en el inicio de sesión, XSS reflejado en la funcionalidad de búsqueda, un fallo de control de acceso en el panel de administración y ausencia de cabeceras HTTP de seguridad.
Posteriormente, la aplicación se analiza con OWASP ZAP, combinando el escaneo automático con comprobaciones manuales desde el navegador. Este análisis permite confirmar las vulnerabilidades introducidas, revisar las alertas generadas por la herramienta y observar también sus limitaciones, especialmente en aquellos fallos que dependen de la lógica interna de la aplicación.
Con los resultados obtenidos, se relacionan las vulnerabilidades detectadas con OWASP Top 10 y OWASP ASVS, y se proponen buenas prácticas y correcciones concretas para cada problema identificado. Finalmente, se plantea una metodología sencilla de desarrollo seguro basada en el diseño, la implementación, las pruebas y la revisión continua, junto con una revisión de costes y planificación del trabajo realizado.

**Palabras clave:** Ciberseguridad, OWASP ZAP, Aplicaciones web, Vulnerabilidades, OWASP Top 10, OWASP ASVS, Desarrollo seguro.

**Abstract:**

This Final Degree Project addresses web application security from both a theoretical and practical perspective. First, it reviews frameworks and guidelines related to secure development, such as NIST SSDF, OWASP Top 10, OWASP ASVS and OWASP ZAP, in order to place the analysis within references commonly used in the cybersecurity field.

Based on this foundation, a vulnerable web application is designed and implemented using Flask and SQLite, running in a local macOS ARM environment. The application deliberately includes several common web vulnerabilities: SQL injection in the login process, reflected XSS in the search functionality, a broken access control issue in the administration panel, and missing HTTP security headers.

The application is then analyzed with OWASP ZAP, combining automated scanning with manual checks from the browser. This analysis makes it possible to confirm the introduced vulnerabilities, review the alerts generated by the tool and observe its limitations, especially in issues that depend on the internal logic of the application.

Based on the results obtained, the detected vulnerabilities are related to OWASP Top 10 and OWASP ASVS, and good practices and specific code corrections are proposed for each identified problem. Finally, a simple secure development methodology is presented, based on design, implementation, testing and continuous review, together with a cost and planning review of the work carried out.

**Keywords:** Cybersecurity, OWASP ZAP, Web applications, Vulnerabilities, OWASP Top 10, OWASP ASVS, Secure development.

**Resum:**

Aquest Treball de Fi de Grau aborda la seguretat d'aplicacions web des d'un enfocament teòric i pràctic. En primer lloc, es revisen marcs i guies relacionats amb el desenvolupament segur, com NIST SSDF, OWASP Top 10, OWASP ASVS i OWASP ZAP, amb l'objectiu de situar l'anàlisi dins de referències utilitzades en l'àmbit de la ciberseguretat.
A partir d'aquesta base, es dissenya i s'implementa una aplicació web vulnerable desenvolupada amb Flask i SQLite, executada en un entorn local macOS ARM. L'aplicació incorpora de forma deliberada diverses vulnerabilitats habituals en aplicacions web: injecció SQL en l'inici de sessió, XSS reflectit en la funcionalitat de cerca, un error de control d'accés en el panell d'administració i absència de capçaleres HTTP de seguretat.
Posteriorment, l'aplicació s'analitza amb OWASP ZAP, combinant l'escaneig automàtic amb comprovacions manuals des del navegador. Aquesta anàlisi permet confirmar les vulnerabilitats introduïdes, revisar les alertes generades per l'eina i observar també les seues limitacions, especialment en aquells errors que depenen de la lògica interna de l'aplicació.
Amb els resultats obtinguts, es relacionen les vulnerabilitats detectades amb OWASP Top 10 i OWASP ASVS, i es proposen bones pràctiques i correccions concretes per a cada problema identificat. Finalment, es planteja una metodologia senzilla de desenvolupament segur basada en el disseny, la implementació, les proves i la revisió contínua, juntament amb una revisió de costos i planificació del treball realitzat.

**Paraules clau:** Ciberseguretat, OWASP ZAP, Aplicacions web, Vulnerabilitats, OWASP Top 10, OWASP ASVS, Desenvolupament segur.

**Agradecimientos:**

En primer lugar, quiero agradecer a mis padres su apoyo durante todos estos años. Su ayuda, confianza y paciencia han sido fundamentales para poder llegar hasta aquí, tanto durante la carrera como en la realización de este Trabajo Fin de Grado.
También quiero agradecer a todas aquellas personas que me han acompañado a lo largo de mis estudios universitarios y que, de una forma u otra, han contribuido a que este camino haya sido más llevadero.
Por último, quiero expresar mi agradecimiento a mi tutor del Trabajo Fin de Grado, Ramón Vicente Cirilo Gimeno, por su orientación, disponibilidad e implicación durante el desarrollo de este proyecto. Sus revisiones y comentarios me han ayudado a mejorar la memoria y a enfocar el trabajo para que cumpliera con los objetivos esperados.

# Índice general


**1. Introducción** **19**

1.1. Introducción . . . 19
1.2. Motivación . . . 20
1.3. Objetivos . . . 20
1.3.1. Objetivo general . . . 20
1.3.2. Objetivos específicos . . . 21
1.4. Organización de la memoria . . . 21

**2. Estado del arte** **23**

2.1. Introducción al estado del arte . . . 23
2.2. Marcos de referencia para el desarrollo seguro . . . 24
2.2.1. NIST Secure Software Development Framework (SSDF) . . . 24
2.2.2. Otros marcos de referencia organizativos . . . 25
2.2.3. Esquema Nacional de Seguridad (ENS) . . . 26
2.3. Marcos orientados a la seguridad en aplicaciones web . . . 27
2.3.1. OWASP Application Security Verification Standard (ASVS) . . . 27
2.3.2. OWASP Software Assurance Maturity Model (SAMM) . . . 28
2.4. Clasificación y priorización de vulnerabilidades . . . 28
2.4.1. OWASP Top 10 (2025) . . . 28
2.5. Herramientas de análisis de seguridad en aplicaciones web . . . 29
2.5.1. OWASP ZAP y el análisis dinámico de aplicaciones web . . . 29
2.6. Selección de marcos de referencia para este Trabajo de Fin de Grado . . . 31

**3. Requisitos, especificaciones, coste, riesgos, viabilidad** **33**

3.1. Descripción del sistema . . . 33
3.1.1. Entorno de pruebas: aplicación vulnerable . . . 33
3.1.2. Definición del modelo de buenas prácticas . . . 34
3.2. Requisitos . . . 34
3.2.1. Requisitos funcionales . . . 34

3.2.2. Requisitos no funcionales . . . 35
3.3. Planificación del trabajo . . . 35
3.3.1. Tareas relacionadas con la aplicación vulnerable . . . 35
3.3.2. Tareas relacionadas con el modelo de buenas prácticas . . . 36
3.4. Costes . . . 36
3.4.1. Coste de desarrollo de la aplicación . . . 37
3.4.2. Coste de definición de buenas prácticas . . . 37
3.4.3. Costes indirectos . . . 38
3.5. Riesgos . . . 38
3.6. Viabilidad . . . 39

**4. Análisis** **41**
4.1. Introducción . . . 41
4.2. Análisis de la aplicación web . . . 41
4.2.1. Actores del sistema . . . 41
4.2.2. Casos de uso . . . 42
4.2.3. Descripción de los casos de uso . . . 43
4.2.4. Modelo conceptual de datos . . . 45
4.2.5. Diagrama de clases de análisis . . . 46
4.3. Análisis de las pruebas de seguridad . . . 47
4.3.1. Objetivo de las pruebas con OWASP ZAP . . . 47
4.3.2. Alcance de las pruebas . . . 47
4.3.3. Puntos de entrada a analizar . . . 48
4.3.4. Vulnerabilidades previstas en el análisis . . . 48
4.3.5. Configuración general de OWASP ZAP . . . 48
4.4. Relación entre el análisis del sistema y las pruebas de seguridad . . . 49

**5. Diseño** **51**
5.1. Introducción . . . 51
5.2. Diseño de la aplicación web . . . 52
5.2.1. Arquitectura general de la aplicación . . . 52
5.2.2. Diseño de componentes . . . 53
5.2.3. Diseño de la interfaz de usuario . . . 54
5.2.4. Diagrama de clases de diseño . . . 55
5.2.5. Modelo lógico de datos . . . 58
5.3. Diseño de las vulnerabilidades introducidas . . . 59
5.3.1. Diseño de la inyección SQL . . . 59

5.3.2. Diseño del XSS reflejado . . . 60
5.3.3. Diseño del fallo de control de acceso . . . 60
5.3.4. Diseño de la configuración insegura de cabeceras HTTP . . . 61
5.4. Diseño de las pruebas con OWASP ZAP . . . 61
5.4.1. Particularización de las pruebas . . . 62
5.4.2. Acciones previstas de OWASP ZAP . . . 62
5.4.3. Diseño de la actividad de test . . . 63
5.4.4. Tratamiento de los resultados obtenidos . . . 64
5.4.5. Criterios de revisión de resultados . . . 64

**6. Implementación y pruebas** **67**
6.1. Introducción . . . 67
6.2. Implementación de la aplicación vulnerable . . . 67
6.2.1. Aspecto final de la aplicación . . . 68
6.2.2. Entorno de desarrollo . . . 70
6.2.3. Estructura del proyecto . . . 71
6.2.4. Implementación de la autenticación . . . 72
6.2.5. Implementación de la gestión de comentarios . . . 72
6.2.6. Implementación de la funcionalidad de búsqueda . . . 72
6.2.7. Implementación del panel de administración . . . 73
6.2.8. Implementación de las vulnerabilidades introducidas . . . 73
6.3. Ejecución de pruebas con OWASP ZAP . . . 74
6.3.1. Preparación del entorno de pruebas . . . 74
6.3.2. Configuración de OWASP ZAP . . . 74
6.3.3. Exploración de la aplicación mediante Spider . . . 75
6.3.4. Ejecución del escaneo activo . . . 76
6.3.5. Revisión de alertas en OWASP ZAP . . . 76
6.3.6. Comprobación de la inyección SQL . . . 77
6.3.7. Comprobación del XSS reflejado . . . 79
6.3.8. Comprobación del control de acceso . . . 80
6.3.9. Comprobación de cabeceras HTTP de seguridad . . . 80
6.4. Resumen de resultados obtenidos . . . 83
6.5. Clasificación de vulnerabilidades . . . 84
6.5.1. Relación con OWASP Top 10 . . . 84
6.5.2. Relación con OWASP ASVS . . . 85
6.6. Conclusión del capítulo . . . 86

**7. Buenas prácticas y metodología de desarrollo seguro** **89**
7.1. Introducción . . . 89
7.2. Buenas prácticas derivadas del análisis realizado . . . 89
7.2.1. Prevención de inyección SQL . . . 90
7.2.2. Prevención de XSS reflejado . . . 91
7.2.3. Control de acceso y autorización . . . 92
7.2.4. Configuración segura de cabeceras HTTP . . . 92
7.3. Corrección de las vulnerabilidades detectadas . . . 93
7.3.1. Corrección de la inyección SQL . . . 93
7.3.2. Corrección del XSS reflejado . . . 94
7.3.3. Corrección del control de acceso . . . 95
7.3.4. Corrección de las cabeceras HTTP de seguridad . . . 96
7.4. Propuesta de metodología de desarrollo seguro . . . 97
7.4.1. Seguridad desde el diseño . . . 97
7.4.2. Implementación segura . . . 98
7.4.3. Pruebas de seguridad durante el desarrollo . . . 99
7.4.4. Revisión y mejora continua . . . 100
7.5. Relación con OWASP ASVS . . . 100
7.6. Conclusión del capítulo . . . 101

**8. Conclusiones** **103**
8.1. Revisión de costes y planificación . . . 103
8.2. Conclusiones . . . 104
8.3. Trabajo futuro . . . 105

**A. Apéndice** **107**
A.1. Configuración del entorno de pruebas . . . 107
A.2. Ejemplo de consulta vulnerable a SQL Injection . . . 107

**Bibliografía** **109**

# Capítulo 1

# Introducción

## 1.1. Introducción

Las aplicaciones web se han consolidado como uno de los principales canales de acceso a servicios digitales en ámbitos como el comercio electrónico, la administración pública, la educación o la banca. Según el Data Breach Investigations Report 2024 de Verizon, las aplicaciones web siguen siendo uno de los vectores más habituales en incidentes de seguridad reportados a nivel global [1]. Como consecuencia, las vulnerabilidades presentes en este tipo de aplicaciones representan hoy en día uno de los principales vectores de riesgo en materia de ciberseguridad.

A pesar de la existencia de estándares como NIST SSDF [2] o guías técnicas como OWASP ASVS [3], siguen apareciendo vulnerabilidades que podrían haberse evitado en fases tempranas del desarrollo. En muchos casos, estas vulnerabilidades no se deben a fallos técnicos complejos, sino a deficiencias en el diseño, la implementación o el uso incorrecto de mecanismos de seguridad conocidos. Esta situación pone de manifiesto la necesidad de integrar la seguridad como un aspecto fundamental dentro del ciclo de vida del desarrollo del software, y no como una fase añadida al final del proceso.

En este contexto, este Trabajo de Fin de Grado se centra en el análisis práctico de vulnerabilidades web y en la aplicación de metodologías orientadas al desarrollo seguro. El trabajo propone el estudio de vulnerabilidades frecuentes en aplicaciones web mediante la construcción de un entorno controlado que reproduce escenarios reales de riesgo, permitiendo observar cómo se manifiestan estos fallos y cómo pueden ser identificados mediante herramientas de análisis de seguridad.

Para ello, se emplea la herramienta OWASP ZAP como apoyo principal en la fase de análisis, combinando técnicas de escaneo automatizado con la interpretación manual de los resultados obtenidos. A partir de este análisis, se plantea una reflexión crítica sobre las limitaciones de las herramientas automáticas y la importancia de apoyarse en marcos de referencia reconocidos para comprender el origen de las vulnerabilidades y definir medidas de mitigación adecuadas.

El trabajo adopta como punto de partida distintos marcos y guías de seguridad orientados a aplicaciones web, con especial atención a aquellos que facilitan la identificación de riesgos frecuentes y la definición de buenas prácticas de desarrollo seguro. A partir de este análisis, se propone un conjunto de directrices orientadas a reducir la aparición de vulnerabilidades durante el desarrollo de aplicaciones web, con un enfoque práctico y

aplicable a proyectos reales de pequeña y mediana escala.

La memoria se estructura de la siguiente manera: en primer lugar, se presenta un estado del arte que revisa los principales marcos de referencia y enfoques relacionados con la seguridad en aplicaciones web. A continuación, se describen los requisitos, la planificación y la viabilidad del proyecto. Posteriormente, se desarrolla el análisis, el diseño y la implementación del entorno de pruebas, junto con la evaluación de las vulnerabilidades detectadas. Finalmente, se exponen las conclusiones obtenidas y se plantean posibles líneas de trabajo futuro.

## 1.2. Motivación

La seguridad en aplicaciones web se ha convertido en un aspecto crítico dentro del desarrollo de software, no solo por el impacto técnico de las vulnerabilidades, sino también por sus consecuencias económicas, legales y reputacionales para las empresas. En muchos proyectos, especialmente de pequeño y mediano tamaño, la seguridad sigue tratándose como un requisito secundario o se limita al uso puntual de herramientas automáticas, sin una aplicación sistemática de metodologías de desarrollo seguro.

Esta situación provoca que vulnerabilidades conocidas y ampliamente documentadas continúen apareciendo de forma recurrente en aplicaciones. En numerosos casos, estos problemas podrían haberse evitado mediante la adopción de marcos de referencia adecuados y la aplicación de buenas prácticas desde las fases iniciales del desarrollo. Sin embargo, la falta de directrices claras o de ejemplos prácticos dificulta que dichas metodologías se integren de forma efectiva en el proceso de desarrollo.

La motivación de este Trabajo de Fin de Grado surge, por tanto, de la necesidad de analizar cómo pueden aplicarse de manera práctica distintas metodologías de seguridad orientadas a aplicaciones web, combinando el uso de herramientas de análisis con marcos de referencia reconocidos. El objetivo no es únicamente identificar vulnerabilidades, sino comprender su origen, su relación con decisiones de diseño e implementación, y las medidas que permiten prevenirlas.

A partir de este enfoque, el trabajo pretende generar un conjunto de directrices y buenas prácticas que sirvan como apoyo a desarrolladores en la creación de aplicaciones web más seguras. Estas directrices buscan ser aplicables a contextos reales, priorizando soluciones viables y alineadas con metodologías de desarrollo habituales, y facilitando la incorporación de la seguridad como un elemento transversal dentro del ciclo de vida del software.

## 1.3. Objetivos

### 1.3.1. Objetivo general

Analizar las vulnerabilidades presentes en aplicaciones web mediante el uso de la herramienta OWASP ZAP y proponer un conjunto de buenas prácticas, basadas en marcos y estándares reconocidos, que sirvan como guía para el desarrollo de aplicaciones web más seguras.

### 1.3.2. Objetivos específicos

- Revisar el estado del arte en materia de seguridad de aplicaciones web, analizando los principales marcos de referencia, estándares y guías existentes.
- Diseñar e implementar una aplicación web que incorpore vulnerabilidades de forma controlada, con el objetivo de facilitar su análisis.
- Utilizar la herramienta OWASP ZAP para realizar pruebas de seguridad sobre la aplicación desarrollada, identificando y clasificando las vulnerabilidades detectadas.
- Analizar los resultados obtenidos durante el proceso de evaluación de seguridad, relacionándolos con los marcos de referencia estudiados.
- Proponer un conjunto de buenas prácticas y recomendaciones orientadas al desarrollo seguro de aplicaciones web, basadas en el análisis realizado.

## 1.4. Organización de la memoria

Este documento se estructura en ocho capítulos. El primer capítulo introduce el contexto, la motivación y los objetivos del trabajo. El segundo capítulo presenta el estado del arte, analizando los principales marcos de referencia, vulnerabilidades web y herramientas de seguridad. El tercer capítulo define los requisitos, la planificación, los costes, los riesgos y la viabilidad del proyecto. El cuarto capítulo recoge el análisis del sistema, incluyendo la aplicación web y la definición inicial de las pruebas de seguridad. El quinto capítulo aborda el diseño del sistema y la particularización de las pruebas OWASP. El sexto capítulo describe la implementación de la aplicación y la ejecución de las pruebas realizadas con OWASP ZAP. El séptimo capítulo presenta las buenas prácticas y la metodología de desarrollo seguro propuesta, incorporando el enfoque *Security by Design*. Finalmente, el octavo capítulo recoge las conclusiones y posibles líneas de trabajo futuro.

# Capítulo 2

# Estado del arte

## 2.1. Introducción al estado del arte

La seguridad en el desarrollo de software, especialmente en aplicaciones web expuestas a Internet, se ha convertido en un aspecto crítico en los últimos años. El informe *Data Breach Investigations Report* de Verizon señala que las aplicaciones web continúan siendo uno de los vectores de ataque más habituales en incidentes de seguridad [1].

Esta situación no se debe únicamente a ataques sofisticados, sino también a errores frecuentes de diseño, implementación o configuración. Cuando una aplicación está accesible públicamente, cualquier fallo puede tener consecuencias que van más allá del plano técnico, afectando también a la organización que la mantiene.

Abordar la seguridad únicamente cuando el software ya está en producción suele resultar ineficiente. El NIST, en la guía SP 800-218, recomienda integrar actividades de seguridad desde las primeras fases del desarrollo [2], ya que muchos problemas tienen su origen en decisiones de diseño adoptadas al inicio del proyecto. Cuando estos fallos se detectan tarde, su corrección suele ser más costosa y compleja. Por este motivo, distintos marcos insisten en tratar la seguridad como parte integral del ciclo de vida del software y no como una actividad posterior.

Para estructurar esta integración de la seguridad, han aparecido marcos como el SSDF del NIST o los estándares de OWASP, que buscan convertir la seguridad en una actividad planificada y repetible. Estos marcos proporcionan recomendaciones estructuradas que abarcan tanto aspectos organizativos y metodológicos como directrices técnicas, permitiendo reducir la probabilidad de introducir vulnerabilidades durante el desarrollo de las aplicaciones.

En el ámbito específico de las aplicaciones web, existen además iniciativas orientadas a la identificación, clasificación y priorización de las vulnerabilidades más habituales, así como modelos que permiten evaluar el grado de madurez de los procesos de seguridad de una organización. La combinación de estos enfoques no solo permite detectar vulnerabilidades, sino también entender por qué aparecen y qué decisiones de diseño o implementación las han provocado.

En las siguientes secciones se revisan los marcos que considero más relevantes para este trabajo, explicando qué aportan y, sobre todo, qué limitaciones tienen cuando el objetivo es analizar vulnerabilidades concretas en aplicaciones web. Se analizan sus objetivos y su alcance para justificar qué marcos se utilizarán en este trabajo y cuáles se descartan

como referencia principal. Este análisis servirá como fundamento teórico para el estudio práctico de vulnerabilidades y la propuesta de buenas prácticas que se desarrollan en los capítulos posteriores.

## 2.2. Marcos de referencia para el desarrollo seguro

En muchos proyectos, las vulnerabilidades se detectan cuando la aplicación ya está en producción. Esto ha llevado a proponer marcos que integran la seguridad desde el inicio del desarrollo. Estos enfoques parten de una idea común: la seguridad no puede depender únicamente de revisiones finales o de la corrección de errores una vez el sistema está en funcionamiento, sino que debe formar parte de las decisiones que se toman durante el diseño, la planificación y la construcción del software.

De forma general, los marcos de desarrollo seguro se apoyan en el concepto de *Secure Software Development Lifecycle* (Secure SDLC), que propone incorporar actividades de seguridad a lo largo de todas las fases del ciclo de vida del software. Entre estas actividades se incluyen la definición temprana de requisitos de seguridad, el análisis de amenazas durante la fase de diseño, la aplicación de principios de arquitectura segura y la verificación continua del cumplimiento de dichos requisitos a lo largo del desarrollo.

En la práctica, uno de los principales problemas es que los fallos de seguridad suelen detectarse cuando el sistema ya está desplegado, lo que obliga a rehacer partes del desarrollo. La corrección de fallos de seguridad en fases avanzadas o en entornos de producción suele implicar mayores costes, retrasos en los despliegues y un aumento del riesgo operativo. En cambio, cuando la seguridad se considera desde las primeras etapas del desarrollo, es posible prevenir categorías completas de vulnerabilidades antes de que lleguen a materializarse.

Estos marcos no sustituyen a las herramientas técnicas, sino que ayudan a organizarlas dentro de un proceso coherente. Su objetivo es proporcionar una base metodológica que permita a las organizaciones estructurar sus procesos, asignar responsabilidades y establecer prácticas coherentes que reduzcan de forma sistemática la aparición de vulnerabilidades en el software desarrollado.

### 2.2.1. NIST Secure Software Development Framework (SSDF)

El *Secure Software Development Framework* (SSDF), publicado por el *National Institute of Standards and Technology* (NIST) en la guía SP 800-218 [2], surge como respuesta a una carencia habitual en muchos procesos de desarrollo de software: la ausencia de una integración sistemática de la seguridad a lo largo del ciclo de vida del desarrollo. En numerosos modelos de desarrollo, la seguridad se aborda de forma tardía o puntual, lo que favorece la aparición recurrente de vulnerabilidades que podrían haberse evitado en fases tempranas del proyecto.

El objetivo principal del SSDF es reducir la cantidad de vulnerabilidades que llegan a los entornos de producción y limitar el impacto de aquellas que no se detectan o corrigen a tiempo. Para ello, el marco pone el foco en las causas raíz de los problemas de seguridad, que no se limitan únicamente a errores de programación, sino que incluyen decisiones de diseño inadecuadas, configuraciones inseguras, supuestos de confianza incorrectos o análisis de riesgos incompletos o desactualizados. Esto supone incorporar actividades de

seguridad antes de que el software llegue a producción, en lugar de limitarse a corregir incidentes una vez detectados.

Desde el punto de vista estructural, el SSDF organiza sus recomendaciones en cuatro grandes grupos de prácticas. En primer lugar, se encuentra la preparación de la organización, que abarca aspectos relacionados con la definición de responsabilidades, la formación de los equipos y la adopción de políticas internas que faciliten el desarrollo seguro. En segundo lugar, se incluyen prácticas orientadas a la protección del software, centradas en salvaguardar los componentes frente a accesos no autorizados o manipulaciones. El tercer grupo se orienta a la producción de software bien asegurado, promoviendo actividades como el diseño seguro, la revisión del código y la realización de pruebas de seguridad durante el desarrollo. Finalmente, el marco contempla la respuesta ante vulnerabilidades, abordando la identificación, gestión y corrección de fallos una vez el software ha sido liberado, así como la mejora continua de los procesos para evitar recurrencias.

Un aspecto especialmente relevante del SSDF es que no se plantea como una lista cerrada de requisitos ni como una guía prescriptiva. El propio NIST subraya que el marco no pretende funcionar como una checklist, sino como una base para adoptar un enfoque de seguridad basado en el riesgo. Las prácticas descritas están orientadas a resultados y no imponen el uso de herramientas, tecnologías o metodologías concretas, lo que permite su adaptación a distintos entornos de desarrollo, lenguajes de programación y modelos de trabajo, como desarrollo ágil o DevOps.

En el contexto de este Trabajo de Fin de Grado, el SSDF se utiliza como un marco de referencia general que permite situar la seguridad dentro del ciclo de vida del software desde una perspectiva organizativa y metodológica. Sus principios sirven para justificar la necesidad de considerar la seguridad desde el diseño y proporcionan una base sobre la que se apoyan marcos más específicos orientados a aplicaciones web, que se analizan en los apartados siguientes.

### 2.2.2. Otros marcos de referencia organizativos

Además de los marcos específicamente orientados al desarrollo de software, existen estándares de carácter más general cuyo objetivo es gestionar la seguridad de la información a nivel organizativo. Entre ellos, destacan las normas ISO/IEC 27001 e ISO/IEC 27002, ampliamente utilizadas en entornos empresariales y administrativos [4, 5].

La norma ISO/IEC 27001 define los requisitos para establecer, implantar y mantener un Sistema de Gestión de la Seguridad de la Información (ISMS). Su enfoque se basa en la identificación y gestión de riesgos asociados a la información que una organización maneja, independientemente de su tamaño, sector o tecnología utilizada. En este sentido, ISO/IEC 27001 no se centra en la seguridad de aplicaciones concretas, sino en proporcionar un marco estructurado que permita a la organización gestionar de forma sistemática los riesgos de seguridad, teniendo en cuenta factores organizativos, humanos y tecnológicos [4].

Por su parte, la norma ISO/IEC 27002 complementa a la 27001 proporcionando un conjunto de controles de seguridad que sirven como guía para la implantación de un ISMS. Estos controles abarcan ámbitos como la gestión de accesos, la protección de la información, la seguridad operativa o los aspectos organizativos y humanos relacionados con la seguridad. En su versión más reciente, ISO/IEC 27002 organiza estos controles en diferentes dominios y permite que cada organización seleccione únicamente aquellos que

resulten adecuados en función de su contexto y nivel de riesgo [5].

Un aspecto relevante de estos estándares es que no están diseñados para analizar vulnerabilidades técnicas específicas ni para evaluar la seguridad de una aplicación web concreta. ISO/IEC 27001 y 27002 proporcionan un marco de gestión y gobierno de la seguridad, pero no profundizan en cómo deben diseñarse o implementarse los mecanismos de seguridad a nivel de código, arquitectura de aplicaciones o pruebas técnicas. Por este motivo, aunque resultan fundamentales para establecer una base organizativa sólida, necesitan complementarse con marcos más técnicos cuando el objetivo es analizar vulnerabilidades en aplicaciones web [4, 5].

En el contexto de este Trabajo de Fin de Grado, las normas ISO/IEC 27001 y 27002 se consideran referencias útiles para comprender cómo se gestiona la seguridad a nivel organizativo. No obstante, debido a su carácter generalista, no se adoptan como marco principal para el análisis práctico de vulnerabilidades web, que requiere enfoques más específicos orientados al desarrollo y evaluación de aplicaciones.

### 2.2.3. Esquema Nacional de Seguridad (ENS)

El Esquema Nacional de Seguridad (ENS), regulado por el Real Decreto 311/2022, establece las condiciones mínimas de seguridad que deben cumplir los sistemas de información que prestan servicios electrónicos en el sector público español [6]. A diferencia de estándares como ISO/IEC 27001 o marcos como SSDF, cuya adopción depende de la organización, el ENS tiene carácter obligatorio dentro de su ámbito de aplicación. Esto incluye no solo a las administraciones públicas, sino también a empresas privadas cuando desarrollan o gestionan sistemas que dan soporte a servicios públicos.

El ENS define un conjunto de principios y requisitos orientados a garantizar la confidencialidad, integridad, disponibilidad y trazabilidad de la información. Su enfoque combina la gestión del riesgo con la implantación de medidas técnicas que deben aplicarse en función de la categoría del sistema. No se limita a formular recomendaciones generales, sino que concreta exigencias que influyen directamente en el diseño, configuración y mantenimiento de los sistemas.

Aunque el ENS no está específicamente orientado al análisis de vulnerabilidades web, sí impone obligaciones que afectan de forma directa al desarrollo y operación de aplicaciones. Entre ellas se incluyen la monitorización continua, la identificación y corrección de vulnerabilidades, la actualización periódica del software, la gestión formalizada de incidentes y el registro de actividad. Asimismo, establece la aplicación del principio de mínimo privilegio en el control de accesos y la necesidad de mantener configuraciones seguras, evitando funcionalidades innecesarias que puedan ampliar la superficie de ataque.

Un aspecto especialmente relevante es que el ENS exige integrar la seguridad a lo largo de todo el ciclo de vida del sistema. No contempla la seguridad como una revisión posterior al despliegue, sino como un requisito que debe considerarse desde la concepción del servicio. En este sentido, aunque su naturaleza es normativa y no metodológica, su planteamiento converge con la idea de desarrollo seguro que promueven marcos como SSDF.

En este trabajo, el ENS se incorpora como referencia nacional que permite contextualizar el análisis de vulnerabilidades dentro de un marco regulado. No se pretende realizar una auditoría formal de cumplimiento, sino mostrar que aspectos como la gestión de vulnerabilidades, el control de accesos o el registro de eventos no son únicamente buenas

prácticas técnicas, sino también obligaciones explícitas en determinados entornos reales.

## 2.3. Marcos orientados a la seguridad en aplicaciones web

### 2.3.1. OWASP Application Security Verification Standard (ASVS)

El *OWASP Application Security Verification Standard* (ASVS), en su versión más reciente (5.0.0), es un estándar orientado a la verificación de los controles técnicos de seguridad en aplicaciones web [3]. A diferencia de otros proyectos de OWASP más centrados en la concienciación o en la clasificación de riesgos, ASVS proporciona un conjunto estructurado de requisitos verificables que pueden utilizarse como referencia durante el desarrollo, la validación y la evaluación de la seguridad de una aplicación web.

El objetivo principal de ASVS es reducir la heterogeneidad existente en la forma en que se evalúa la seguridad de las aplicaciones, ofreciendo un estándar abierto y reutilizable que permita establecer un nivel de confianza razonable sobre los controles de seguridad implementados. Para ello, el estándar define requisitos claros que cubren aspectos como la autenticación, la gestión de sesiones, la validación de entradas, la protección frente a inyecciones o la gestión de errores, entre otros, sirviendo tanto como guía para desarrolladores como base para pruebas de verificación técnica.

Una de las características más relevantes de ASVS es su organización en distintos niveles de verificación. El nivel 1 define un conjunto básico de requisitos de seguridad que deberían cumplirse en cualquier aplicación web, orientado a proteger frente a amenazas comunes. El nivel 2 introduce un mayor grado de rigor y está pensado para aplicaciones que manejan información sensible o realizan transacciones relevantes. Por último, el nivel 3 está dirigido a aplicaciones críticas, como las del ámbito financiero, sanitario o gubernamental, e incluye requisitos avanzados que requieren técnicas como el modelado de amenazas, revisiones exhaustivas de código o pruebas de seguridad en profundidad. Esta estructura permite adaptar el esfuerzo de verificación al contexto y al riesgo real de la aplicación, evitando enfoques uniformes que no siempre resultan adecuados.

Es importante destacar que ASVS no es una herramienta de análisis ni una lista de vulnerabilidades, sino un estándar de requisitos técnicos. Su función no es detectar fallos automáticamente, sino definir qué controles de seguridad deben existir y cómo pueden verificarse. En este sentido, ASVS complementa a las herramientas de análisis automático, proporcionando un marco de referencia que ayuda a interpretar los resultados de las pruebas y a evaluar si una aplicación cumple unos mínimos razonables de seguridad.

En el ámbito del desarrollo seguro, ASVS puede utilizarse como apoyo a lo largo del ciclo de vida del software, sirviendo como checklist técnico durante el desarrollo, como referencia en revisiones de código o como base para pruebas de validación antes del despliegue. Además, su alineación con otros marcos y estándares facilita su integración con enfoques organizativos más amplios, permitiendo conectar la verificación técnica con procesos de gestión de la seguridad a mayor escala.

### 2.3.2. OWASP Software Assurance Maturity Model (SAMM)

El *OWASP Software Assurance Maturity Model* (SAMM) es un modelo de madurez orientado a evaluar y mejorar la seguridad del software desde una perspectiva organizativa. A diferencia de otros estándares centrados en requisitos técnicos o en la verificación de aplicaciones concretas, SAMM está diseñado para analizar cómo una organización integra la seguridad en sus procesos de desarrollo y mantenimiento de software a lo largo de todo el ciclo de vida [7].

El objetivo principal de SAMM es proporcionar a las organizaciones una forma estructurada y medible de conocer su nivel actual de madurez en seguridad del software y de definir un plan de mejora progresivo y realista. El modelo no persigue que todas las organizaciones alcancen el máximo nivel de madurez en todas las áreas, sino que promueve una evolución gradual basada en el contexto, el riesgo y las capacidades reales de cada entorno. Esto hace que SAMM sea útil cuando se quiere mejorar progresivamente, sin imponer el máximo nivel en todas las áreas desde el principio.

Desde el punto de vista estructural, SAMM se organiza en cinco grandes funciones de negocio que abarcan las principales áreas implicadas en el desarrollo de software: gobierno de la seguridad, diseño, implementación, verificación y operación. Cada una de estas funciones se descompone en prácticas concretas, que a su vez se estructuran en distintos niveles de madurez. Estos niveles permiten evaluar hasta qué punto las prácticas de seguridad están formalizadas, son repetibles y se encuentran integradas de forma consistente en la organización.

Una diferencia clave respecto a otros marcos como ASVS es que SAMM no define controles técnicos verificables ni requisitos de seguridad a nivel de aplicación. Su foco no está en determinar si una aplicación cumple determinadas medidas técnicas, sino en evaluar si la organización dispone de procesos adecuados para definir, implementar, verificar y mantener la seguridad de forma sistemática. En este sentido, SAMM actúa a un nivel más abstracto, analizando la capacidad de la organización para gestionar la seguridad del software de manera sostenible en el tiempo.

Es importante destacar que SAMM no es una herramienta de análisis ni un checklist técnico, ni está orientado a la detección directa de vulnerabilidades. No realiza pruebas sobre aplicaciones ni produce listados de fallos de seguridad. En la práctica, SAMM sirve para comprobar si la organización tiene procesos claros para gestionar la seguridad.

En el contexto de este Trabajo de Fin de Grado, OWASP SAMM se considera un marco de referencia valioso para comprender la seguridad del software desde una perspectiva organizativa y de proceso. No obstante, debido a que no define requisitos técnicos ni permite analizar vulnerabilidades concretas en aplicaciones web, no se adopta como marco principal para el análisis práctico realizado en este trabajo, sino como complemento conceptual a otros enfoques más técnicos.

## 2.4. Clasificación y priorización de vulnerabilidades

### 2.4.1. OWASP Top 10 (2025)

OWASP Top 10 es un documento de concienciación que recoge, a partir de datos reales y consenso comunitario, las categorías de riesgo más relevantes en aplicaciones web [8].

No se plantea como un estándar técnico ni como una lista exhaustiva de vulnerabilidades, sino como una referencia inicial que ayuda a comprender qué tipos de fallos aparecen con mayor frecuencia y por qué resultan especialmente críticos.

La edición de 2025 se apoya en un volumen considerable de datos procedentes de millones de aplicaciones analizadas y de vulnerabilidades registradas en bases públicas como el NVD. A diferencia de enfoques puramente descriptivos, el documento utiliza un modelo de cálculo explícito que combina tasa de incidencia, cobertura de pruebas, explotabilidad e impacto técnico. Además, junto al análisis estadístico, incorpora categorías promovidas mediante encuesta comunitaria, lo que pone de manifiesto que el Top 10 no es únicamente un ejercicio basado en datos históricos, sino también en la experiencia práctica de la comunidad.

El propio documento reconoce sus limitaciones. No pretende ofrecer una cobertura completa de todas las vulnerabilidades posibles ni puede interpretarse como una garantía de seguridad. Tampoco establece requisitos verificables, como ocurre en ASVS, ni define cómo deben implementarse los controles técnicos. Además, subraya que el riesgo no depende únicamente de la existencia de una vulnerabilidad concreta, sino también del contexto en el que se encuentra la aplicación, su nivel de exposición y el impacto que podría tener un incidente desde el punto de vista organizativo. En este sentido, el Top 10 clasifica categorías de riesgo, no vulnerabilidades individuales.

En este trabajo, OWASP Top 10 se utiliza principalmente como marco de clasificación y contextualización. Las vulnerabilidades incorporadas deliberadamente en la aplicación —como inyecciones, fallos de control de acceso o configuraciones inseguras— pueden relacionarse con categorías como Injection, Broken Access Control o Security Misconfiguration en la edición 2025. Esto permite situar los resultados obtenidos dentro de un marco ampliamente reconocido en el ámbito profesional.

No obstante, dado su carácter general y no prescriptivo, el Top 10 no se emplea como instrumento de evaluación técnica detallada. Para valorar de forma más precisa la implementación de controles de seguridad, resulta necesario complementarlo con marcos más específicos, como ASVS.

## 2.5. Herramientas de análisis de seguridad en aplicaciones web

### 2.5.1. OWASP ZAP y el análisis dinámico de aplicaciones web

OWASP Zed Attack Proxy (ZAP) es una herramienta de análisis dinámico de aplicaciones web (DAST) desarrollada por la OWASP Foundation [9]. Su objetivo es evaluar la seguridad de una aplicación mientras esta se encuentra en ejecución, interactuando con ella del mismo modo que lo haría un usuario o, potencialmente, un atacante.

A diferencia de los estándares y marcos revisados anteriormente, que establecen principios, requisitos o modelos organizativos, ZAP es una herramienta técnica orientada a la detección práctica de vulnerabilidades. En lugar de analizar el código fuente, examina el comportamiento observable de la aplicación, enviando peticiones HTTP y estudiando las respuestas generadas por el servidor.

El funcionamiento de este tipo de herramientas se basa en dos mecanismos principales.

Por un lado, la exploración automática de la aplicación, que permite identificar recursos, formularios y puntos de entrada accesibles desde el exterior. Por otro, el escaneo activo, mediante el cual se envían cargas de prueba diseñadas para comprobar si existen patrones asociados a vulnerabilidades conocidas, como inyecciones, errores de configuración o ausencia de determinadas cabeceras de seguridad.

Entre los problemas que pueden detectarse mediante análisis dinámico se encuentran fallos de validación de entrada, inyecciones, configuraciones inseguras o debilidades básicas en la gestión de sesiones. Sin embargo, este enfoque también presenta limitaciones claras. Al operar desde el exterior del sistema, la herramienta no tiene visibilidad sobre el código ni sobre la lógica interna de la aplicación, lo que dificulta la detección de vulnerabilidades relacionadas con reglas de negocio complejas o controles de acceso mal implementados. Además, los resultados obtenidos pueden incluir falsos positivos que requieren revisión manual.

En el contexto de este Trabajo de Fin de Grado, OWASP ZAP se emplea como herramienta principal para el análisis experimental de la aplicación desarrollada. Su elección responde a su carácter abierto, su amplia adopción en el ámbito profesional y su capacidad para ilustrar de forma práctica cómo se manifiestan determinadas vulnerabilidades en una aplicación web real. No obstante, los resultados obtenidos no se interpretan de forma automática, sino que se analizan críticamente y se relacionan con los marcos teóricos revisados previamente.

Antes de concretar qué marcos se adoptan como referencia principal en este trabajo, resulta útil sintetizar sus características principales. La Tabla 2.1 resume de forma comparativa su ámbito de aplicación, nivel de detalle técnico y el rol que desempeñan en el planteamiento del TFG.

Tabla 2.1: Comparativa de marcos y herramientas analizados

| **Marco / Herramienta** | **Ámbito principal** | **Nivel técnico** | **Rol en este trabajo** |
|---|---|---|---|
| SSDF (NIST) | Metodológico / Desarrollo seguro | Medio | Marco general para integrar la seguridad en el ciclo de vida |
| ISO/IEC 27001–27002 | Gestión organizativa | Bajo | Contextualización del gobierno de seguridad |
| ENS | Normativo (sector público español) | Medio | Referencia nacional y obligaciones aplicables en entornos reales |
| OWASP ASVS | Requisitos técnicos verificables | Alto | Referencia para evaluar controles de seguridad en la aplicación |
| OWASP Top 10 (2025) | Clasificación y priorización de riesgos | Medio | Marco para clasificar y contextualizar las vulnerabilidades |
| OWASP ZAP | Herramienta DAST | Alto | Herramienta para el análisis dinámico en la fase experimental |

## 2.6. Selección de marcos de referencia para este Trabajo de Fin de Grado

Tras revisar los distintos marcos analizados en el estado del arte, resulta necesario concretar qué papel desempeña cada uno dentro del enfoque adoptado en este trabajo. No todos los marcos estudiados tienen el mismo objetivo ni el mismo nivel de aplicación práctica, por lo que su utilización debe justificarse en función del tipo de análisis que se pretende realizar.

El NIST Secure Software Development Framework (SSDF) se toma como referencia metodológica general. Su principal aportación en este contexto es reforzar la idea de que la seguridad debe integrarse desde las fases iniciales del desarrollo, abordando las causas raíz de las vulnerabilidades y no únicamente su corrección una vez detectadas. No obstante, debido a su carácter organizativo y orientado a buenas prácticas amplias, no se emplea como herramienta directa de evaluación técnica.

Las normas ISO/IEC 27001 y 27002, junto con el Esquema Nacional de Seguridad (ENS), permiten contextualizar la seguridad dentro de un marco organizativo y, en el caso del ENS, también normativo. Se incluyen porque ayudan a entender que la seguridad no es solo una cuestión técnica, sino que también depende de decisiones organizativas y de la gestión del riesgo. Sin embargo, estos estándares no ofrecen criterios específicos para analizar vulnerabilidades concretas a nivel de aplicación, por lo que no constituyen el eje central del análisis práctico desarrollado en este trabajo.

En el plano técnico, OWASP Application Security Verification Standard (ASVS) se adopta como marco de apoyo para la definición y evaluación de controles de seguridad. A diferencia de OWASP Top 10, ASVS establece requisitos verificables que permiten analizar de forma estructurada si una aplicación implementa medidas adecuadas frente a amenazas conocidas. Por este motivo, se utiliza como referencia para interpretar los hallazgos obtenidos durante el análisis experimental.

OWASP Top 10 (2025) se emplea principalmente como sistema de clasificación y contextualización de riesgos. Su función en este trabajo no es definir controles ni validar implementaciones, sino situar las vulnerabilidades detectadas dentro de categorías ampliamente reconocidas en el ámbito profesional. Esto facilita conectar el análisis práctico con los riesgos más relevantes identificados a nivel internacional.

Finalmente, OWASP ZAP se selecciona como herramienta principal para la fase experimental. Su elección se justifica por su carácter abierto, su amplia adopción y su capacidad para identificar automáticamente vulnerabilidades representativas en aplicaciones web. En cualquier caso, los resultados obtenidos mediante herramientas automáticas deben revisarse de forma crítica, ya que por sí solos no garantizan un análisis completo de seguridad.

En conjunto, la combinación de estos marcos permite estructurar el trabajo en tres niveles complementarios: un nivel metodológico y organizativo (SSDF, ISO y ENS), un nivel técnico de requisitos y clasificación (ASVS y OWASP Top 10) y un nivel práctico basado en herramientas (OWASP ZAP). Esta estructura permite conectar la parte teórica con el análisis práctico que se desarrolla en los capítulos siguientes.

# Capítulo 3

# Requisitos, especificaciones, coste, riesgos, viabilidad

## 3.1. Descripción del sistema

### 3.1.1. Entorno de pruebas: aplicación vulnerable

La parte práctica de este trabajo se basa en el desarrollo de una aplicación web vulnerable que se utiliza como entorno de pruebas para el análisis de seguridad. La idea no es crear una aplicación completa ni lista para producción, sino tener un caso sencillo sobre el que poder estudiar cómo aparecen ciertas vulnerabilidades y cómo se pueden detectar mediante técnicas de análisis dinámico.

La aplicación planteada es un sistema web básico de gestión de usuarios y comentarios. Incluye funcionalidades como el registro de usuarios, el inicio de sesión, la consulta del perfil, la visualización y publicación de comentarios, una búsqueda sencilla y un panel de administración accesible mediante una ruta concreta. Con esto se consigue tener varios puntos de entrada e interacción, que son justo los que interesan para este tipo de análisis.

A nivel técnico, la aplicación se desarrollará utilizando Flask junto con una base de datos SQLite, y una interfaz web simple basada en HTML y CSS. Se ha optado por estas tecnologías porque permiten trabajar en local de forma cómoda y sin añadir complejidad innecesaria, además de facilitar la introducción controlada de vulnerabilidades.

Precisamente, uno de los puntos clave del trabajo es que la aplicación incluirá vulnerabilidades de forma intencionada. En concreto, se introducirá una inyección SQL en el proceso de autenticación, un XSS reflejado en la funcionalidad de búsqueda, un problema de control de acceso en el panel de administración y una configuración insegura relacionada con cabeceras HTTP. El objetivo de esto no es "romper" la aplicación sin más, sino poder observar cómo se comportan estos fallos y cómo herramientas como OWASP ZAP son capaces de detectarlos. Además, la selección de estas vulnerabilidades no es aleatoria, sino que responde a su relevancia dentro de las categorías más críticas recogidas en OWASP Top 10. Se han elegido aquellas que pueden observarse con facilidad mediante herramientas de análisis dinámico como OWASP ZAP.

La aplicación se ejecutará en local, en un entorno macOS ARM, sin necesidad de desplegarla públicamente ni conectarla con sistemas externos. De esta forma se trabaja siempre en un entorno controlado, adecuado para un contexto académico.

### 3.1.2. Definición del modelo de buenas prácticas

Además del desarrollo de la aplicación vulnerable, una parte importante del trabajo consiste en definir un conjunto de buenas prácticas orientadas al desarrollo seguro de aplicaciones web.

La idea no es únicamente detectar vulnerabilidades, sino entender por qué aparecen y qué decisiones durante el desarrollo han favorecido su aparición. A partir de ahí, se pretende extraer una serie de recomendaciones que ayuden a evitar este tipo de problemas desde el principio.

Para construir este modelo de buenas prácticas, se tomarán como referencia los resultados obtenidos durante el análisis de la aplicación, junto con los marcos OWASP Top 10 y OWASP ASVS. De esta forma, no se trata solo de apoyarse en teoría, sino de combinarla con lo que realmente se ha observado en un caso práctico.

El objetivo final es disponer de una guía sencilla que permita orientar el desarrollo hacia un enfoque más seguro. En este sentido, la intención es ver cómo la aplicación de estas buenas prácticas puede ayudar a evitar la introducción de código inseguro y mejorar la calidad del software desde el punto de vista de la seguridad.

Además, se espera poder identificar qué prácticas concretas (como validar correctamente las entradas, controlar bien los accesos o hacer consultas a la base de datos sin concatenar directamente lo que introduce el usuario) habrían evitado las vulnerabilidades que se han introducido en la aplicación. De esta forma, no solo se obtiene una lista de recomendaciones, sino que también se puede ver de forma bastante clara qué fallos han ocurrido y qué se podría haber hecho para evitarlos.

## 3.2. Requisitos

Estos requisitos se refieren únicamente a la aplicación web utilizada como entorno de pruebas, y no al conjunto completo del trabajo.

En esta sección se recogen los requisitos del sistema propuesto, diferenciando entre requisitos funcionales, que describen las funcionalidades que debe ofrecer la aplicación, y requisitos no funcionales, que establecen las condiciones y restricciones bajo las cuales debe operar.

### 3.2.1. Requisitos funcionales

RF1. El sistema permitirá el registro de nuevos usuarios mediante un formulario web.

RF2. El sistema permitirá a los usuarios autenticarse mediante un mecanismo de inicio de sesión basado en nombre de usuario y contraseña.

RF3. El sistema permitirá a los usuarios autenticados consultar su información de perfil.

RF4. El sistema permitirá visualizar un conjunto de comentarios almacenados en el sistema.

RF5. El sistema permitirá a los usuarios autenticados publicar nuevos comentarios.

RF6. El sistema proporcionará una funcionalidad de búsqueda que permita introducir

texto y visualizar resultados relacionados.

RF7. El sistema incluirá un panel de administración accesible a través de una ruta específica.

RF8. El sistema permitirá la interacción mediante peticiones HTTP que puedan ser analizadas por herramientas de análisis dinámico como OWASP ZAP.

### 3.2.2. Requisitos no funcionales

RNF1. La aplicación se ejecutará en un entorno local, sin exposición a redes públicas.

RNF2. El sistema será compatible con la herramienta OWASP ZAP para la realización de análisis dinámico de seguridad.

RNF3. La aplicación se desarrollará utilizando tecnologías ligeras que permitan su implementación y ejecución en un entorno macOS ARM.

RNF4. El sistema incorporará vulnerabilidades de forma deliberada con fines de análisis académico.

RNF5. La aplicación no incluirá mecanismos avanzados de protección que impidan la detección de vulnerabilidades por herramientas automáticas.

RNF6. El sistema deberá permitir la identificación clara de puntos de entrada (formularios, parámetros, rutas) para facilitar el análisis de seguridad.

RNF7. La aplicación mantendrá una estructura sencilla que facilite la comprensión de su funcionamiento y la trazabilidad de las vulnerabilidades introducidas.

## 3.3. Planificación del trabajo

Para organizar el desarrollo del trabajo, se han definido una serie de tareas concretas que permiten estructurar tanto la construcción de la aplicación como el análisis de seguridad y la definición de buenas prácticas.

Estas tareas se dividen en dos bloques: por un lado, las relacionadas con el desarrollo de la aplicación vulnerable, y por otro, las orientadas al análisis de resultados y la definición de recomendaciones de seguridad.

### 3.3.1. Tareas relacionadas con la aplicación vulnerable

- T1. Diseño de la aplicación web vulnerable, definiendo sus funcionalidades y los puntos donde se introducirán las vulnerabilidades de forma intencionada.
- T2. Implementación de la aplicación utilizando Flask y SQLite, desarrollando las funcionalidades previstas (registro, login, perfil, comentarios, búsqueda y panel de administración).
- T3. Configuración del entorno de análisis mediante OWASP ZAP, incluyendo el uso de la herramienta como proxy para la interceptación de peticiones HTTP.
- T4. Ejecución de pruebas de seguridad sobre la aplicación, utilizando técnicas de exploración automática (spider) y escaneo activo.

- T5. Recopilación y análisis de los resultados obtenidos durante las pruebas, identificando las vulnerabilidades detectadas.

### 3.3.2. Tareas relacionadas con el modelo de buenas prácticas

- T6. Clasificación de las vulnerabilidades identificadas según las categorías de OWASP Top 10.
- T7. Relación de las vulnerabilidades con los controles definidos en OWASP ASVS.
- T8. Análisis de las causas que han provocado la aparición de las vulnerabilidades en la aplicación.
- T9. Definición de un conjunto de buenas prácticas orientadas a evitar este tipo de problemas en el desarrollo de aplicaciones web.

Estas tareas permiten estructurar el trabajo de forma clara y facilitan la validación de los resultados obtenidos, al poder comprobar el grado de cumplimiento de cada una de ellas al finalizar el proyecto.

## 3.4. Costes

Aunque este proyecto se ha desarrollado en un contexto académico y no ha supuesto un coste económico real, sí es posible hacer una estimación aproximada del esfuerzo necesario para llevarlo a cabo, así como del coste que tendría en un entorno profesional.

Para ello, se han identificado las principales tareas del proyecto y se ha estimado el tiempo dedicado a cada una de ellas. Estas estimaciones se han realizado mediante juicio de experto, es decir, basándose en la experiencia propia y en la dificultad de cada fase.

En este caso, todas las tareas han sido asumidas por un único perfil, correspondiente al desarrollador (estudiante).

Sin embargo, si este mismo proyecto se realizara fuera del ámbito académico, lo habitual sería contar con varios perfiles especializados. En concreto, para un trabajo de este tipo podrían intervenir los siguientes:

- Desarrollador de software, encargado de la implementación de la aplicación.
- Analista de seguridad, responsable del análisis de vulnerabilidades y pruebas con OWASP ZAP.
- Jefe de proyecto, encargado de la planificación, seguimiento y coordinación del trabajo.

Para estimar el coste en un entorno profesional, se han tomado como referencia dos fuentes complementarias: por un lado, la actualización de las tablas salariales del XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado, publicada en el BOE [10]; y, por otro, la *Guía del Mercado Laboral 2026* de Hays, concretamente el sector de Tecnologías de la Información [11].

El convenio publicado en el BOE se utiliza como referencia oficial del sector, mientras que la guía de Hays permite contrastar los importes con salarios reales de mercado para perfiles tecnológicos. En este trabajo no se calcula el salario neto de un trabajador, sino un coste profesional aproximado por hora. Por ello, los importes utilizados son superiores al salario bruto prorrateado, ya que tienen en cuenta el coste que supondría contar con estos perfiles en un contexto profesional.

De forma aproximada, se consideran los siguientes costes por hora:

- Desarrollador de software: 30 €/hora
- Analista de seguridad: 40 €/hora
- Jefe de proyecto: 55 €/hora

Aunque en este caso todas estas tareas han sido asumidas por una única persona, esta división permite hacer una estimación más realista del esfuerzo y del coste del proyecto en un contexto profesional.

### 3.4.1. Coste de desarrollo de la aplicación

Las tareas T1 a T5 corresponden al desarrollo de la aplicación vulnerable y a la preparación del entorno de análisis. La estimación de horas por perfil es la siguiente:

| **Tarea** | **Desarrollador** | **Analista** | **Jefe proyecto** | **Total** |
|---|---|---|---|---|
| T1. Diseño de la aplicación | 20 | 0 | 10 | 30 |
| T2. Implementación | 90 | 0 | 0 | 90 |
| T3. Configuración OWASP ZAP | 0 | 20 | 0 | 20 |
| T4. Pruebas de seguridad | 0 | 35 | 0 | 35 |
| T5. Análisis de resultados | 0 | 35 | 0 | 35 |
| **Total** | 110 | 90 | 10 | 210 |

Tabla 3.1: Estimación de esfuerzo para el desarrollo de la aplicación

### 3.4.2. Coste de definición de buenas prácticas

Las tareas T6 a T9 corresponden al análisis de las vulnerabilidades detectadas y a la elaboración del modelo de buenas prácticas. La estimación de horas por perfil es la siguiente:

| **Tarea** | **Desarrollador** | **Analista** | **Jefe proyecto** | **Total** |
|---|---|---|---|---|
| T6. Clasificación OWASP Top 10 | 0 | 15 | 0 | 15 |
| T7. Relación con OWASP ASVS | 0 | 20 | 0 | 20 |
| T8. Análisis de causas | 0 | 20 | 0 | 20 |
| T9. Definición de buenas prácticas | 10 | 20 | 5 | 35 |
| **Total** | 10 | 75 | 5 | 90 |

Tabla 3.2: Estimación de esfuerzo para la definición de buenas prácticas

Estas estimaciones se han ajustado para adaptarse a la distribución por tareas y perfiles definida en este capítulo.

Por tanto, el esfuerzo total estimado del proyecto es de 300 horas. A partir de los costes por hora definidos anteriormente, el coste de personal se calcula de la siguiente forma:

$$(120 \times 30) + (165 \times 40) + (15 \times 55) = 11025\,€$$

### 3.4.3. Costes indirectos

Además del coste de personal, en un entorno profesional también habría que considerar los costes indirectos asociados al desarrollo.

En este caso, se puede considerar el uso de un equipo informático con un coste aproximado de 1200 €, una vida útil estimada de 4 años y un uso del 25 % dedicado al proyecto. Esto supone un coste aproximado de amortización de:

$$1200 \times \frac{1}{4} \times 0{,}25 = 75\,€$$

A este valor habría que añadir otros costes menores, como el consumo eléctrico y la conexión a internet. Para el consumo eléctrico se estima un coste aproximado de 0,15 €/hora durante las 300 horas de trabajo, lo que supone unos 45 €. En cuanto a la conexión a internet, considerando una cuota mensual aproximada de 30 € durante tres meses, se obtiene un coste de 90 €. Además, se aplica un 10 % adicional sobre el coste de personal en concepto de gastos generales de empresa (overhead).

$$11025 \times 0{,}10 = 1102{,}50\,€$$

Por tanto, el coste total estimado del proyecto en un entorno profesional sería:

$$11025 + 75 + 45 + 90 + 1102{,}50 = 12337{,}50\,€$$

De forma redondeada, el coste total del proyecto se estima en unos 12350 €.

## 3.5. Riesgos

A lo largo del desarrollo de este proyecto pueden surgir distintos problemas que afecten tanto a la implementación de la aplicación como al análisis de seguridad. Aunque se trata de un entorno controlado y de un sistema relativamente sencillo, es importante tener en cuenta estos posibles riesgos para poder reaccionar a tiempo si aparecen.

Uno de los principales riesgos tiene que ver con la forma en la que se introducen las vulnerabilidades. Al ser un desarrollo intencionadamente inseguro, puede ocurrir que alguna vulnerabilidad no se implemente correctamente o que no se comporte como se espera. En ese caso, su detección posterior podría complicarse o incluso no llegar a producirse.

Otro punto a tener en cuenta es el uso de OWASP ZAP. Aunque es una herramienta bastante extendida, no siempre detecta todos los fallos presentes en una aplicación. Esto

implica que algunos problemas podrían pasar desapercibidos o que los resultados obtenidos necesiten revisarse con más detalle para interpretarlos correctamente.

También pueden aparecer dificultades a la hora de configurar el entorno de análisis, sobre todo en lo relacionado con el uso de OWASP ZAP como proxy. Si esta configuración no es correcta, el análisis puede no realizarse bien o dar lugar a resultados incompletos.

Por otra parte, al tratarse de una aplicación sencilla, existe el riesgo de que no se cubran todos los casos posibles. Esto hace que el análisis sea útil como ejemplo práctico, pero no necesariamente representativo de todas las situaciones reales que se pueden dar en aplicaciones web más complejas.

Por último, como en cualquier TFG, hay que tener en cuenta el tiempo. Una mala organización o retrasos en alguna fase pueden acabar afectando al resto del trabajo, sobre todo en la parte final, donde se concentran el análisis y la redacción.

Aun así, al trabajar en un entorno controlado, estos riesgos son más fáciles de manejar y, en caso de que aparezcan, se pueden corregir sin que el proyecto se vea demasiado afectado.

## 3.6. Viabilidad

A partir de lo planteado en este trabajo, se puede considerar que el proyecto es viable tanto desde el punto de vista técnico como académico.

En primer lugar, a nivel técnico, el desarrollo de la aplicación no presenta una complejidad elevada. Se basa en tecnologías bastante accesibles como Flask y SQLite, que permiten implementar una aplicación funcional sin necesidad de una infraestructura compleja. Además, el hecho de trabajar en local simplifica mucho la configuración y evita problemas relacionados con despliegues o entornos externos.

Por otro lado, las herramientas utilizadas para el análisis, como OWASP ZAP, están bien documentadas y son relativamente sencillas de utilizar una vez se entiende su funcionamiento básico. Esto facilita que el análisis dinámico se pueda llevar a cabo sin necesidad de contar con recursos adicionales.

En cuanto a la viabilidad económica, el proyecto no requiere inversión, ya que todas las herramientas empleadas son de uso gratuito. Esto hace que el desarrollo sea completamente asumible dentro de un contexto académico.

Respecto a la viabilidad temporal, el proyecto encaja dentro de la dedicación habitual de un Trabajo Fin de Grado. Al tratarse de una aplicación sencilla y de un análisis acotado, es posible completar todas las fases del trabajo dentro del periodo establecido, siempre que se mantenga una planificación razonable.

Por último, desde el punto de vista académico, el proyecto resulta adecuado, ya que combina una parte teórica bien fundamentada con una parte práctica que permite aplicar los conceptos estudiados. Esto facilita extraer conclusiones útiles y plantear propuestas de mejora relacionadas con el desarrollo seguro de aplicaciones web.

En conjunto, se puede concluir que el proyecto es viable y coherente con los objetivos planteados, tanto en alcance como en medios disponibles.

# Capítulo 4

# Análisis

## 4.1. Introducción

Antes de pasar al diseño o a la implementación, conviene dejar claro qué se va a construir y cómo se espera que funcione. En este capítulo se realiza ese análisis inicial, identificando los elementos principales del sistema y la relación que existe entre ellos.

El capítulo se divide en dos partes. En primer lugar, se analiza la aplicación web vulnerable que se utilizará como entorno de pruebas. Para ello se identifican los actores del sistema, los casos de uso, las entidades principales que maneja la aplicación y las clases de análisis que permiten representar su funcionamiento.

En segundo lugar, se define el planteamiento inicial de las pruebas de seguridad con OWASP ZAP. En esta parte se define qué zonas de la aplicación se van a analizar, qué puntos de entrada resultan más relevantes y qué vulnerabilidades se espera estudiar durante el desarrollo del trabajo.

En este capítulo no se incluyen todavía resultados ni detalles de implementación. Su objetivo es servir como base para el diseño del capítulo siguiente y para la ejecución de las pruebas que se recogerán más adelante.

## 4.2. Análisis de la aplicación web

### 4.2.1. Actores del sistema

La aplicación contempla tres tipos principales de usuario. Además, durante la fase de análisis se tendrá en cuenta OWASP ZAP como herramienta externa, ya que interactuará con la aplicación para explorar sus rutas y detectar posibles vulnerabilidades.

- **Usuario no autenticado.** Representa a cualquier persona que accede a la aplicación sin haber iniciado sesión. Puede registrarse o iniciar sesión, pero no debería acceder al resto de funcionalidades.

- **Usuario autenticado.** Representa al usuario que ya ha iniciado sesión correctamente. Puede consultar su perfil, ver comentarios, publicar nuevos comentarios y utilizar la funcionalidad de búsqueda.

- **Administrador.** Representa al usuario que debería tener acceso al panel de administración. En este trabajo, esta parte resulta especialmente relevante porque se introduce de forma deliberada un fallo de control de acceso en dicha funcionalidad.

- **OWASP ZAP.** No se trata de un usuario de la aplicación, sino de una herramienta externa de análisis. Se utilizará como proxy entre el navegador y la aplicación para interceptar peticiones, recorrer rutas y lanzar pruebas de seguridad.

### 4.2.2. Casos de uso

A partir de los usuarios identificados anteriormente se establecen los principales casos de uso de la aplicación.

- **CU-01. Registrarse.** El usuario no autenticado introduce un nombre de usuario y una contraseña para crear una cuenta en el sistema.

- **CU-02. Iniciar sesión.** El usuario no autenticado introduce sus credenciales para acceder a la aplicación. Este caso de uso es especialmente importante, ya que en el proceso de autenticación se introduce de forma deliberada una vulnerabilidad de inyección SQL.

- **CU-03. Consultar perfil.** El usuario autenticado accede a una vista con la información asociada a su cuenta.

- **CU-04. Visualizar comentarios.** El usuario autenticado consulta los comentarios almacenados en la aplicación.

- **CU-05. Publicar comentario.** El usuario autenticado añade un nuevo comentario, que queda almacenado y visible dentro del sistema.

- **CU-06. Realizar búsqueda.** El usuario autenticado introduce un término de búsqueda. Esta funcionalidad será relevante en el análisis porque el texto introducido se refleja en la respuesta del servidor.

- **CU-07. Acceder al panel de administración.** El administrador debería acceder al panel de administración. Sin embargo, al introducirse un fallo de control de acceso, esta ruta podrá ser solicitada por usuarios que no deberían tener permisos.

En la figura 4.1 se muestra el diagrama de casos de uso del sistema.

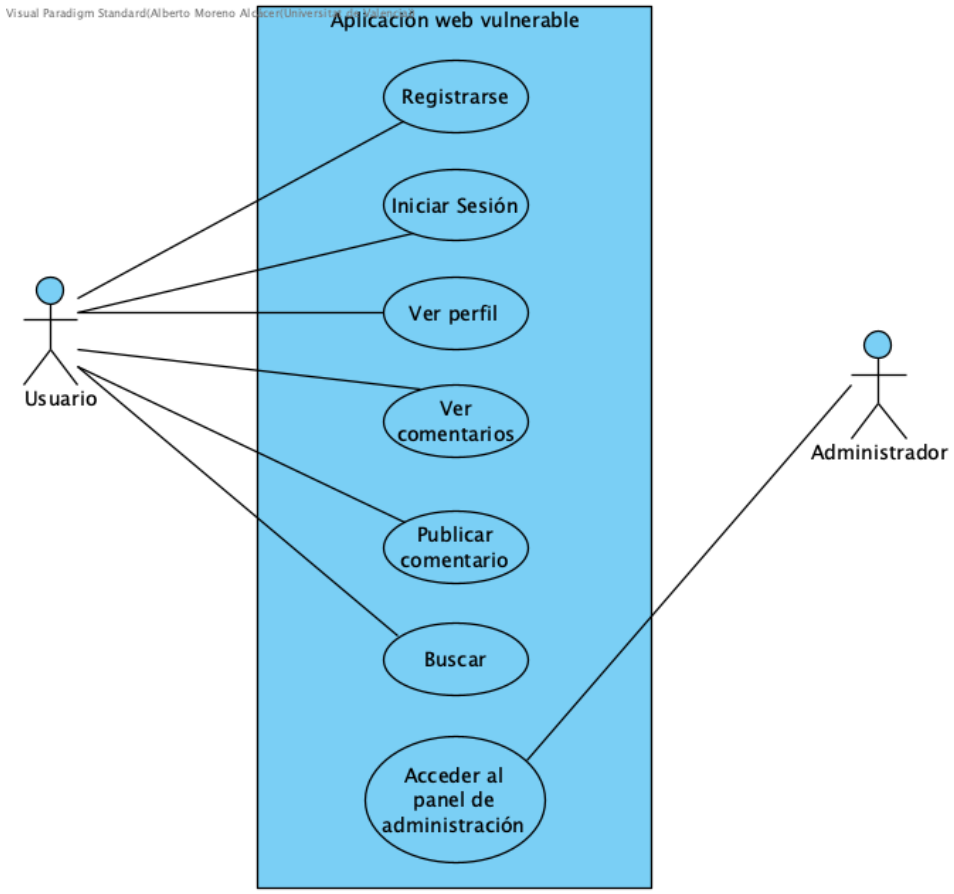



Figura 4.1: Diagrama de casos de uso del sistema

### 4.2.3. Descripción de los casos de uso

A continuación se describe cada caso de uso con más detalle, indicando el actor que lo inicia, las precondiciones necesarias y el flujo básico de la interacción.

#### CU-01. Registrarse

- **Actor:** Usuario no autenticado.
- **Precondición:** El usuario no tiene cuenta en el sistema.
- **Descripción:** El usuario accede al formulario de registro, introduce un nombre de usuario y una contraseña, y envía el formulario. Si los datos son válidos, se crea la cuenta y el usuario puede iniciar sesión.
- **Postcondición:** Existe un nuevo registro de usuario en la base de datos.

#### CU-02. Iniciar sesión

- **Actor:** Usuario no autenticado.
- **Precondición:** El usuario tiene una cuenta registrada en el sistema.

- **Descripción:** El usuario introduce sus credenciales en el formulario de login. La aplicación consulta la base de datos para verificarlas. La consulta SQL se construye concatenando directamente los valores introducidos por el usuario, sin comprobarlos ni tratarlos previamente, lo que introduce de forma deliberada una vulnerabilidad de inyección SQL.
- **Postcondición:** Si las credenciales son correctas, o si se explota la vulnerabilidad, el usuario accede a la aplicación con sesión iniciada.

**CU-03. Consultar perfil**

- **Actor:** Usuario autenticado.
- **Precondición:** El usuario ha iniciado sesión.
- **Descripción:** El usuario accede a su página de perfil, donde se muestra la información asociada a su cuenta almacenada en el sistema.
- **Postcondición:** Se muestra la información del perfil del usuario.

**CU-04. Visualizar comentarios**

- **Actor:** Usuario autenticado.
- **Precondición:** El usuario ha iniciado sesión.
- **Descripción:** El usuario accede a la sección de comentarios, donde se listan los que están almacenados en la base de datos.
- **Postcondición:** Se muestran los comentarios disponibles en el sistema.

**CU-05. Publicar comentario**

- **Actor:** Usuario autenticado.
- **Precondición:** El usuario ha iniciado sesión.
- **Descripción:** El usuario escribe un comentario y lo envía. El texto se almacena en la base de datos y pasa a ser visible para el resto de usuarios.
- **Postcondición:** El comentario queda guardado y aparece en la lista de comentarios.

**CU-06. Realizar búsqueda**

- **Actor:** Usuario autenticado.
- **Precondición:** El usuario ha iniciado sesión.

- **Descripción:** El usuario introduce un término en el campo de búsqueda y envía la consulta. La aplicación devuelve una respuesta que incluye directamente el texto introducido por el usuario, sin aplicar ningún tratamiento previo. Esto constituye el punto de entrada para una vulnerabilidad de Cross-Site Scripting reflejado (XSS reflejado) introducida en la aplicación.
- **Postcondición:** Se muestra una página con los resultados y el término introducido reflejado en la respuesta.

**CU-07. Acceder al panel de administración**

- **Actor:** Administrador, o cualquier usuario que conozca la ruta.
- **Precondición:** Ninguna en la práctica, dado que no hay control de acceso efectivo.
- **Descripción:** Cualquier usuario que acceda a la ruta específica del panel de administración puede visualizar su contenido. La aplicación no comprueba si el usuario tiene permisos para ello, lo que representa el fallo de control de acceso introducido de forma deliberada.
- **Postcondición:** Se muestra el panel de administración con independencia del rol del usuario que ha realizado la petición.

### 4.2.4. Modelo conceptual de datos

El modelo conceptual de datos recoge las entidades principales que maneja la aplicación y las relaciones entre ellas. En este nivel de análisis no se detallan todavía tablas, claves o tipos de datos concretos, ya que esos aspectos pertenecen al diseño.

Las entidades identificadas son las siguientes:

- **Usuario.** Representa a una persona registrada en el sistema. Tiene asociado un nombre de usuario, una contraseña y un rol, que permite diferenciar entre un usuario normal y un administrador.
- **Comentario.** Representa un mensaje publicado dentro de la aplicación. Cada comentario está asociado al usuario que lo ha creado.
- **Sesión.** Representa el estado de autenticación de un usuario mientras utiliza la aplicación. Permite identificar qué usuario ha iniciado sesión en un momento determinado.

Las relaciones entre estas entidades son las siguientes:

- Un **Usuario** puede publicar varios **Comentarios**, mientras que cada comentario pertenece a un único usuario.
- Un **Usuario** puede tener asociada una **Sesión** activa durante el uso de la aplicación.

En la figura 4.2 se muestra el diagrama con las entidades y relaciones descritas.

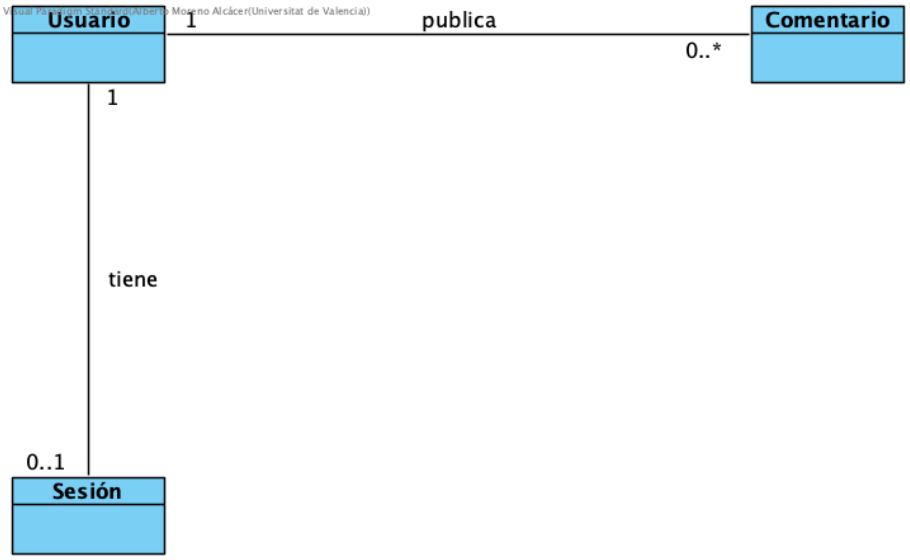



Figura 4.2: Modelo conceptual de datos

### 4.2.5. Diagrama de clases de análisis

El diagrama de clases de análisis representa la estructura lógica inicial del sistema. No busca reflejar todavía la implementación final, sino identificar los componentes principales de la aplicación y las responsabilidades que tendrá cada uno.

Las clases identificadas son las siguientes:

- **AplicacionWeb.** Actúa como componente central de la aplicación y coordina el acceso al resto de funcionalidades.
- **GestorAutenticacion.** Se encarga del registro y del inicio de sesión de los usuarios. Es una clase relevante para el análisis, ya que en esta parte se introduce deliberadamente la vulnerabilidad de inyección SQL.
- **GestorComentarios.** Gestiona la visualización y publicación de comentarios. Se relaciona tanto con los usuarios como con los comentarios almacenados.
- **Buscador.** Gestiona la funcionalidad de búsqueda. En este componente se refleja el texto introducido por el usuario sin aplicar un tratamiento adecuado, lo que permite estudiar una vulnerabilidad de XSS reflejado.
- **PanelAdministracion.** Representa el acceso al panel de administración. En este caso se mantiene deliberadamente una comprobación de permisos incorrecta para poder analizar un fallo de control de acceso.
- **Usuario.** Representa la información básica de cada usuario registrado, incluyendo su rol dentro del sistema.
- **Comentario.** Representa los comentarios publicados por los usuarios.

En la figura 4.3 se muestra el diagrama de clases de análisis del sistema.

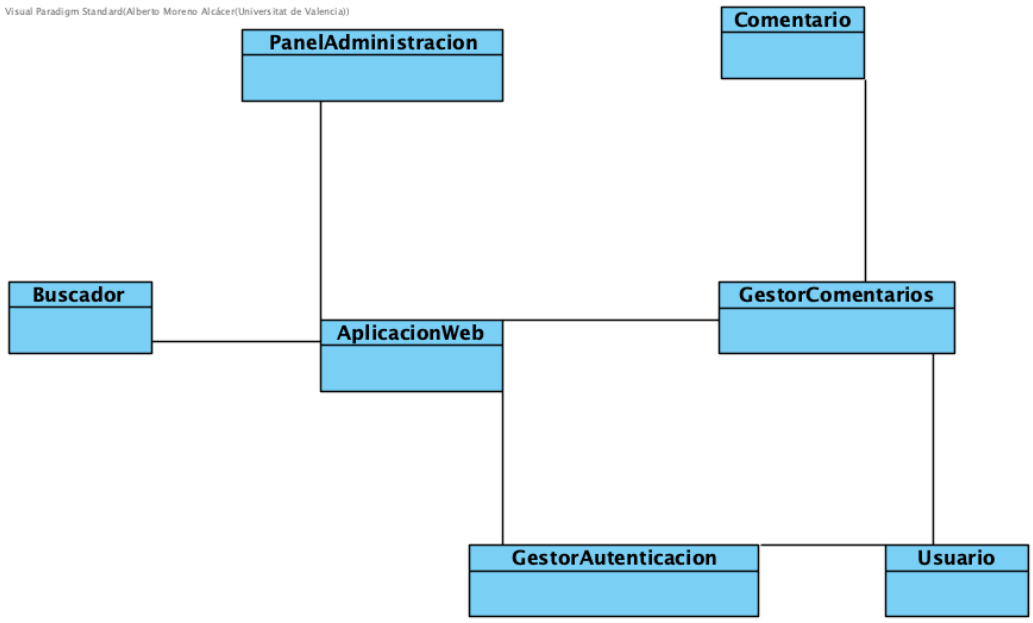



Figura 4.3: Diagrama de clases de análisis

## 4.3. Análisis de las pruebas de seguridad

### 4.3.1. Objetivo de las pruebas con OWASP ZAP

El objetivo de esta parte es definir cómo se va a utilizar OWASP ZAP para analizar la aplicación vulnerable. No se trata únicamente de ejecutar un escaneo automático, sino de preparar una prueba que permita comprobar qué vulnerabilidades puede detectar la herramienta y cuáles requieren una revisión manual.

Con este planteamiento, la aplicación se analiza desde el exterior, observando cómo responde ante distintas peticiones HTTP. También permite valorar las limitaciones de una herramienta de análisis dinámico, especialmente en fallos relacionados con la lógica de la aplicación o con el control de acceso.

Los resultados obtenidos durante la ejecución de estas pruebas se analizarán en el capítulo 6 y servirán posteriormente como base para la propuesta de buenas prácticas del capítulo 7.

### 4.3.2. Alcance de las pruebas

Las pruebas se realizarán sobre la aplicación web vulnerable ejecutándose en local, en un entorno macOS ARM. No se va a analizar ningún sistema externo ni se va a exponer la aplicación a ninguna red pública.

El análisis se centrará en las rutas y funcionalidades de la propia aplicación, cubriendo todos los puntos de entrada previstos: formularios, parámetros de URL y rutas accesibles. Quedan fuera del alcance cualquier sistema operativo, infraestructura de red o componente externo a la propia aplicación.

### 4.3.3. Puntos de entrada a analizar

Los puntos de entrada son aquellas zonas de la aplicación en las que se reciben datos o peticiones desde el exterior. Identificarlos antes del análisis permite orientar mejor el escaneo y saber dónde buscar.

Los puntos de entrada que se van a analizar son los siguientes:

- `/register`: formulario de registro con campos de nombre de usuario y contraseña.
- `/login`: formulario de autenticación, que es donde está la inyección SQL.
- `/profile`: vista del perfil del usuario autenticado.
- `/comments`: página de visualización y publicación de comentarios.
- `/search?q=`: campo de búsqueda con el parámetro `q` reflejado en la respuesta, que es el punto de entrada del XSS reflejado.
- `/admin`: ruta del panel de administración, accesible sin ningún control de acceso.

### 4.3.4. Vulnerabilidades previstas en el análisis

Como las vulnerabilidades se han introducido de forma deliberada, se parte de una idea previa de los fallos que deberían aparecer durante el análisis. Esto permite comprobar hasta qué punto OWASP ZAP es capaz de detectarlos y qué aspectos requieren una revisión manual.

Las vulnerabilidades previstas son las siguientes:

- **Inyección SQL en el formulario de login.** La consulta a la base de datos se construye concatenando directamente los valores introducidos por el usuario, sin utilizar consultas parametrizadas. Esta vulnerabilidad se relaciona con la categoría *Injection* de OWASP Top 10.
- **XSS reflejado en el buscador.** El término de búsqueda se devuelve en la respuesta sin escaparse correctamente, lo que permite inyectar código JavaScript en la página. Este fallo también se relaciona con la categoría *Injection* de OWASP Top 10.
- **Control de acceso roto en el panel de administración.** La ruta `/admin` puede ser solicitada por usuarios que no deberían tener permisos para acceder a ella. Esta vulnerabilidad se relaciona con la categoría *Broken Access Control*.
- **Cabeceras de seguridad ausentes.** La aplicación no incluye determinadas cabeceras HTTP de seguridad, como `Content-Security-Policy` o `X-Content-Type-Options`. Este problema se relaciona con la categoría *Security Misconfiguration*.

### 4.3.5. Configuración general de OWASP ZAP

Para llevar a cabo el análisis, OWASP ZAP se configurará como proxy entre el navegador y la aplicación. Esto permite que toda la comunicación HTTP pase por ZAP, que la registra y la analiza.

La configuración prevista es la siguiente:

- ZAP actuará como proxy local, con el navegador configurado para redirigir el tráfico a través de él.
- Se usará el spider tradicional de ZAP para explorar las rutas accesibles de la aplicación de forma automática.
- Una vez completada la exploración, se lanzará el escaneo activo, que envía cargas de prueba sobre los formularios y parámetros encontrados para detectar vulnerabilidades.
- Los resultados se revisarán manualmente para descartar falsos positivos y entender mejor cada alerta generada.

En este capítulo no se ejecuta todavía nada. Todo lo descrito aquí es la planificación del análisis; la ejecución real y los resultados se recogen en el capítulo 6.

## 4.4. Relación entre el análisis del sistema y las pruebas de seguridad

Existe una relación directa entre los casos de uso definidos y las vulnerabilidades que se van a analizar. Las vulnerabilidades no se han situado de forma aleatoria, sino en funcionalidades concretas de la aplicación para que el análisis tenga un sentido práctico.

El formulario de inicio de sesión, correspondiente al caso de uso CU-02, se utilizará para analizar la inyección SQL. La funcionalidad de búsqueda, definida en el caso de uso CU-06, permitirá estudiar el XSS reflejado. El acceso al panel de administración, recogido en el caso de uso CU-07, servirá para observar el fallo de control de acceso. Por último, la ausencia de determinadas cabeceras de seguridad afectará al comportamiento general de la aplicación.

Esta relación entre funcionalidades y vulnerabilidades permite que las pruebas con OWASP ZAP no sean un análisis aislado, sino una forma de comprobar cómo determinados errores de diseño o implementación pueden manifestarse en una aplicación web concreta. A partir de este análisis se podrán extraer conclusiones útiles para los capítulos posteriores, especialmente para la definición de buenas prácticas de desarrollo seguro.

# Capítulo 5

# Diseño

## 5.1. Introducción

En el capítulo anterior se ha definido el análisis de la aplicación web vulnerable y de las pruebas de seguridad que se realizarán sobre ella. A partir de ese análisis, en este capítulo se concreta el diseño del sistema, es decir, cómo se organizará internamente la aplicación, qué componentes la formarán, cómo se estructurará la base de datos y de qué forma se prepararán las vulnerabilidades que serán estudiadas posteriormente.

El objetivo de este capítulo no es mostrar todavía la implementación final ni los resultados de las pruebas, sino dejar definido el diseño que servirá de base para construir la aplicación y ejecutar el análisis con OWASP ZAP. Por este motivo, se describen las decisiones principales de diseño sin entrar en fragmentos concretos de código ni en capturas de ejecución. Esos detalles se recogerán más adelante, en el capítulo dedicado a la implementación y las pruebas.

La aplicación se plantea como un entorno de pruebas sencillo, desarrollado con Flask y SQLite, que permite reproducir varias vulnerabilidades web de forma controlada. No se busca construir una aplicación completa para un entorno real, sino disponer de una base suficientemente representativa para analizar problemas habituales como la inyección SQL, el XSS reflejado, el control de acceso roto y la ausencia de determinadas cabeceras de seguridad.

Además del diseño de la aplicación, también se concreta cómo se van a particularizar las pruebas con OWASP ZAP. En el análisis anterior ya se identificaron los puntos de entrada principales de la aplicación, como el formulario de inicio de sesión, el buscador o la ruta del panel de administración. En este capítulo se avanza un paso más y se define cómo se relacionará cada punto de entrada con la vulnerabilidad que se pretende estudiar y con el tipo de prueba que se aplicará.

También se define qué se hará con la información obtenida durante las pruebas. Los resultados no se usarán solo como una lista de alertas, sino como una base para analizar la causa de cada vulnerabilidad y relacionarla con las buenas prácticas que se propondrán más adelante. Con esto, el diseño de las pruebas no queda aislado, sino que sirve como base para el marco de programación seguro que se planteará en el capítulo 7.

Así, el diseño sirve como enlace entre el análisis realizado en el capítulo anterior y la implementación que se desarrollará en el capítulo siguiente. Primero se define la estructura de la aplicación y de sus datos, después se explica cómo se introducen las vulnerabilidades

de forma intencionada y, por último, se diseña la actividad de test que permitirá evaluarlas posteriormente.

## 5.2. Diseño de la aplicación web

Después del análisis del sistema, el siguiente paso es definir cómo se va a organizar la aplicación desde el punto de vista del diseño. En esta sección se describe la arquitectura general, los componentes principales, el diagrama de clases de diseño y el modelo lógico de datos.

La aplicación se ha diseñado como un sistema sencillo. Su finalidad no es ofrecer un servicio completo ni preparado para un entorno real, sino servir como base para analizar vulnerabilidades web en un entorno controlado. Por ello, se prioriza una estructura clara y fácil de seguir, que permita relacionar cada funcionalidad con los puntos de entrada definidos en el capítulo anterior y con las vulnerabilidades que se introducirán más adelante.

### 5.2.1. Arquitectura general de la aplicación

La aplicación se diseña como una aplicación web local desarrollada con Flask. El usuario interactúa con ella a través de un navegador web, mientras que Flask recibe las peticiones HTTP, las procesa y devuelve las respuestas correspondientes. Para almacenar la información se utiliza SQLite, ya que permite trabajar con una base de datos relacional sencilla sin tener que instalar ni configurar un servidor de base de datos independiente.

La arquitectura propuesta se compone de los siguientes elementos principales:

- **Navegador web**: actúa como cliente de la aplicación. Desde él se accede a los formularios de registro, inicio de sesión, búsqueda, comentarios y panel de administración.
- **Aplicación Flask**: contiene la lógica principal del sistema. Define las rutas de la aplicación, procesa los formularios, gestiona la sesión del usuario y coordina el acceso a la base de datos.
- **Base de datos SQLite**: almacena la información persistente de la aplicación, principalmente los usuarios registrados y los comentarios publicados.
- **OWASP ZAP**: se utilizará durante la fase de pruebas como proxy entre el navegador y la aplicación. No forma parte de la lógica interna del sistema, sino que actúa como herramienta externa para interceptar peticiones, explorar rutas y lanzar pruebas de seguridad.

La aplicación se ejecutará en local, por lo que no se contempla un despliegue público ni la conexión con servicios externos. Esta decisión permite trabajar en un entorno controlado y reduce la complejidad de la infraestructura. Además, facilita que el análisis se centre en las vulnerabilidades de la propia aplicación y no en aspectos externos como la configuración de un servidor web, certificados TLS o reglas de red.

Desde el punto de vista lógico, el funcionamiento general será el siguiente: el usuario realizará una petición desde el navegador, la aplicación Flask recibirá dicha petición,

procesará la ruta correspondiente, accederá a la base de datos si es necesario y devolverá una respuesta HTML. Durante las pruebas, OWASP ZAP se situará entre el navegador y la aplicación, registrando el tráfico HTTP generado.

Esta separación entre la aplicación y la herramienta de análisis es importante. OWASP ZAP no forma parte del sistema desarrollado, sino del entorno de pruebas. Por tanto, en el diseño se considera como un elemento externo que permitirá observar el comportamiento de la aplicación desde fuera.

### 5.2.2. Diseño de componentes

La aplicación se organiza en varios componentes funcionales, cada uno con una responsabilidad concreta. Aunque en la implementación final estos componentes puedan materializarse mediante rutas, funciones o módulos de Flask, en el diseño se describen por separado para facilitar la comprensión del sistema.

Los componentes principales son los siguientes:

- **Aplicación principal Flask**: es el punto de entrada del sistema. Se encarga de inicializar la aplicación, definir las rutas disponibles y coordinar la interacción entre el resto de componentes.
- **Componente de autenticación**: gestiona el registro de usuarios, el inicio de sesión y el cierre de sesión. También se encarga de guardar en la sesión la información mínima necesaria para identificar al usuario autenticado.
- **Componente de perfil**: permite mostrar la información básica del usuario que ha iniciado sesión. Representa una funcionalidad privada que solo debería estar disponible para usuarios autenticados.
- **Componente de comentarios**: permite visualizar los comentarios almacenados y publicar nuevos comentarios. Este componente se comunica con la base de datos para recuperar y guardar la información asociada a cada comentario.
- **Componente de búsqueda**: recibe un término introducido por el usuario y genera una página de respuesta en la que se muestra dicho término. Este componente es relevante para el estudio del XSS reflejado, ya que en él se diseñará el reflejo del parámetro de búsqueda en la respuesta HTML.
- **Componente de administración**: representa el panel de administración de la aplicación. En una aplicación segura, este componente debería comprobar que el usuario autenticado dispone del rol adecuado antes de permitir el acceso. Sin embargo, en este trabajo se diseñará con un control de permisos incorrecto para poder estudiar un fallo de control de acceso.
- **Componente de acceso a datos**: agrupa las operaciones de lectura y escritura sobre la base de datos SQLite. Se encarga de crear usuarios, buscar credenciales, recuperar comentarios y guardar nuevas publicaciones.

Esta división permite mantener una relación clara entre las funcionalidades analizadas en el capítulo anterior y los elementos que se diseñan para implementarlas. Además, ayuda a localizar dónde se introducirá cada vulnerabilidad: la inyección SQL se situará en

el componente de autenticación, el XSS reflejado en el componente de búsqueda, el fallo de control de acceso en el componente de administración y la configuración insegura de cabeceras HTTP en la configuración general de la aplicación.

El diseño no pretende ser más complejo de lo necesario. Al tratarse de una aplicación pequeña, no se plantea una arquitectura por capas extensa ni el uso de patrones avanzados. Aun así, separar responsabilidades permite que el sistema sea más fácil de explicar, probar y relacionar con los objetivos del trabajo.

### 5.2.3. Diseño de la interfaz de usuario

La interfaz de la aplicación se ha diseñado de forma sencilla, ya que el objetivo principal del sistema no es ofrecer una experiencia de usuario avanzada, sino facilitar la interacción con las funcionalidades que se van a analizar posteriormente. Por este motivo, se ha optado por una estructura clara, con una barra de navegación superior y una zona central donde se muestra el contenido de cada página.

Antes de implementar la interfaz final, se planteó una maqueta sencilla para definir la disposición general de las pantallas. Esta maqueta no busca representar el diseño visual definitivo, sino mostrar cómo se organizarían los elementos principales de la aplicación: cabecera, barra de navegación, zona de contenido y formularios o tablas según la funcionalidad de cada página.

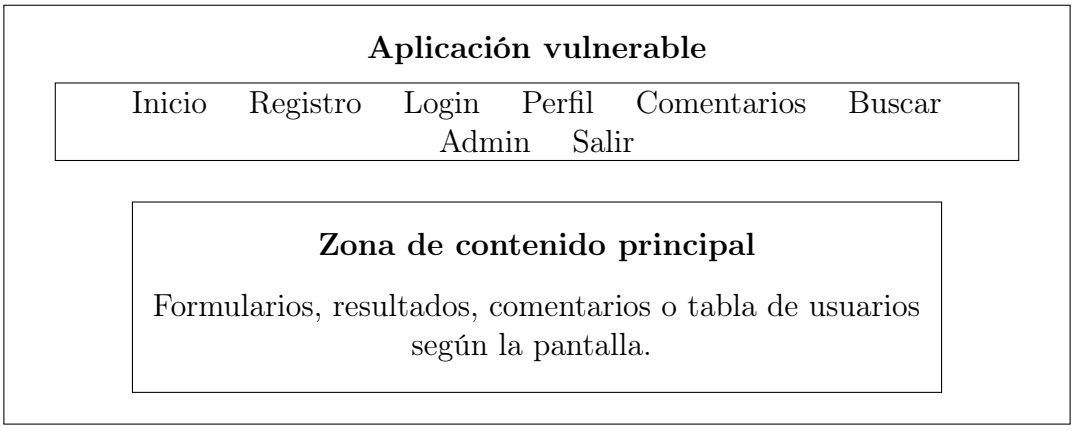



Figura 5.1: Maqueta general de la interfaz de la aplicación

Como se observa en la Figura 5.1, todas las pantallas siguen una misma estructura. La parte superior contiene el título de la aplicación y la navegación principal, mientras que la zona central se reserva para el contenido específico de cada funcionalidad. Esta disposición permite mantener una interfaz simple y coherente durante toda la aplicación.

La barra de navegación permite acceder a las principales funcionalidades de la aplicación: inicio, registro, inicio de sesión, perfil, comentarios, búsqueda, panel de administración y cierre de sesión. Esta organización permite recorrer la aplicación de forma rápida durante las pruebas y facilita que OWASP ZAP identifique las rutas principales durante la exploración.

Las pantallas principales previstas en la interfaz son las siguientes:

- **Página de inicio**: presenta brevemente la finalidad de la aplicación como entorno local de pruebas.

- **Registro**: incluye un formulario básico con nombre de usuario y contraseña para crear nuevos usuarios.
- **Inicio de sesión**: permite introducir credenciales y acceder a la aplicación. Esta pantalla será relevante para la prueba de inyección SQL.
- **Perfil**: muestra la información básica del usuario autenticado, como el nombre de usuario y el rol.
- **Comentarios**: permite visualizar comentarios existentes y publicar nuevos mensajes cuando hay una sesión iniciada.
- **Búsqueda**: incluye un formulario que recibe un término mediante el parámetro `q`. Esta pantalla será utilizada para analizar el XSS reflejado.
- **Panel de administración**: muestra una tabla con los usuarios registrados. En una aplicación segura debería estar restringido a usuarios administradores, pero en este trabajo se mantiene accesible de forma incorrecta para estudiar un fallo de control de acceso.

Desde el punto de vista visual, la interfaz se mantiene deliberadamente simple. Se utiliza una cabecera común para todas las páginas, una zona de contenido centrada y formularios básicos para las funcionalidades que requieren entrada del usuario. Esta decisión permite que las pruebas se centren en el comportamiento de la aplicación y no en elementos visuales complejos que no aportan valor al análisis de seguridad.

Además, mantener una interfaz uniforme facilita la toma de evidencias durante la fase de implementación y pruebas. Las capturas de pantalla permiten comprobar que las funcionalidades diseñadas se han implementado correctamente y ayudan a relacionar cada pantalla con la vulnerabilidad que se analiza en el capítulo siguiente.

### 5.2.4. Diagrama de clases de diseño

El diagrama de clases de diseño concreta la estructura planteada en el análisis, pero con elementos más cercanos a la futura implementación. En el capítulo anterior se identificaron clases generales, como el gestor de autenticación, el gestor de comentarios o el buscador. En esta fase, esas responsabilidades se transforman en componentes más próximos a una aplicación Flask organizada mediante controladores, repositorios y clases de datos.

Como la aplicación es pequeña, no se ha planteado una jerarquía de clases compleja. Se ha optado por una estructura sencilla, suficiente para representar las funcionalidades principales y, al mismo tiempo, fácil de seguir. La idea es separar la parte que recibe y procesa las peticiones HTTP de la parte que accede a la base de datos.

Las clases y componentes principales del diseño son los siguientes:

- **AplicacionFlask**: representa la aplicación principal. Inicializa Flask, registra las rutas disponibles y coordina el acceso a los distintos controladores.
- **AuthController**: gestiona las operaciones relacionadas con el registro, el inicio de sesión y el cierre de sesión. En este componente se situará la lógica asociada al formulario de autenticación.

- **ProfileController**: gestiona la visualización de la información del usuario autenticado. Aunque su funcionalidad es sencilla, permite representar una zona privada de la aplicación.
- **CommentController**: gestiona la visualización y publicación de comentarios. Para ello se comunica con el repositorio de comentarios.
- **SearchController**: gestiona la funcionalidad de búsqueda. Recibe el parámetro introducido por el usuario y genera la respuesta correspondiente.
- **AdminController**: gestiona el acceso al panel de administración. En una aplicación segura debería comprobar correctamente el rol del usuario antes de mostrar el contenido del panel.
- **UserRepository**: agrupa las operaciones de acceso a datos relacionadas con los usuarios, como crear una cuenta o buscar credenciales durante el inicio de sesión.
- **CommentRepository**: agrupa las operaciones de acceso a datos relacionadas con los comentarios, como recuperar el listado de comentarios o insertar uno nuevo.
- **DatabaseManager**: gestiona la conexión con la base de datos SQLite y proporciona el acceso necesario para ejecutar las operaciones sobre ella.
- **Usuario**: representa a un usuario registrado en la aplicación. Incluye atributos como el identificador, el nombre de usuario, la contraseña y el rol.
- **Comentario**: representa un comentario publicado por un usuario. Incluye el identificador del comentario, el identificador del usuario que lo ha creado, el contenido y la fecha de creación.

En este diseño, los controladores actúan como intermediarios entre las peticiones HTTP y las operaciones de la aplicación. Los repositorios concentran el acceso a la base de datos, de forma que las consultas no queden mezcladas con la lógica de las rutas. Esta separación no pretende convertir la aplicación en un sistema complejo, sino hacer que cada parte tenga una responsabilidad clara.

La relación entre las clases principales puede resumirse de la siguiente manera: `AplicacionFlask` registra y utiliza los distintos controladores; los controladores que necesitan datos persistentes se apoyan en los repositorios; y los repositorios utilizan `DatabaseManager` para comunicarse con SQLite. Además, los repositorios trabajan con las clases de datos `Usuario` y `Comentario`, que representan la información principal manejada por el sistema.

Para hacer más clara la relación entre el análisis y el diseño, en la Tabla 5.1 se muestra la trazabilidad entre las clases identificadas en el diagrama de análisis y las clases o componentes definidos en el diseño. En algunos casos la correspondencia es directa, mientras que en otros una clase de análisis se divide en varios elementos de diseño para separar mejor las responsabilidades.

| Clase de análisis | Clase o componente de diseño | Justificación |
|---|---|---|
| `AplicacionWeb` | `AplicacionFlask` | La clase general de análisis se concreta en la aplicación Flask, que inicializa el sistema, registra las rutas y coordina el acceso a los distintos controladores. |
| `GestorAutenticacion` | `AuthController`, `UserRepository`, `Usuario` | La autenticación se divide en un controlador para gestionar las peticiones, un repositorio para acceder a los datos de usuario y la clase `Usuario` para representar la información almacenada. |
| `GestorComentarios` | `CommentController`, `CommentRepository`, `Comentario` | La gestión de comentarios se separa entre la parte que atiende las rutas, la parte que consulta o inserta comentarios en la base de datos y la clase que representa cada comentario. |
| `Buscador` | `SearchController` | La funcionalidad de búsqueda se transforma en un controlador específico encargado de recibir el parámetro introducido por el usuario y generar la respuesta correspondiente. |
| `PanelAdministracion` | `AdminController` | El acceso al panel de administración se concreta en un controlador propio, donde se centraliza la lógica asociada a esta ruta. |
| `Usuario` | `Usuario`, `UserRepository` | La entidad de análisis se mantiene como clase de datos, pero en el diseño aparece también un repositorio para separar la representación del usuario de las operaciones de acceso a la base de datos. |
| `Comentario` | `Comentario`, `CommentRepository` | La entidad de análisis se mantiene como clase de datos y se complementa con un repositorio encargado de recuperar e insertar comentarios en SQLite. |
| No aparece como clase propia en análisis | `ProfileController` | Se añade en el diseño para separar la visualización del perfil del resto de funcionalidades, aunque en el análisis esta responsabilidad estaba incluida dentro del comportamiento general del usuario autenticado. |
| No aparece como clase propia en análisis | `DatabaseManager` | Se introduce en el diseño para centralizar la conexión con SQLite y evitar que cada controlador o repositorio gestione directamente la base de datos. |

Tabla 5.1: Trazabilidad entre clases de análisis y clases de diseño

Como se puede observar, el paso del análisis al diseño no consiste solo en cambiar el nombre de las clases. Algunas clases de análisis se mantienen de forma directa, como `Usuario` o `Comentario`, mientras que otras se dividen en varios componentes para separar mejor las responsabilidades. Esta separación permite diferenciar la lógica asociada a las rutas de la aplicación, el acceso a la base de datos y la representación de los datos principales del sistema.

En la Figura 5.2 se muestra el diagrama de clases de diseño de la aplicación.

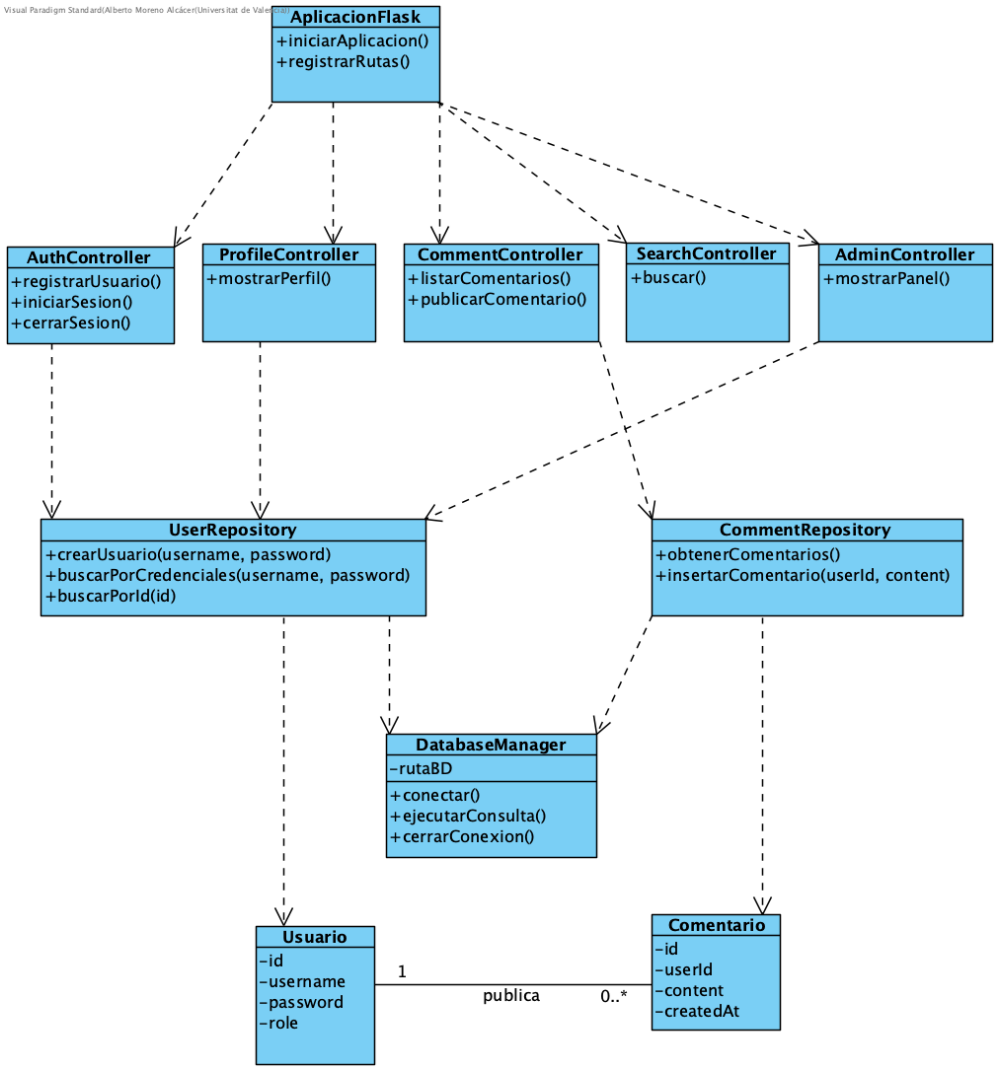



Figura 5.2: Diagrama de clases de diseño de la aplicación

### 5.2.5. Modelo lógico de datos

El modelo lógico de datos concreta cómo se almacenará la información de la aplicación en la base de datos SQLite. A diferencia del modelo conceptual del capítulo anterior, aquí ya se definen las tablas, los campos principales y la relación entre ellas.

La aplicación utiliza dos tablas principales: `users` y `comments`. La primera almacena la información de los usuarios registrados, mientras que la segunda guarda los comentarios publicados dentro de la aplicación.

La tabla `users` incluye un identificador único, el nombre de usuario, la contraseña y el rol asociado al usuario. El campo `role` permite diferenciar entre usuarios normales y usuarios administradores, aunque en este trabajo se introducirá de forma intencionada un fallo en la comprobación de permisos del panel de administración.

La tabla `comments` almacena los comentarios publicados. Cada comentario tiene un identificador propio, el identificador del usuario que lo ha creado, el contenido del comentario y la fecha de creación. La relación entre ambas tablas se establece mediante el campo `user_id`, que referencia al identificador del usuario en la tabla `users`.

En la Figura 5.3 se muestra el modelo lógico de datos de la aplicación.

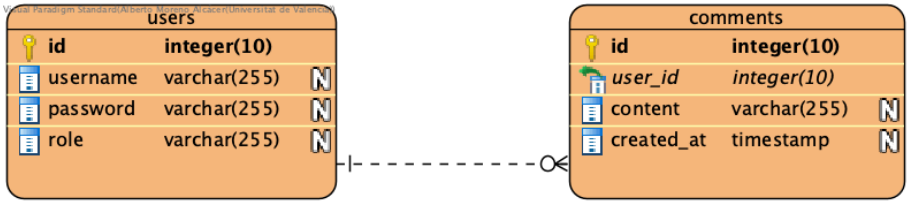



Figura 5.3: Modelo lógico de datos de la aplicación

La relación entre `users` y `comments` es de uno a muchos: un usuario puede publicar varios comentarios, pero cada comentario pertenece a un único usuario. Esta estructura es suficiente para cubrir las funcionalidades previstas en la aplicación, como el registro, el inicio de sesión, la visualización del perfil y la publicación de comentarios.

La sesión no se representa como una tabla independiente. Aunque en el análisis se identificó como un elemento del sistema, en el diseño se gestionará mediante el mecanismo de sesión proporcionado por Flask. Por tanto, no será necesario almacenar la sesión como entidad propia en la base de datos SQLite.

## 5.3. Diseño de las vulnerabilidades introducidas

En esta sección se describe cómo se diseñan las vulnerabilidades que se incorporarán a la aplicación. Estas vulnerabilidades no aparecen por accidente, sino que se introducen de forma controlada para poder analizarlas más adelante con OWASP ZAP y con una revisión manual cuando sea necesario.

La selección de vulnerabilidades responde a dos criterios. Por un lado, se han elegido fallos habituales en aplicaciones web y suficientemente representativos para un entorno de pruebas sencillo. Por otro lado, se han situado en funcionalidades concretas de la aplicación, de manera que exista una relación clara entre el punto de entrada, el comportamiento vulnerable y la prueba de seguridad que se realizará posteriormente.

Las vulnerabilidades diseñadas son las siguientes: inyección SQL en el proceso de inicio de sesión, XSS reflejado en la funcionalidad de búsqueda, fallo de control de acceso en el panel de administración y ausencia de determinadas cabeceras HTTP de seguridad. Su implementación concreta se mostrará en el capítulo siguiente; en este apartado solo se describe la decisión de diseño y el comportamiento esperado de cada una.

### 5.3.1. Diseño de la inyección SQL

La inyección SQL se ubica en el proceso de inicio de sesión, concretamente en la comprobación de las credenciales introducidas por el usuario. Esta funcionalidad se corresponde con el formulario de login y con el componente de autenticación descrito en el diseño de la aplicación.

En una aplicación segura, la comprobación de usuario y contraseña debería realizarse mediante consultas parametrizadas, evitando que los datos introducidos por el usuario formen parte directa de la consulta SQL. Sin embargo, para este entorno de pruebas se plantea una construcción insegura de la consulta, en la que los valores recibidos desde el formulario se incorporan directamente a la sentencia SQL.

El objetivo de este diseño es que el campo de usuario o contraseña pueda alterar la lógica de la consulta si se introduce una cadena especialmente construida. De esta forma, el formulario de inicio de sesión se convierte en el punto de entrada para estudiar una vulnerabilidad de inyección SQL.

Esta vulnerabilidad se sitúa en el componente de autenticación y afecta a la comunicación con la tabla `users`. Durante las pruebas, se comprobará si OWASP ZAP identifica este comportamiento como un posible fallo de inyección y si es necesario revisar manualmente la alerta para confirmar su validez.

### 5.3.2. Diseño del XSS reflejado

El XSS reflejado se plantea en la funcionalidad de búsqueda. Esta parte de la aplicación recibe un término introducido por el usuario mediante el parámetro de consulta y genera una respuesta HTML en la que se muestra dicho término.

Cuando una aplicación vuelve a mostrar datos introducidos por el usuario, esos datos deberían tratarse antes de incluirse en la respuesta. Esto permite evitar que el navegador interprete ese contenido como código HTML o JavaScript. Sin embargo, en este entorno de pruebas se plantea que el término de búsqueda se devuelva directamente en la página, sin aplicar un escape adecuado.

Con este diseño, si el usuario introduce una cadena que contiene código JavaScript, la aplicación reflejará ese contenido en la respuesta. El objetivo es reproducir un caso sencillo de XSS reflejado, donde el valor enviado en la petición aparece inmediatamente en la página generada por el servidor.

Esta vulnerabilidad se sitúa en el componente de búsqueda y afecta al punto de entrada `/search?q=`. Durante las pruebas, OWASP ZAP podrá enviar diferentes cargas de prueba sobre este parámetro para comprobar si la respuesta de la aplicación permite la ejecución o el reflejo de contenido no tratado correctamente.

### 5.3.3. Diseño del fallo de control de acceso

El fallo de control de acceso se introduce en el panel de administración. Esta funcionalidad debería estar reservada únicamente a usuarios con rol de administrador, ya que representa una zona privada de la aplicación.

En una aplicación segura, antes de mostrar el contenido del panel sería necesario comprobar dos condiciones: que el usuario haya iniciado sesión y que su rol sea el adecuado. En este trabajo, sin embargo, se plantea un diseño inseguro en el que esa comprobación no se realiza correctamente. Como consecuencia, la ruta del panel de administración podrá ser solicitada por usuarios que no deberían tener acceso.

El objetivo de este diseño es estudiar un fallo de control de acceso roto. A diferencia de otras vulnerabilidades como la inyección SQL o el XSS reflejado, este tipo de problema no siempre puede detectarse de forma fiable mediante un escaneo automático, porque depende de la lógica interna de permisos de la aplicación. Por este motivo, su revisión requerirá una comprobación manual además del análisis con OWASP ZAP.

Esta vulnerabilidad se sitúa en el componente de administración y afecta a la ruta `/admin`. Durante las pruebas se comprobará si un usuario sin rol de administrador puede acceder al panel y si OWASP ZAP genera alguna alerta relacionada con esta exposición.

### 5.3.4. Diseño de la configuración insegura de cabeceras HTTP

La última vulnerabilidad diseñada está relacionada con la ausencia de determinadas cabeceras HTTP de seguridad. A diferencia de los casos anteriores, no se sitúa en un formulario o en una ruta concreta, sino en la configuración general de las respuestas generadas por la aplicación.

Las cabeceras HTTP de seguridad permiten añadir restricciones al comportamiento del navegador y reducir ciertos riesgos habituales. Por ejemplo, cabeceras como `Content-Security-Policy`, `X-Content-Type-Options` o `X-Frame-Options` ayudan a limitar la ejecución de contenido no esperado, evitar interpretaciones incorrectas de tipos de contenido o reducir el riesgo de ataques de clickjacking.

En este entorno de pruebas se plantea que la aplicación no establezca estas cabeceras de forma explícita. El objetivo es comprobar cómo OWASP ZAP identifica este tipo de configuración insegura mediante su análisis pasivo, ya que no requiere enviar cargas de ataque específicas, sino observar las respuestas HTTP devueltas por la aplicación.

Esta vulnerabilidad afecta al comportamiento general del sistema, por lo que podrá observarse en distintas rutas de la aplicación. Los resultados obtenidos servirán posteriormente para relacionar este problema con buenas prácticas de configuración segura en aplicaciones web.

## 5.4. Diseño de las pruebas con OWASP ZAP

En esta sección se define cómo se diseñarán las pruebas de seguridad sobre la aplicación vulnerable. En el capítulo anterior se introdujo OWASP ZAP como herramienta externa de análisis; ahora se concreta qué acciones realizará, sobre qué puntos de entrada actuará y cómo se utilizará la información obtenida durante las pruebas.

La prueba no consistirá únicamente en lanzar OWASP ZAP y revisar lo que devuelva la herramienta. Antes del escaneo será necesario preparar la navegación por la aplicación, identificar las rutas principales y comprobar que el tráfico HTTP pasa correctamente por OWASP ZAP. Después se combinarán técnicas automáticas, como el spider y el escaneo activo, con una revisión manual de determinados comportamientos de la aplicación.

Esta combinación es necesaria porque no todos los fallos se comprueban igual. Problemas como la ausencia de cabeceras HTTP pueden identificarse mediante análisis pasivo, mientras que otros, como la inyección SQL o el XSS reflejado, requieren enviar cargas de prueba sobre formularios o parámetros. En cambio, los fallos de control de acceso suelen necesitar una validación manual, ya que dependen de la lógica de permisos de la aplicación.

Los resultados obtenidos durante estas pruebas se analizarán en el capítulo 6. En este capítulo no se muestran todavía alertas ni evidencias de ejecución, sino que se define la forma en la que se realizará la comprobación y cómo se tratará la información generada para conectarla posteriormente con la propuesta de buenas prácticas.

### 5.4.1. Particularización de las pruebas

Las pruebas se particularizan a partir de los puntos de entrada definidos en el análisis y de las vulnerabilidades diseñadas en este capítulo. Cada prueba se asocia a una funcionalidad concreta de la aplicación, de forma que el análisis no quede como una exploración genérica, sino como una comprobación orientada a los fallos que se quieren estudiar.

La Tabla 5.2 resume la relación entre los puntos de entrada, la vulnerabilidad prevista y el tipo de comprobación que se realizará.

| **Punto de entrada** | **Vulnerabilidad prevista** | **Comprobación diseñada** |
|---|---|---|
| `/login` | Inyección SQL | Escaneo activo sobre el formulario de inicio de sesión y revisión manual de la respuesta. |
| `/search?q=` | XSS reflejado | Escaneo activo sobre el parámetro `q` y comprobación de si el valor introducido se refleja sin tratamiento adecuado. |
| `/admin` | Control de acceso roto | Acceso manual con distintos perfiles de usuario y revisión del comportamiento observado. |
| Respuestas HTTP | Cabeceras de seguridad ausentes | Análisis pasivo de las respuestas para comprobar la ausencia de cabeceras de seguridad relevantes. |
| `/register`, `/profile`, `/comments` | Superficie de ataque adicional | Navegación manual y spider para que OWASP ZAP identifique rutas y formularios disponibles. |

Tabla 5.2: Particularización de las pruebas con OWASP ZAP

Esta tabla no recoge resultados, sino el diseño de la comprobación. La ejecución real de estas pruebas y las alertas obtenidas se presentarán más adelante, en el capítulo de implementación y pruebas.

### 5.4.2. Acciones previstas de OWASP ZAP

Una vez definida la relación entre puntos de entrada y vulnerabilidades, se concretan las acciones que realizará OWASP ZAP durante la actividad de test. Aunque en el análisis se haya incluido como herramienta externa, en el diseño conviene detallar qué papel tendrá realmente dentro de las pruebas.

La primera acción de OWASP ZAP será actuar como proxy entre el navegador y la aplicación Flask. De esta forma, las peticiones realizadas desde el navegador y las respuestas devueltas por la aplicación quedarán registradas en la herramienta. Esta fase permitirá comprobar que las rutas principales aparecen correctamente en el árbol de sitios de ZAP y que el tráfico HTTP se está capturando sin problemas.

La segunda acción será registrar la navegación manual por la aplicación. Antes de lanzar pruebas automáticas, se recorrerán las funcionalidades principales: registro, inicio de sesión, perfil, comentarios, búsqueda y panel de administración. Esto permitirá que ZAP conozca las rutas y formularios más importantes, incluyendo aquellos que requieren haber iniciado sesión previamente.

Después se utilizará el spider tradicional para completar la exploración de la aplicación. El objetivo del spider será localizar rutas enlazadas, formularios y recursos accesibles desde

las páginas visitadas. Esta acción servirá para ampliar la superficie de análisis antes de ejecutar pruebas más agresivas.

A continuación, se ejecutará el escaneo activo sobre los puntos de entrada definidos. Esta acción permitirá que OWASP ZAP envíe cargas de prueba sobre formularios y parámetros de la aplicación, especialmente en el formulario de inicio de sesión y en el parámetro de búsqueda. El escaneo activo se utilizará con cuidado, ya que modifica las peticiones enviadas a la aplicación y puede generar entradas no deseadas en la base de datos.

Por último, se revisarán las alertas generadas por la herramienta. Esta revisión no se tomará como una respuesta definitiva, sino como una base sobre la que trabajar después. Cada alerta deberá interpretarse teniendo en cuenta la ruta afectada, el tipo de vulnerabilidad, la confianza indicada por ZAP y la posibilidad de que se trate de un falso positivo.

En resumen, las acciones previstas para OWASP ZAP serán: interceptar el tráfico HTTP, registrar la navegación manual, explorar la aplicación mediante spider, ejecutar escaneos activos sobre los puntos de entrada relevantes y generar alertas que serán revisadas posteriormente.

### 5.4.3. Diseño de la actividad de test

La actividad de test se diseña como una secuencia ordenada de pasos. La intención es evitar que el análisis dependa únicamente de lanzar la herramienta sin preparación previa. Para que los resultados sean útiles, primero se debe preparar el entorno, después recorrer la aplicación y, finalmente, ejecutar las pruebas automáticas y revisar lo obtenido.

El primer paso será iniciar la aplicación Flask en local y comprobar que todas las funcionalidades básicas están disponibles. En esta fase no se analizan todavía vulnerabilidades, sino que se verifica que la aplicación responde correctamente y que las rutas principales pueden utilizarse desde el navegador.

El segundo paso será configurar el navegador para que todo el tráfico pase a través de OWASP ZAP. Esto permitirá que la herramienta registre las peticiones HTTP generadas durante la navegación. Si esta configuración no se realiza correctamente, el análisis posterior podría quedar incompleto, ya que ZAP no tendría visibilidad sobre la interacción entre el navegador y la aplicación.

El tercer paso consistirá en navegar manualmente por la aplicación. Se accederá al formulario de registro, al inicio de sesión, al perfil, a la página de comentarios, al buscador y al panel de administración. Esta navegación manual es importante porque permite generar tráfico real y asegurar que ZAP conoce las funcionalidades que después se analizarán.

El cuarto paso será ejecutar el spider tradicional sobre la aplicación. Con ello se pretende que ZAP complete la exploración de rutas y recursos accesibles, partiendo de la navegación realizada previamente. Esta fase ayudará a comprobar si existen páginas o formularios que no se hayan recorrido manualmente.

El quinto paso será ejecutar el escaneo activo sobre los puntos de entrada más relevantes. En concreto, se prestará especial atención al formulario de inicio de sesión y al parámetro de búsqueda, ya que son los puntos donde se han diseñado la inyección SQL y el XSS reflejado. El escaneo activo permitirá comprobar si ZAP detecta comportamientos compatibles con estas vulnerabilidades.

El sexto paso será realizar una revisión manual de aquellas vulnerabilidades que no

dependan únicamente de patrones detectables por la herramienta. El caso más claro es el panel de administración, donde será necesario comprobar si un usuario sin permisos puede acceder a la ruta `/admin`. Esta comprobación permitirá validar el fallo de control de acceso aunque ZAP no lo detecte de forma automática.

Finalmente, se recopilarán las alertas y observaciones obtenidas. En este capítulo no se presentan todavía esos resultados, pero sí queda definido que se organizarán por punto de entrada, tipo de vulnerabilidad y necesidad de validación manual. Esta organización facilitará el análisis posterior en el capítulo 6.

### 5.4.4. Tratamiento de los resultados obtenidos

Una vez ejecutadas las pruebas, los resultados obtenidos no se tratarán como una simple lista de alertas generadas por OWASP ZAP. Cada alerta deberá revisarse y ponerse en relación con el punto de entrada afectado, la vulnerabilidad que se pretendía comprobar y el comportamiento real observado en la aplicación.

El primer paso será separar los resultados según el tipo de vulnerabilidad: inyección SQL, XSS reflejado, control de acceso roto y cabeceras de seguridad ausentes. Esta clasificación permitirá comparar las vulnerabilidades diseñadas en este capítulo con las alertas que realmente aparezcan durante la ejecución de las pruebas.

El segundo paso será revisar la causa de cada problema. Para ello se analizará qué decisión de diseño o implementación ha permitido que aparezca la vulnerabilidad. Por ejemplo, en el caso de la inyección SQL se estudiará la forma en la que se construye la consulta de autenticación, mientras que en el caso del XSS reflejado se revisará cómo se devuelve el parámetro de búsqueda en la respuesta HTML.

El tercer paso será relacionar cada vulnerabilidad con categorías de OWASP Top 10 y con controles o requisitos de OWASP ASVS. Esta relación permitirá conectar el análisis práctico con los marcos de referencia estudiados en el estado del arte, para que los resultados no queden como observaciones sueltas.

Por último, los resultados se utilizarán como base para el capítulo 7. En ese capítulo se propondrán buenas prácticas y una metodología de desarrollo seguro a partir de los fallos observados. Por tanto, el objetivo no será solo saber qué vulnerabilidades aparecen, sino entender por qué se han producido y qué medidas habrían ayudado a evitarlas desde el diseño y la implementación.

### 5.4.5. Criterios de revisión de resultados

Para revisar los resultados de las pruebas se utilizarán varios criterios. Esto es necesario porque las alertas generadas por una herramienta automática no deben aceptarse sin más, especialmente cuando se trabaja con vulnerabilidades que pueden requerir interpretación manual.

El primer criterio será la severidad de la alerta. OWASP ZAP clasifica los hallazgos según su nivel de riesgo, lo que permite priorizar aquellos problemas que podrían tener un mayor impacto sobre la aplicación. No obstante, la severidad indicada por la herramienta se revisará teniendo en cuenta el contexto del entorno de pruebas.

El segundo criterio será la confianza de la alerta. Una alerta con alta confianza tendrá más probabilidad de corresponderse con un problema real, mientras que una alerta con

confianza baja deberá revisarse con más cuidado para descartar falsos positivos.

El tercer criterio será el punto de entrada afectado. Cada resultado se asociará a una ruta o funcionalidad concreta, como el formulario de inicio de sesión, el buscador o el panel de administración. Esto permitirá comprobar si las alertas coinciden con las vulnerabilidades previstas en el diseño.

El cuarto criterio será la necesidad de validación manual. Algunas vulnerabilidades, especialmente las relacionadas con el control de acceso, no pueden evaluarse únicamente a partir de una alerta automática. En estos casos se comprobará manualmente el comportamiento de la aplicación con distintos tipos de usuario.

Por último, se tendrá en cuenta la relación de cada hallazgo con OWASP Top 10 y OWASP ASVS. Esta relación será importante para transformar los resultados técnicos en recomendaciones de desarrollo seguro, que se desarrollarán posteriormente en el capítulo 7.

# Capítulo 6

# Implementación y pruebas

## 6.1. Introducción

En el capítulo anterior se definió el diseño de la aplicación web vulnerable y de las pruebas de seguridad que se iban a realizar con OWASP ZAP. A partir de ese diseño, en este capítulo se explica cómo se ha implementado la aplicación y cómo se han llevado a cabo las pruebas sobre ella.

El capítulo se divide en dos partes principales. En primer lugar, se describe la implementación de la aplicación vulnerable, indicando el entorno utilizado, la estructura del proyecto y las funcionalidades desarrolladas. La aplicación incluye registro de usuarios, inicio de sesión, perfil, comentarios, búsqueda y panel de administración. Sobre estas funcionalidades se han introducido de forma intencionada varias vulnerabilidades vistas en los capítulos anteriores: inyección SQL, XSS reflejado, fallo de control de acceso y ausencia de cabeceras HTTP de seguridad.

En segundo lugar, se describe la ejecución de las pruebas con OWASP ZAP. Para ello se muestra cómo se preparó el entorno, cómo se configuró el escaneo automático y qué alertas se obtuvieron. Además, los resultados de ZAP se complementan con comprobaciones manuales, ya que algunas vulnerabilidades, como el acceso indebido al panel de administración, se entienden mejor observando directamente el comportamiento de la aplicación.

El objetivo de este capítulo no es solo mostrar capturas de la herramienta, sino comprobar si las vulnerabilidades introducidas aparecen realmente durante las pruebas y analizar qué información proporciona OWASP ZAP sobre ellas. Estos resultados servirán después como base para proponer buenas prácticas de desarrollo seguro.

## 6.2. Implementación de la aplicación vulnerable

La aplicación vulnerable se ha implementado siguiendo el diseño definido en el capítulo anterior. Se trata de una aplicación web sencilla, desarrollada con Flask y SQLite, cuyo objetivo es servir como entorno controlado para analizar vulnerabilidades web mediante OWASP ZAP.

No se ha buscado crear una aplicación completa ni preparada para producción, sino un prototipo pequeño que permita reproducir fallos de seguridad habituales de forma

clara. Por este motivo, se han implementado únicamente las funcionalidades necesarias para el análisis: registro de usuarios, inicio de sesión, visualización del perfil, publicación de comentarios, búsqueda y panel de administración.

Las vulnerabilidades se han incorporado de manera deliberada. Esto permite comprobar primero su comportamiento de forma manual y, posteriormente, analizar si OWASP ZAP es capaz de detectarlas durante el escaneo automático. La aplicación, por tanto, no debe interpretarse como un sistema seguro, sino como un entorno de pruebas creado específicamente para este trabajo.

### 6.2.1. Aspecto final de la aplicación

Una vez implementadas las funcionalidades principales, la aplicación presenta una interfaz sencilla y uniforme. Todas las páginas mantienen una cabecera común con el título de la aplicación y una barra de navegación que permite acceder a las distintas secciones: inicio, registro, login, perfil, comentarios, búsqueda, panel de administración y cierre de sesión.

La Figura 6.1 muestra la página principal de la aplicación. Esta pantalla sirve como punto de entrada y permite comprobar que la aplicación se ejecuta correctamente en el entorno local.

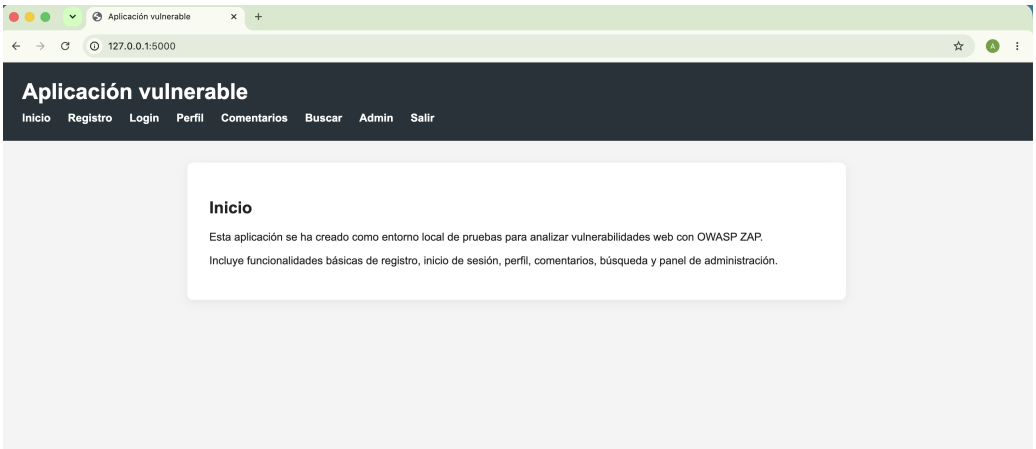



Figura 6.1: Página principal de la aplicación vulnerable

La Figura 6.2 muestra el formulario de inicio de sesión. Esta pantalla es relevante porque sobre ella se ha introducido la vulnerabilidad de inyección SQL que se analiza posteriormente durante las pruebas.

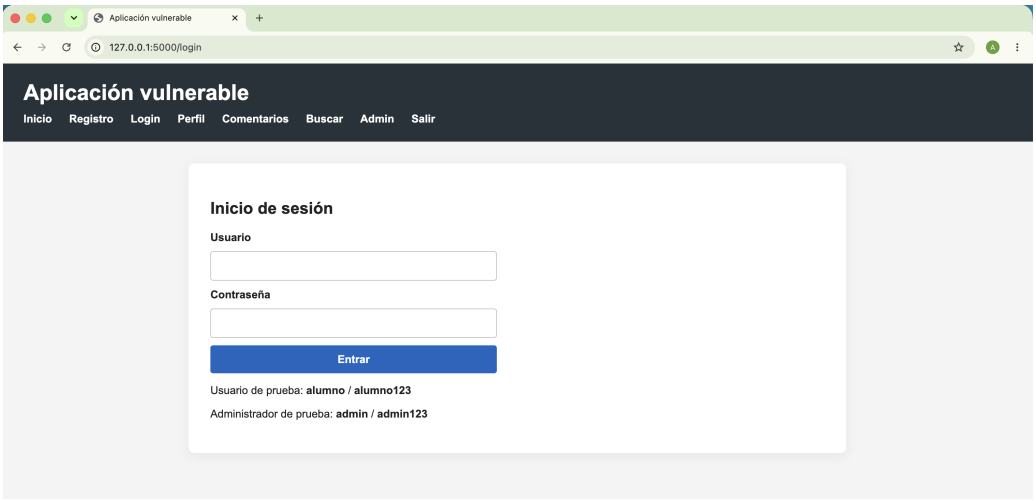



Figura 6.2: Formulario de inicio de sesión de la aplicación

La Figura 6.3 muestra la funcionalidad de búsqueda. Esta pantalla recibe un término introducido por el usuario y lo refleja en la respuesta, por lo que se utilizará más adelante para comprobar el XSS reflejado.

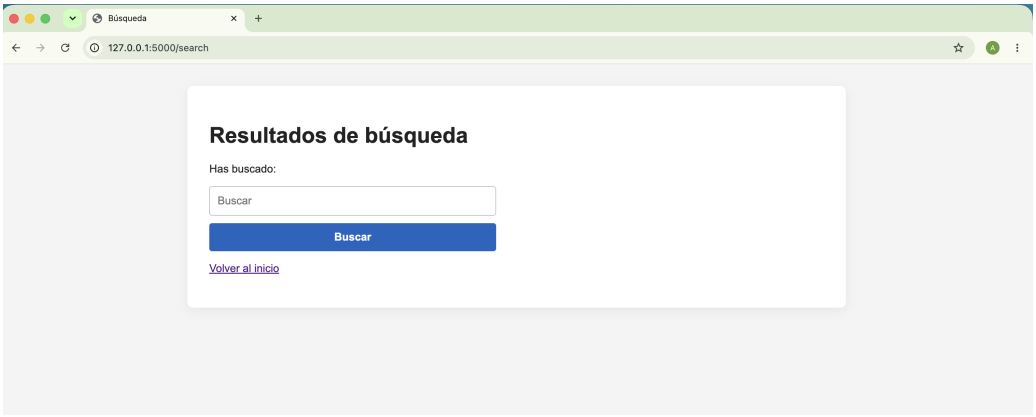



Figura 6.3: Pantalla de búsqueda de la aplicación

Por último, la Figura 6.4 muestra el panel de administración. En la aplicación desarrollada esta ruta presenta un fallo de control de acceso, ya que permite visualizar usuarios registrados sin comprobar correctamente si existe una sesión iniciada ni si el usuario tiene permisos de administrador.

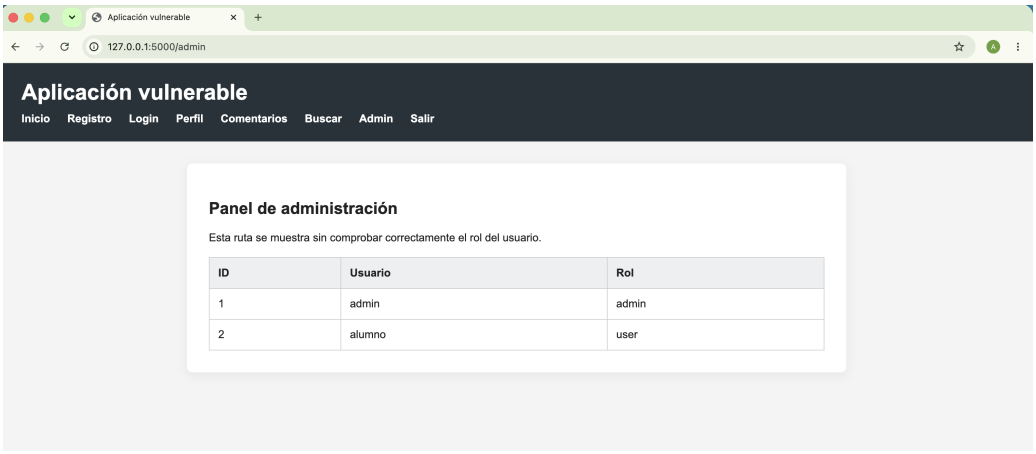



Figura 6.4: Panel de administración de la aplicación

Estas capturas permiten ver el aspecto final de la aplicación antes de entrar en la ejecución de las pruebas. Además, ayudan a relacionar cada pantalla con la vulnerabilidad que se analiza después: el login con la inyección SQL, la búsqueda con el XSS reflejado y el panel de administración con el fallo de control de acceso.

### 6.2.2. Entorno de desarrollo

El desarrollo y las pruebas se han realizado en un entorno local sobre macOS ARM. La aplicación se ha ejecutado únicamente en la máquina del alumno, sin publicarse en Internet ni desplegarse en un servidor externo. Esta decisión permite mantener el análisis dentro de un entorno controlado y evita que las vulnerabilidades introducidas puedan afectar a terceros.

Las tecnologías utilizadas han sido las siguientes:

- **Python**: lenguaje utilizado para implementar la lógica principal de la aplicación.
- **Flask**: microframework utilizado para definir las rutas, procesar las peticiones HTTP y generar las respuestas HTML.
- **SQLite**: sistema de base de datos empleado para almacenar los usuarios y los comentarios.
- **HTML y CSS**: tecnologías utilizadas para construir una interfaz web sencilla y suficientemente clara para realizar las pruebas.
- **OWASP ZAP**: herramienta utilizada para analizar dinámicamente la aplicación en ejecución.

La aplicación se ejecuta mediante el servidor de desarrollo de Flask, accesible desde la dirección local `http://127.0.0.1:5000`. Aunque este servidor no está pensado para un entorno real de producción, resulta suficiente para el contexto de este TFG, ya que el objetivo es trabajar en local sobre una aplicación de pruebas.

Para aislar las dependencias del proyecto se ha utilizado un entorno virtual de Python. Una vez activado dicho entorno, se instalaron las dependencias necesarias, se inicializó la base de datos y se arrancó la aplicación Flask desde la terminal.

### 6.2.3. Estructura del proyecto

El proyecto se ha organizado de forma sencilla, separando el código principal de la aplicación, la inicialización de la base de datos, las plantillas HTML, los estilos CSS y las evidencias obtenidas durante las pruebas. La estructura general utilizada ha sido la siguiente:

```
tfg_app_vulnerable/
|-- app.py
|-- init_db.py
|-- requirements.txt
|-- README.md
|-- database.db
|-- static/
|   |-- style.css
|-- templates/
|   |-- ...
|-- evidencias/
|   |-- app_inicio.png
|   |-- app_login.png
|   |-- app_busqueda.png
|   |-- app_admin.png
|   |-- admin_sin_sesion.png
|   |-- xss_reflejado_alerta.png
|   |-- sql_injection_payload.png
|   |-- sql_injection_login_exitoso.png
|   |-- zap_configuracion_objetivo.png
|   |-- zap_alertas_generales.png
|   |-- zap_respuesta_http_cabeceras.png
|   |-- zap_alerta_sql_injection.png
|   |-- zap_alerta_csp.png
|   |-- zap_alerta_x_content_type_options.png
|   |-- zap_alerta_anti_clickjacking.png
```

El archivo `app.py` contiene la lógica principal de la aplicación Flask. En él se definen las rutas, el tratamiento de los formularios, la gestión básica de sesión y las consultas realizadas sobre la base de datos.

El archivo `init_db.py` se utiliza para inicializar la base de datos SQLite. Este script crea las tablas necesarias e introduce algunos datos de prueba, entre ellos un usuario normal y un usuario administrador. La base de datos generada se almacena en el archivo `database.db`.

El archivo `requirements.txt` recoge las dependencias necesarias para ejecutar la aplicación. En este caso, la dependencia principal es Flask.

La carpeta `templates` contiene las plantillas HTML utilizadas para generar las distintas páginas de la aplicación. La carpeta `static` contiene el archivo `style.css`, donde se definen los estilos visuales de la interfaz. Por último, la carpeta `evidencias` almacena las capturas obtenidas durante las pruebas manuales y durante el análisis con OWASP ZAP.

### 6.2.4. Implementación de la autenticación

La autenticación se ha implementado mediante un formulario de inicio de sesión formado por dos campos: nombre de usuario y contraseña. Cuando el usuario envía el formulario, la aplicación recoge esos valores y realiza una consulta sobre la tabla `users` para comprobar si existe un usuario con esas credenciales.

Si la consulta devuelve un usuario, la aplicación guarda en la sesión algunos datos básicos, como el identificador del usuario, el nombre de usuario y el rol. A partir de ese momento, el usuario puede acceder a las funcionalidades que requieren sesión, como el perfil o la publicación de comentarios.

Además del inicio de sesión, la aplicación incluye una funcionalidad de registro. En este caso, el usuario introduce un nombre y una contraseña, y la aplicación crea un nuevo registro en la base de datos con rol de usuario normal. Esta parte se ha mantenido sencilla, ya que su objetivo principal es permitir crear usuarios de prueba para interactuar con la aplicación.

En el caso del formulario de inicio de sesión, la consulta se ha construido de forma deliberadamente insegura, concatenando directamente los valores introducidos por el usuario. Esta decisión se ha tomado para introducir la vulnerabilidad de inyección SQL que se analizará posteriormente. Por tanto, aunque la autenticación funciona desde el punto de vista funcional, no se considera una implementación segura.

### 6.2.5. Implementación de la gestión de comentarios

La gestión de comentarios permite que un usuario autenticado consulte los comentarios almacenados y publique nuevos mensajes. Para acceder a esta funcionalidad, la aplicación comprueba previamente que exista una sesión iniciada. Si el usuario no ha iniciado sesión, se muestra un mensaje informativo y se redirige al formulario de login.

Cuando un usuario accede a la página de comentarios, la aplicación consulta la base de datos para recuperar los comentarios existentes. En la consulta se obtiene también el nombre del usuario que publicó cada comentario, de forma que la página pueda mostrar tanto el contenido como su autor.

La publicación de comentarios se realiza mediante un formulario. Si el usuario introduce un texto y lo envía, la aplicación inserta un nuevo registro en la tabla `comments`, asociándolo al identificador del usuario almacenado en sesión. Después de guardar el comentario, se redirige de nuevo a la misma página para mostrar el listado actualizado.

Esta funcionalidad se ha incluido porque permite que la aplicación tenga una parte privada con interacción real entre usuario y base de datos. Aunque no es la vulnerabilidad principal del análisis, sirve para dar más contexto a la aplicación y para que OWASP ZAP disponga de más rutas y formularios durante la exploración.

### 6.2.6. Implementación de la funcionalidad de búsqueda

La funcionalidad de búsqueda se ha implementado mediante la ruta `/search`. Esta ruta recibe un parámetro llamado `q` a través de la URL y genera una página HTML en la que se muestra el valor introducido por el usuario.

La página incluye un pequeño formulario de búsqueda, de forma que el usuario puede

introducir un término y enviarlo mediante una petición GET. El valor recibido se utiliza para mostrar un mensaje del tipo "Has buscado:" seguido del texto introducido.

En una aplicación segura, ese valor debería escaparse antes de incluirse en la respuesta HTML. Sin embargo, en este caso se inserta directamente en la página generada por el servidor. Esta implementación se ha realizado así de forma intencionada para reproducir un caso de XSS reflejado.

Gracias a esta decisión, si el usuario introduce una cadena con código JavaScript, el navegador puede llegar a interpretarla como parte de la página. Esta vulnerabilidad se comprobará posteriormente tanto de forma manual como mediante las alertas generadas por OWASP ZAP.

### 6.2.7. Implementación del panel de administración

El panel de administración se ha implementado mediante la ruta `/admin`. Esta página muestra una tabla con los usuarios registrados en la aplicación, incluyendo su identificador, nombre de usuario y rol.

Desde el punto de vista funcional, el panel permite visualizar información que debería estar reservada únicamente a usuarios administradores. Por ese motivo, en una aplicación real sería necesario comprobar dos aspectos antes de mostrar la página: que el usuario haya iniciado sesión y que su rol sea `admin`.

En esta aplicación, sin embargo, esa comprobación no se realiza. La ruta `/admin` consulta directamente la base de datos y muestra el listado de usuarios, aunque quien acceda no tenga sesión iniciada. Esta implementación reproduce de forma clara un fallo de control de acceso.

Durante las pruebas manuales se comprobó que era posible acceder al panel de administración desde una ventana de incógnito, sin iniciar sesión previamente. Esta situación confirma que el problema no afecta solo a usuarios autenticados con un rol incorrecto, sino también a usuarios no autenticados que conozcan la ruta.

### 6.2.8. Implementación de las vulnerabilidades introducidas

Las vulnerabilidades de la aplicación se han introducido de forma deliberada y en puntos concretos del sistema. La intención no era generar una aplicación insegura de manera accidental, sino disponer de un entorno donde poder observar fallos habituales y comprobar cómo los detecta OWASP ZAP.

La primera vulnerabilidad se encuentra en el inicio de sesión. La consulta SQL se construye concatenando directamente los campos introducidos por el usuario. Esto permite modificar la lógica de la consulta mediante una entrada especialmente preparada, dando lugar a una inyección SQL.

La segunda vulnerabilidad aparece en la funcionalidad de búsqueda. El parámetro `q` se inserta directamente en la respuesta HTML, sin aplicar un escape adecuado. Como consecuencia, es posible introducir código JavaScript en la URL y hacer que el navegador lo ejecute al cargar la página.

La tercera vulnerabilidad se encuentra en el panel de administración. La ruta `/admin` no comprueba si existe una sesión iniciada ni si el usuario tiene rol de administrador. Por

tanto, cualquier usuario que conozca la ruta puede acceder al listado de usuarios.

La cuarta vulnerabilidad está relacionada con la configuración general de la aplicación. No se han añadido cabeceras HTTP de seguridad como `Content-Security-Policy`, `X-Content-Type-Options` o `X-Frame-Options`. Esta ausencia permite que OWASP ZAP detecte problemas de configuración mediante el análisis pasivo de las respuestas.

En conjunto, estas vulnerabilidades cubren varios tipos de fallos frecuentes en aplicaciones web: entrada no validada, salida no escapada, control de acceso insuficiente y configuración insegura. En los apartados siguientes se describe cómo se probaron estas vulnerabilidades y qué resultados se obtuvieron durante el análisis.

## 6.3. Ejecución de pruebas con OWASP ZAP

Una vez implementada la aplicación vulnerable y comprobado que las rutas principales funcionaban correctamente, se pasó a ejecutar las pruebas con OWASP ZAP. La aplicación se mantuvo en ejecución en local, accesible desde `http://127.0.0.1:5000`, y ZAP se utilizó para explorarla, analizar sus respuestas y generar alertas sobre posibles vulnerabilidades.

El objetivo de esta fase no era únicamente obtener una lista de alertas, sino comprobar si la herramienta detectaba las vulnerabilidades introducidas en la aplicación y qué información proporcionaba sobre cada una de ellas. Para ello, se combinó el escaneo automático con comprobaciones manuales realizadas desde el navegador.

### 6.3.1. Preparación del entorno de pruebas

Antes de iniciar el análisis con OWASP ZAP, se arrancó la aplicación Flask desde la terminal. Para ello se accedió a la carpeta del proyecto, se activó el entorno virtual de Python y se ejecutó el archivo principal de la aplicación.

Los comandos utilizados fueron los siguientes:

```
cd ~/Downloads/tfg_app_vulnerable
source .venv/bin/activate
python app.py
```

Tras ejecutar la aplicación, Flask mostró que el servidor de desarrollo estaba disponible en la dirección `http://127.0.0.1:5000`. A partir de ese momento, se comprobó desde el navegador que las rutas principales cargaban correctamente: página de inicio, registro, login, perfil, comentarios, búsqueda y panel de administración.

Esta comprobación previa era necesaria para evitar que los resultados de OWASP ZAP se vieran afectados por errores de ejecución de la aplicación o por rutas que no estuvieran disponibles. También permitió confirmar que las vulnerabilidades introducidas podían probarse manualmente antes de lanzar el escaneo automático.

### 6.3.2. Configuración de OWASP ZAP

Una vez arrancada la aplicación, se abrió OWASP ZAP en modo estándar. Al iniciar la herramienta se seleccionó la opción de no guardar la sesión, ya que el análisis se iba a

realizar como una prueba puntual y las evidencias se guardarían mediante capturas.

Para el escaneo automático se indicó como objetivo la dirección local de la aplicación:

```
http://127.0.0.1:5000
```

Además, se activó el spider tradicional y se desactivó el spider Ajax. Esta configuración se eligió porque la aplicación desarrollada no depende de JavaScript para construir sus rutas principales, por lo que el spider tradicional era suficiente para recorrer las páginas disponibles. También se evitaban posibles problemas de configuración con navegadores externos.

En la Figura 6.5 se muestra la configuración utilizada antes de iniciar el escaneo.

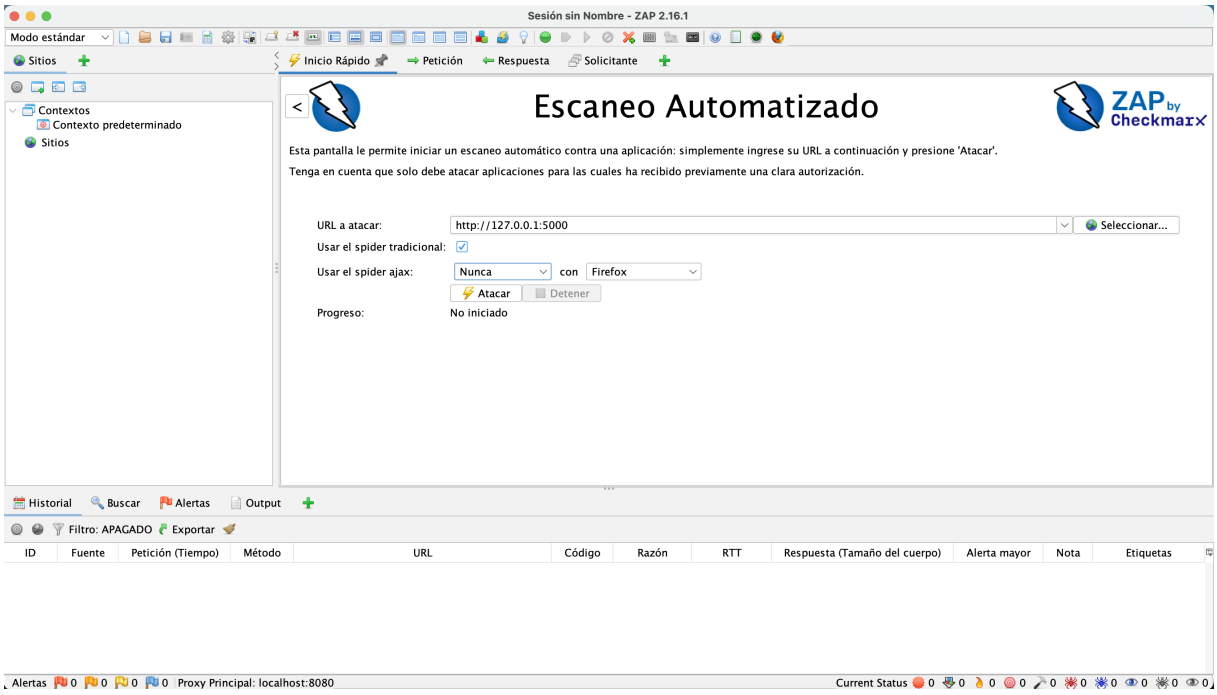



Figura 6.5: Configuración del objetivo de análisis en OWASP ZAP

### 6.3.3. Exploración de la aplicación mediante Spider

El primer paso del análisis automático fue la exploración de la aplicación mediante el spider tradicional de OWASP ZAP. Esta fase permite recorrer las rutas enlazadas desde el punto de entrada inicial y registrar los recursos encontrados.

Durante esta exploración, ZAP identificó distintas rutas de la aplicación, como la página principal, el formulario de login, el registro, la búsqueda, el panel de administración y los recursos estáticos asociados. Esta información sirvió como base para que la herramienta pudiera realizar posteriormente el escaneo activo sobre los formularios y parámetros detectados.

La exploración mediante spider no tiene como objetivo principal explotar vulnerabilidades, sino construir una visión inicial de la superficie de ataque. En este caso, resultó útil para confirmar que ZAP podía acceder a las rutas principales de la aplicación y registrar las peticiones HTTP generadas.

### 6.3.4. Ejecución del escaneo activo

Después de la exploración inicial se ejecutó el escaneo activo. A diferencia del spider, esta fase sí envía cargas de prueba a la aplicación con el objetivo de detectar comportamientos vulnerables. Por este motivo, se realizó únicamente sobre el entorno local creado para el TFG.

El escaneo activo generó distintas peticiones sobre los formularios y parámetros disponibles. En particular, resultó relevante el análisis del formulario de inicio de sesión, ya que en él se había introducido de forma deliberada la vulnerabilidad de inyección SQL. También se analizaron otras rutas de la aplicación, lo que permitió detectar problemas de configuración relacionados con cabeceras HTTP ausentes.

Durante esta fase, algunas peticiones provocaron errores internos en la aplicación. Esto era esperable, ya que OWASP ZAP introduce valores diseñados para alterar el comportamiento normal de los formularios. En el caso concreto de la inyección SQL, estos errores sirvieron como evidencia para que la herramienta identificara un posible problema en la construcción de la consulta.

### 6.3.5. Revisión de alertas en OWASP ZAP

Una vez finalizado el escaneo, se revisaron manualmente las alertas generadas por OWASP ZAP. Esta revisión fue necesaria porque las herramientas automáticas no siempre permiten confirmar por sí solas el impacto real de una vulnerabilidad.

En la pestaña de alertas se observaron varios resultados relevantes para el trabajo, entre ellos una alerta de inyección SQL sobre el formulario de login y varias alertas relacionadas con la ausencia de cabeceras HTTP de seguridad. También aparecieron otras alertas informativas o de menor prioridad, que se tuvieron en cuenta pero no se consideraron el foco principal del análisis.

En la Figura 6.6 se muestra la vista general de alertas obtenida tras el escaneo.

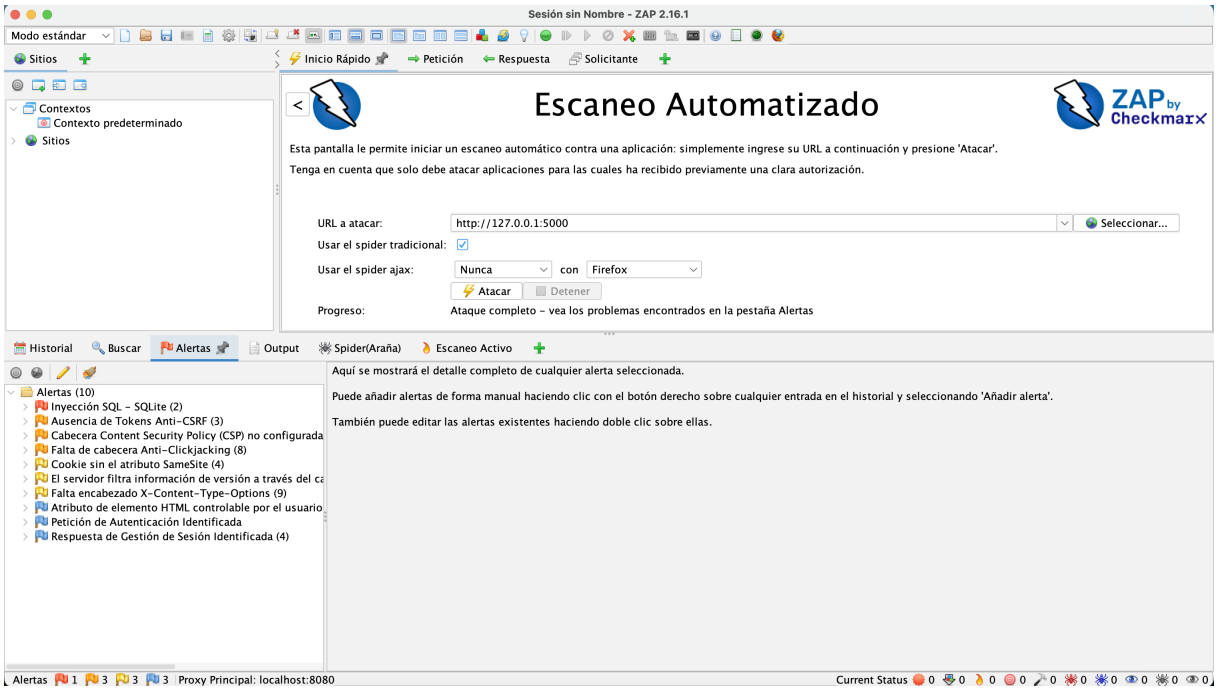



Figura 6.6: Alertas generales detectadas por OWASP ZAP

La revisión de alertas se realizó seleccionando cada hallazgo relevante dentro de OWASP ZAP y comprobando la información mostrada por la herramienta: URL afectada, parámetro implicado, nivel de riesgo, confianza de la alerta y evidencia asociada. Esta revisión permitió diferenciar entre alertas útiles para el análisis y otros avisos de menor relevancia para los objetivos del trabajo.

### 6.3.6. Comprobación de la inyección SQL

La vulnerabilidad de inyección SQL se había introducido en el formulario de inicio de sesión. Durante el escaneo activo, OWASP ZAP detectó una alerta de tipo *Inyección SQL - SQLite* sobre la ruta `/login`. La herramienta identificó el problema al manipular los parámetros del formulario y observar errores devueltos por la base de datos.

En la Figura 6.7 se muestra el detalle de la alerta generada por OWASP ZAP. En ella se observa que la alerta afecta al formulario de login, tiene riesgo alto y está asociada a un error de sintaxis SQL.

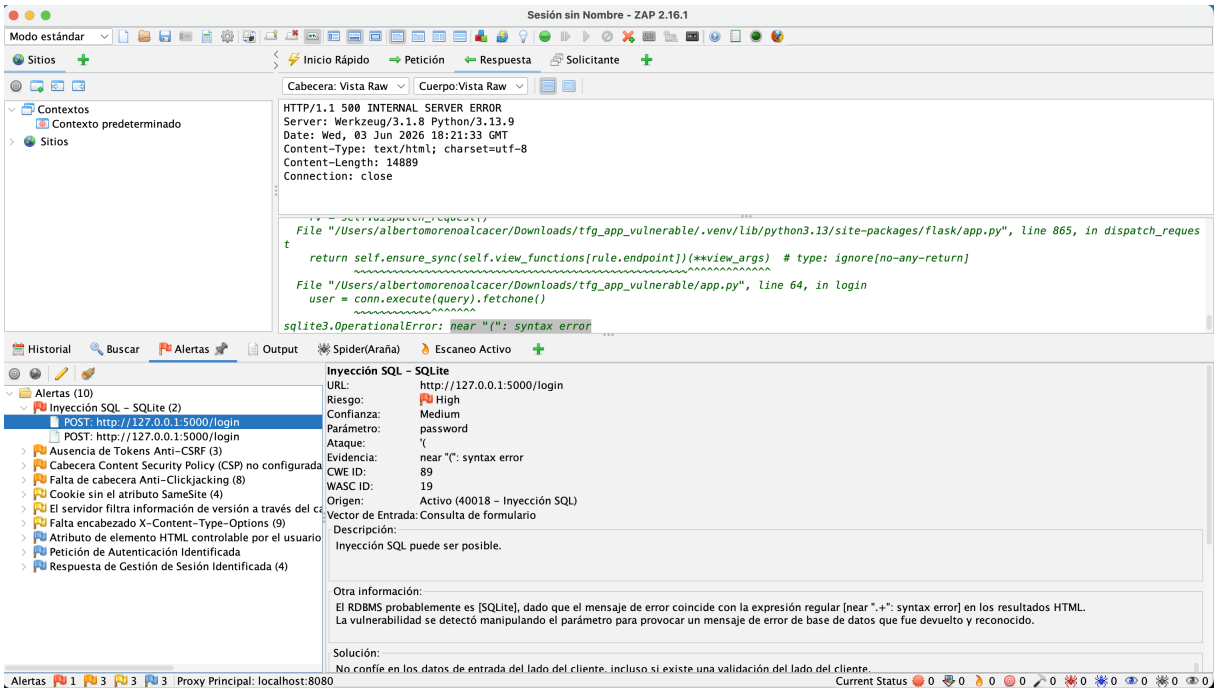



Figura 6.7: Alerta de inyección SQL detectada por OWASP ZAP

Además de la detección automática, se realizó una comprobación manual desde el navegador. Para ello se introdujo una cadena preparada en el campo de usuario del formulario de login y una contraseña cualquiera. Con esta entrada, la aplicación inició sesión sin utilizar unas credenciales válidas, lo que confirmó que la consulta SQL podía alterarse desde el formulario.

En la Figura 6.8 se muestra el formulario de inicio de sesión con la carga utilizada durante la prueba manual.

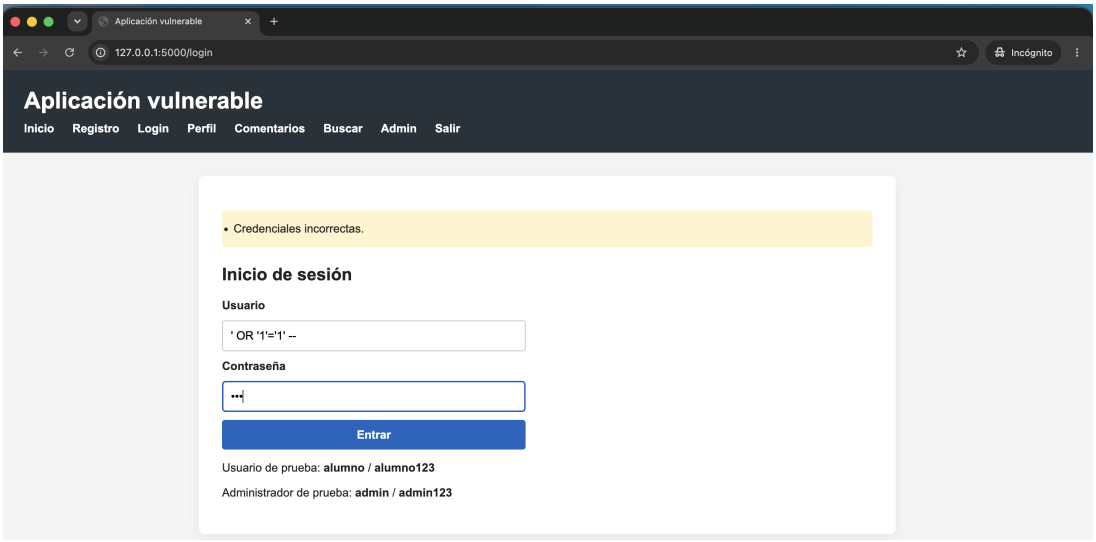



Figura 6.8: Carga introducida en el formulario de login para comprobar la inyección SQL

En la Figura 6.9 se observa que, tras enviar el formulario, la aplicación muestra una sesión iniciada correctamente. Esta comprobación permite confirmar que la alerta de OWASP ZAP corresponde a una vulnerabilidad real y no a un falso positivo.

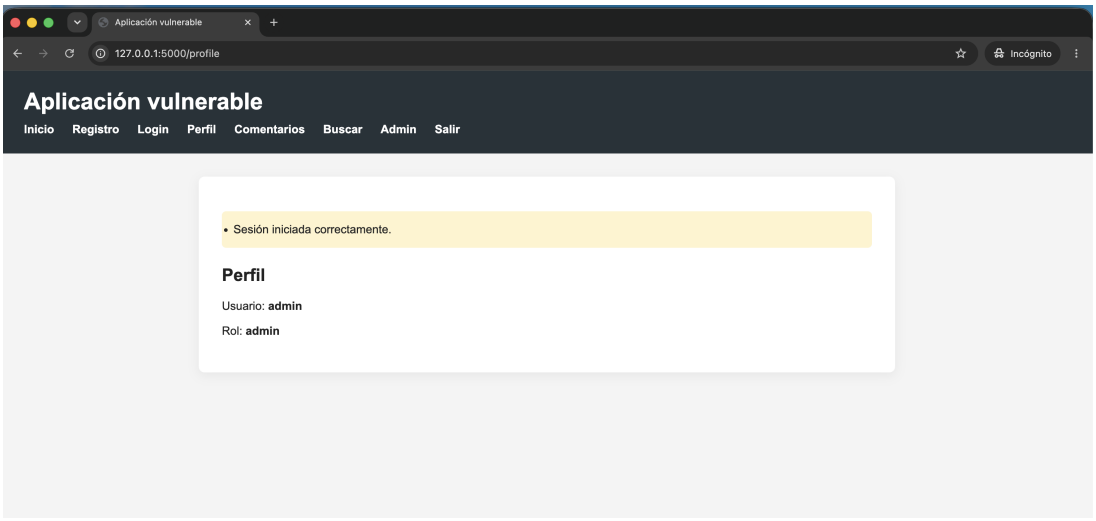



Figura 6.9: Inicio de sesión conseguido mediante la vulnerabilidad de inyección SQL

Este resultado confirma que la causa del problema se encuentra en la construcción insegura de la consulta SQL. Al concatenar directamente los valores introducidos por el usuario, la aplicación permite modificar la lógica de autenticación.

### 6.3.7. Comprobación del XSS reflejado

La vulnerabilidad de XSS reflejado se había introducido en la funcionalidad de búsqueda. Esta ruta recibe el parámetro `q` y lo devuelve dentro de la respuesta HTML sin aplicar un escape adecuado.

Durante las pruebas manuales se introdujo una carga JavaScript en el parámetro de búsqueda. Al cargar la página, el navegador interpretó ese contenido como código y mostró una alerta. Este comportamiento confirma que la aplicación refleja directamente la entrada del usuario en la respuesta.

En la Figura 6.10 se muestra la ejecución del XSS reflejado en el navegador.

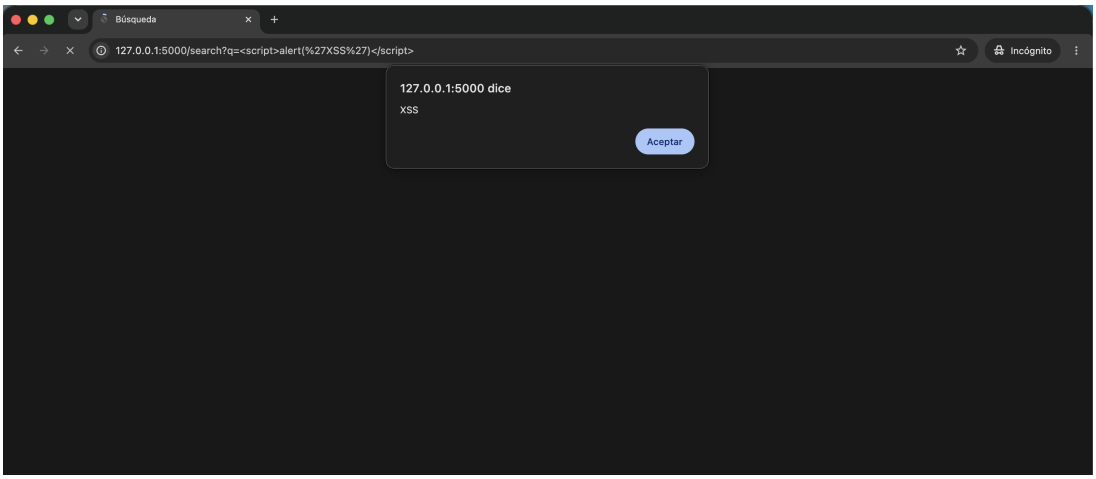



Figura 6.10: Comprobación manual del XSS reflejado en la funcionalidad de búsqueda

En este caso, la comprobación manual resulta especialmente útil, ya que permite ver

de forma directa el efecto de la vulnerabilidad. La alerta del navegador demuestra que el valor enviado en la URL no se trata correctamente antes de incorporarse a la página.

La causa del problema está en la falta de escape de la salida HTML. Aunque el parámetro se recibe correctamente desde el punto de vista funcional, la aplicación no diferencia entre texto introducido por el usuario y contenido interpretable por el navegador.

### 6.3.8. Comprobación del control de acceso

El fallo de control de acceso se había introducido en la ruta `/admin`. Esta ruta debería estar reservada a usuarios con rol de administrador, pero la aplicación no comprueba si existe una sesión iniciada ni si el usuario tiene permisos suficientes.

Durante la prueba manual se accedió directamente a `/admin` desde una ventana de incógnito, sin iniciar sesión previamente. A pesar de ello, la aplicación mostró el panel de administración con el listado de usuarios registrados.

En la Figura 6.11 se muestra el acceso al panel de administración sin una sesión autenticada.

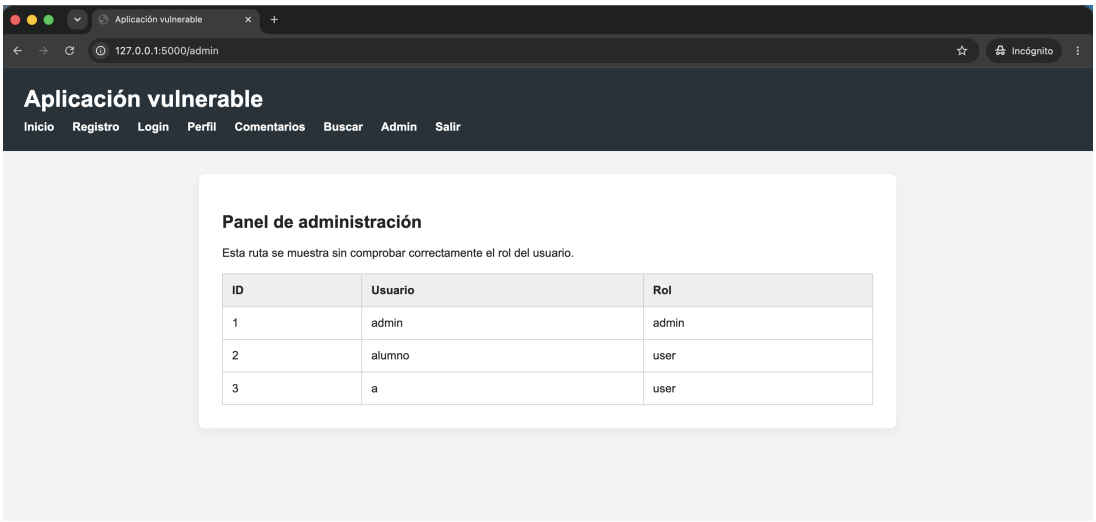



Figura 6.11: Acceso al panel de administración sin iniciar sesión

Este resultado confirma que el fallo no depende únicamente de tener un rol incorrecto, sino que también afecta a usuarios no autenticados. Cualquier usuario que conozca la ruta puede acceder al contenido del panel.

A diferencia de la inyección SQL o de la ausencia de cabeceras, este tipo de vulnerabilidad no siempre aparece de forma clara en las alertas automáticas de OWASP ZAP, porque depende de la lógica interna de autorización. Por este motivo, la revisión manual ha sido necesaria para confirmar el problema.

### 6.3.9. Comprobación de cabeceras HTTP de seguridad

Además de las vulnerabilidades anteriores, OWASP ZAP detectó varios problemas relacionados con la ausencia de cabeceras HTTP de seguridad. Estas alertas se generaron mediante análisis pasivo, observando las respuestas devueltas por la aplicación.

Entre las cabeceras ausentes se identificaron `Content-Security-Policy`, `X-Content-Type-Options` y mecanismos de protección frente a clickjacking. Aunque estas alertas no implican por sí solas una explotación directa como ocurre con la inyección SQL, sí indican una configuración de seguridad incompleta.

Además de revisar la descripción de la alerta, también se comprobó la respuesta HTTP asociada desde la propia interfaz de OWASP ZAP. Esta revisión permite observar las cabeceras devueltas por la aplicación y confirmar que no se incluye una política `Content-Security-Policy` en la respuesta.

En la Figura 6.12 se muestra la respuesta HTTP asociada a una de las alertas de cabeceras. En ella se observan cabeceras como `Server`, `Content-Type`, `Content-Length` y `Connection`, pero no aparece una cabecera `Content-Security-Policy`. Esta comprobación refuerza la revisión manual de la alerta generada por OWASP ZAP.

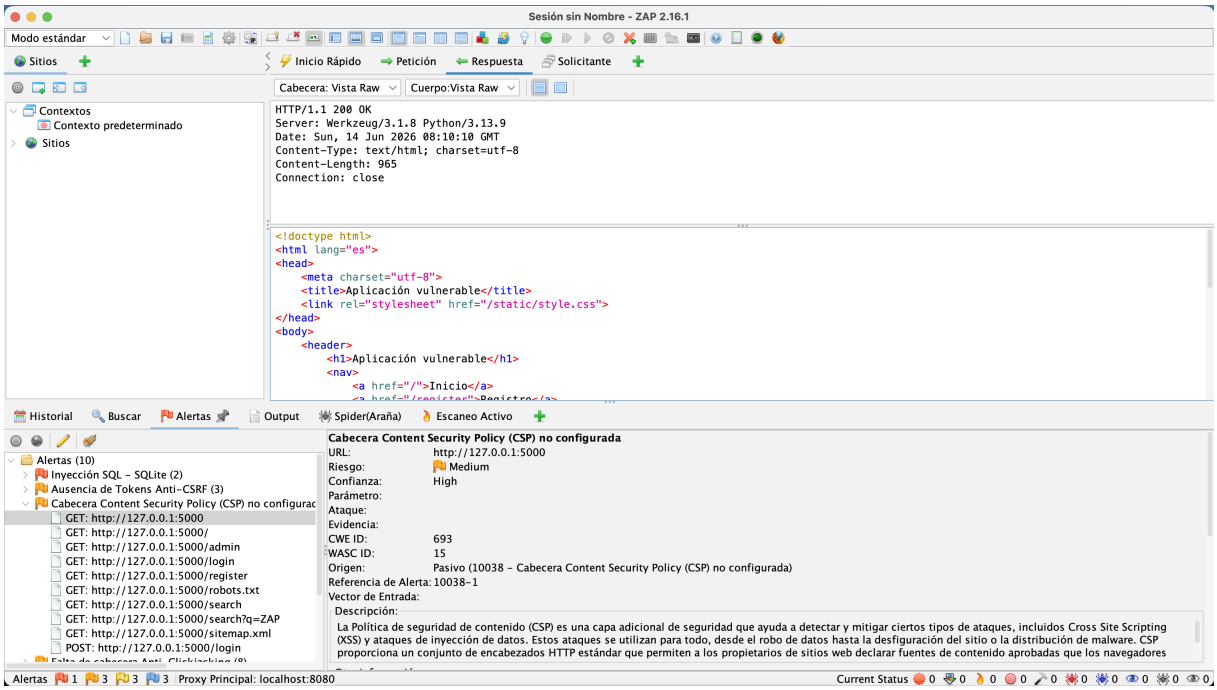



Figura 6.12: Revisión de la respuesta HTTP asociada a la alerta de cabeceras en OWASP ZAP

En la Figura 6.13 se muestra la alerta correspondiente a la cabecera `Content-Security-Policy` no configurada.

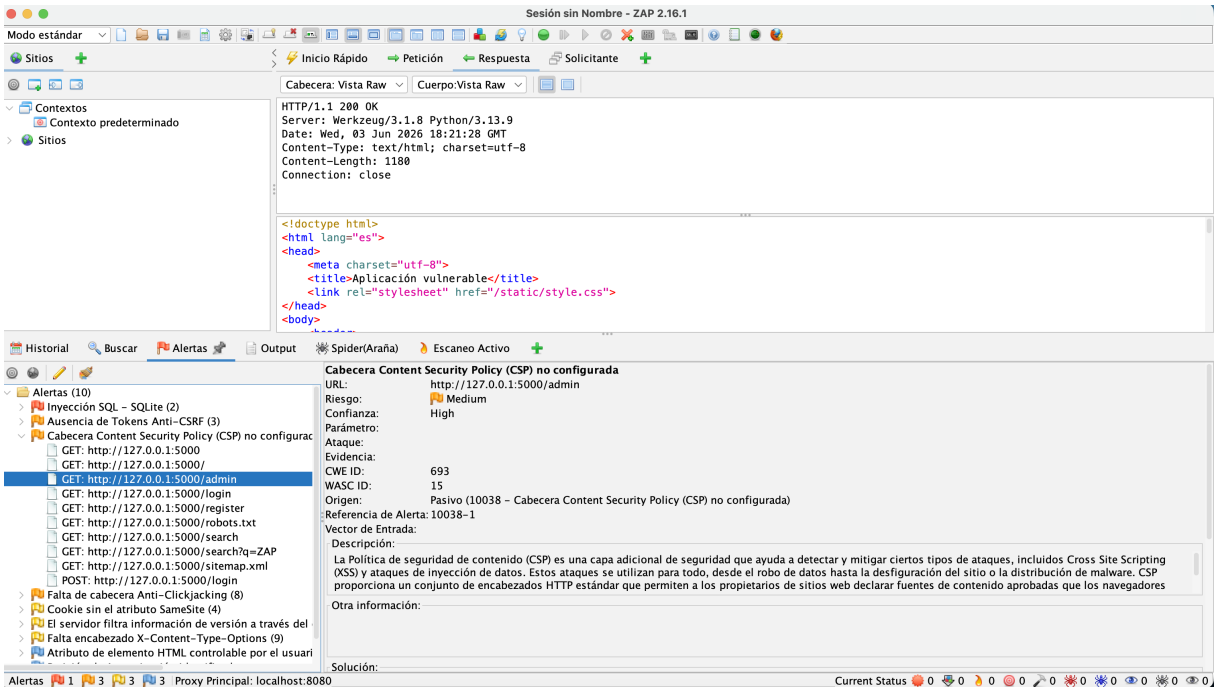



Figura 6.13: Alerta de cabecera Content-Security-Policy no configurada

En la Figura 6.14 se muestra la alerta relacionada con la ausencia de la cabecera `X-Content-Type-Options`.

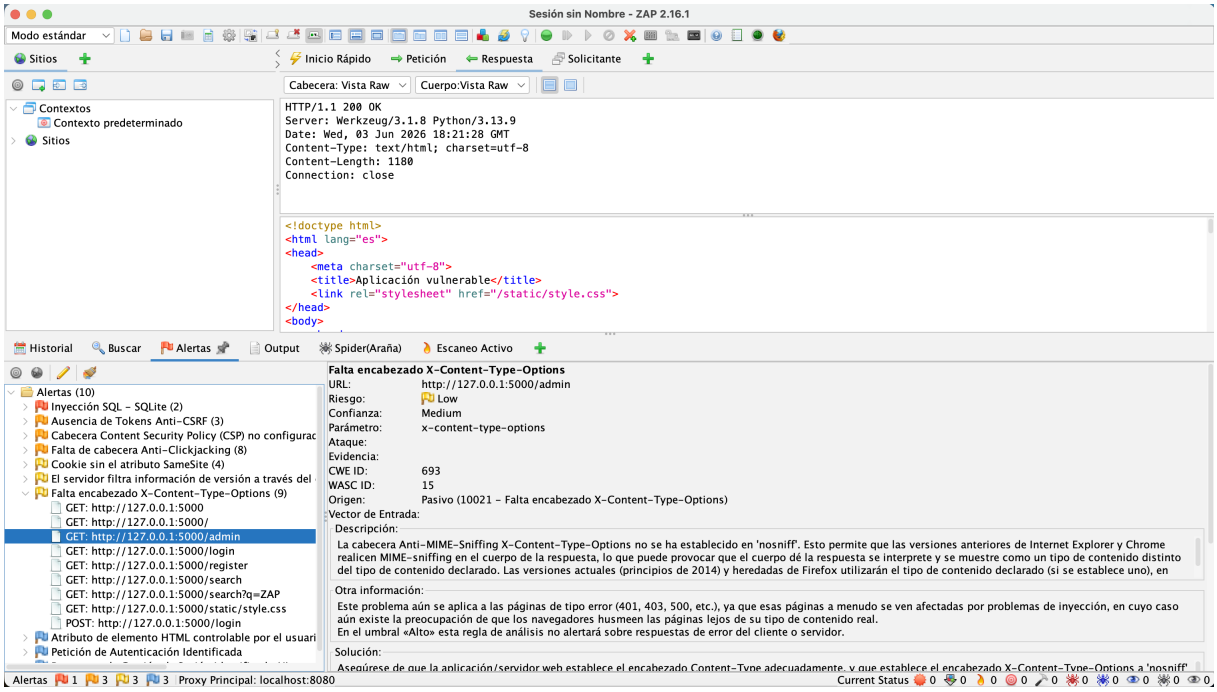



Figura 6.14: Alerta por ausencia de la cabecera X-Content-Type-Options

Por último, en la Figura 6.15 se recoge la alerta asociada a la falta de protección frente a clickjacking.

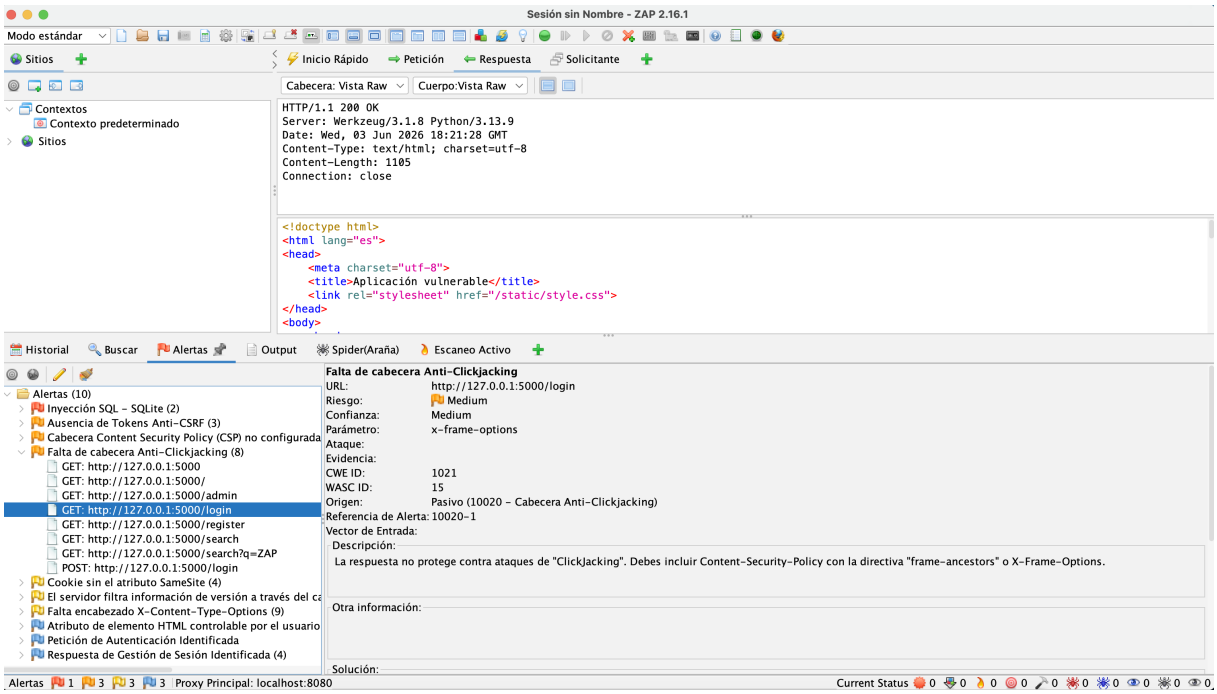



Figura 6.15: Alerta por falta de protección Anti-Clickjacking

Estos resultados muestran que OWASP ZAP es útil para detectar problemas de configuración que pueden pasar desapercibidos durante el desarrollo. En este caso, la causa está en que la aplicación Flask no añade explícitamente cabeceras de seguridad a las respuestas HTTP.

## 6.4. Resumen de resultados obtenidos

Una vez realizadas las pruebas con OWASP ZAP y las comprobaciones manuales desde el navegador, se obtuvo un conjunto de resultados que confirma la presencia de las vulnerabilidades introducidas de forma deliberada en la aplicación.

La Tabla 6.1 resume los principales hallazgos obtenidos, indicando el método de detección utilizado y la evidencia asociada en cada caso.

| **Vulnerabilidad** | **Método de detección** | **Evidencia principal** | **Resultado** |
|---|---|---|---|
| Inyección SQL en el login | OWASP ZAP y comprobación manual | Alerta de ZAP y acceso mediante carga SQL | Confirmada. La consulta de autenticación podía modificarse mediante la entrada del usuario. |
| XSS reflejado en la búsqueda | Comprobación manual desde navegador | Ejecución de una alerta JavaScript en el parámetro `q` | Confirmada. El valor introducido se reflejaba en la respuesta HTML sin escape adecuado. |
| Acceso al panel de administración sin sesión | Comprobación manual desde navegador | Acceso directo a `/admin` desde ventana de incógnito | Confirmada. La ruta permitía visualizar usuarios sin comprobar sesión ni rol. |
| Cabeceras HTTP de seguridad ausentes | OWASP ZAP mediante análisis pasivo | Alertas sobre `Content-Security-Policy`, `X-Content-Type-Options` y protección anti-clickjacking | Confirmada. La aplicación no añadía cabeceras de seguridad en las respuestas HTTP. |

Tabla 6.1: Resumen de resultados obtenidos durante las pruebas

Como se observa en la tabla, no todas las vulnerabilidades se identificaron de la misma forma. OWASP ZAP resultó útil para detectar la inyección SQL y los problemas de configuración relacionados con cabeceras HTTP. Sin embargo, el XSS reflejado y el fallo de control de acceso requirieron una comprobación manual para confirmar su impacto real.

Con este resumen se separa la explicación detallada del procedimiento, incluida en la sección anterior, de la síntesis final de resultados. De esta forma se evita repetir el mismo análisis y se deja una visión global de los hallazgos antes de clasificarlos según OWASP Top 10 y OWASP ASVS.

## 6.5. Clasificación de vulnerabilidades

Una vez revisados los resultados obtenidos, las vulnerabilidades se clasifican utilizando como referencia OWASP Top 10 y OWASP ASVS. Esta clasificación permite relacionar los fallos observados en la aplicación con categorías y controles reconocidos, de forma que los resultados no queden únicamente como incidencias aisladas de la herramienta.

El objetivo de esta sección no es realizar una auditoría completa de cumplimiento, sino situar cada vulnerabilidad dentro de un marco de referencia que ayude a entender su importancia y las medidas que deberían aplicarse para evitarla.

### 6.5.1. Relación con OWASP Top 10

Las vulnerabilidades identificadas en la aplicación se relacionan con varias categorías de OWASP Top 10. Esta relación permite contextualizar los resultados dentro de una clasificación ampliamente utilizada en seguridad web.

La Tabla 6.2 muestra la relación entre las vulnerabilidades analizadas y las categorías correspondientes de OWASP Top 10.

| **Vulnerabilidad detectada** | **Categoría OWASP Top 10** | **Justificación** |
|---|---|---|
| Inyección SQL en el login | Injection | La aplicación construye una consulta SQL concatenando directamente los valores introducidos por el usuario, lo que permite modificar la lógica de autenticación. |
| XSS reflejado en la búsqueda | Injection | El parámetro de búsqueda se inserta en la respuesta HTML sin un tratamiento adecuado, permitiendo la ejecución de código JavaScript en el navegador. |
| Acceso al panel de administración sin sesión | Broken Access Control | La ruta `/admin` no comprueba si el usuario ha iniciado sesión ni si tiene rol de administrador antes de mostrar información restringida. |
| Ausencia de cabeceras HTTP de seguridad | Security Misconfiguration | La aplicación no define cabeceras como `Content-Security-Policy`, `X-Content-Type-Options` o `X-Frame-Options`, lo que refleja una configuración de seguridad incompleta. |

Tabla 6.2: Relación entre vulnerabilidades detectadas y OWASP Top 10

Como se puede observar, las vulnerabilidades no pertenecen todas a la misma categoría. Algunas están relacionadas con el tratamiento incorrecto de datos de entrada, como la inyección SQL y el XSS reflejado. Otras tienen que ver con decisiones de autorización, como el acceso al panel de administración, o con la configuración general de la aplicación, como la ausencia de cabeceras HTTP de seguridad.

Esta clasificación ayuda a ver que los problemas detectados no son casos aislados, sino ejemplos concretos de categorías de riesgo habituales en aplicaciones web.

### 6.5.2. Relación con OWASP ASVS

Además de OWASP Top 10, también se utiliza OWASP ASVS como referencia técnica para relacionar las vulnerabilidades con controles de seguridad más concretos. Mientras que OWASP Top 10 permite clasificar los riesgos de forma general, ASVS ayuda a identificar qué tipo de controles deberían haberse aplicado durante el diseño y la implementación.

La Tabla 6.3 resume la relación entre cada vulnerabilidad y el tipo de control de seguridad asociado en OWASP ASVS.

| **Vulnerabilidad detectada** | **Control relacionado en ASVS** | **Medida esperada** |
|---|---|---|
| Inyección SQL en el login | Validación de entradas y protección frente a inyecciones | Utilizar consultas parametrizadas y evitar construir sentencias SQL concatenando datos introducidos por el usuario. |
| XSS reflejado en la búsqueda | Codificación de salida y gestión segura de datos no confiables | Escapar correctamente los datos antes de incluirlos en una respuesta HTML y evitar que el navegador interprete entradas del usuario como código. |
| Acceso al panel de administración sin sesión | Control de acceso y autorización | Verificar que el usuario está autenticado y que dispone del rol adecuado antes de permitir el acceso a rutas restringidas. |
| Ausencia de cabeceras HTTP de seguridad | Configuración segura de respuestas HTTP | Añadir cabeceras de seguridad como `Content-Security-Policy`, `X-Content-Type-Options` y `X-Frame-Options`. |

Tabla 6.3: Relación entre vulnerabilidades detectadas y controles de OWASP ASVS

Esta relación permite extraer una conclusión clara: las vulnerabilidades detectadas podrían haberse evitado aplicando controles básicos de desarrollo seguro. En el caso de la inyección SQL, el problema no está en SQLite ni en Flask, sino en la forma de construir la consulta. En el XSS reflejado, el fallo aparece al devolver contenido del usuario sin escaparlo. En el control de acceso, el problema está en no comprobar permisos antes de mostrar información sensible. Y, en el caso de las cabeceras HTTP, la causa está en no definir una configuración mínima de seguridad para las respuestas.

Por tanto, ASVS resulta útil para pasar del resultado técnico a una recomendación concreta. Esta conexión será la base del capítulo siguiente, donde se propondrán buenas prácticas para evitar que este tipo de vulnerabilidades aparezcan durante el desarrollo.

## 6.6. Conclusión del capítulo

En este capítulo se ha descrito la implementación de la aplicación web vulnerable y la ejecución de las pruebas de seguridad realizadas con OWASP ZAP. La aplicación se ha desarrollado como un entorno local y controlado, con las funcionalidades necesarias para reproducir vulnerabilidades concretas y analizarlas posteriormente.

Durante la implementación se han incorporado de forma intencionada distintos fallos de seguridad: una inyección SQL en el formulario de inicio de sesión, un XSS reflejado en la funcionalidad de búsqueda, un problema de control de acceso en el panel de administración y la ausencia de varias cabeceras HTTP de seguridad. Estos fallos se han introducido de manera controlada para poder observar su comportamiento y comprobar su detección durante las pruebas.

Las pruebas realizadas han combinado el análisis automático de OWASP ZAP con comprobaciones manuales desde el navegador. Esta combinación ha sido importante porque no todas las vulnerabilidades se detectan de la misma forma. La inyección SQL y las cabeceras ausentes fueron identificadas por ZAP, mientras que el XSS reflejado y el acceso indebido al panel de administración se comprobaron de forma manual para confirmar su impacto real.

Los resultados obtenidos muestran que OWASP ZAP resulta útil para detectar varios de los problemas presentes en la aplicación, sobre todo aquellos relacionados con formularios, parámetros de entrada y cabeceras HTTP. Aun así, también se ha visto que no basta con aceptar las alertas de forma automática, ya que algunos fallos necesitan una revisión manual para entender realmente su impacto.

Por último, las vulnerabilidades detectadas se han relacionado con OWASP Top 10 y OWASP ASVS. Esta clasificación permite conectar los resultados técnicos con buenas prácticas y controles de seguridad concretos. A partir de estos resultados, en el siguiente capítulo se propondrán medidas para corregir o prevenir los problemas identificados durante el análisis.

# Capítulo 7

# Buenas prácticas y metodología de desarrollo seguro

## 7.1. Introducción

En el capítulo anterior se implementó la aplicación web vulnerable y se ejecutaron las pruebas de seguridad con OWASP ZAP. A partir de los resultados obtenidos, en este capítulo se propone un conjunto de buenas prácticas orientadas a evitar los fallos detectados durante el análisis.

La finalidad de este capítulo no es presentar una lista genérica de recomendaciones de seguridad, sino relacionar cada buena práctica con los problemas observados en la aplicación desarrollada. De esta forma, las medidas propuestas parten de casos concretos: la inyección SQL en el formulario de inicio de sesión, el XSS reflejado en la funcionalidad de búsqueda, el fallo de control de acceso en el panel de administración y la ausencia de cabeceras HTTP de seguridad.

El enfoque seguido parte de una idea sencilla: la seguridad debe tenerse en cuenta desde las primeras fases del desarrollo y no únicamente al final, cuando la aplicación ya está implementada. Herramientas como OWASP ZAP son útiles para detectar problemas, pero no sustituyen a un diseño seguro ni a una implementación cuidadosa.

Por este motivo, el capítulo se divide en varias partes. En primer lugar, se proponen buenas prácticas concretas para cada vulnerabilidad detectada. A continuación, se describen las modificaciones que deberían realizarse en el código para corregir los fallos observados en la aplicación. Finalmente, se plantea una metodología sencilla de desarrollo seguro, pensada para proyectos web de pequeño y mediano tamaño, en la que se integran actividades de diseño, implementación, pruebas y revisión.

## 7.2. Buenas prácticas derivadas del análisis realizado

Las vulnerabilidades detectadas en la aplicación permiten identificar varias prácticas que deberían aplicarse durante el desarrollo de aplicaciones web. Aunque el entorno utilizado en este trabajo es sencillo, los problemas analizados son habituales en aplicaciones reales y pueden tener un impacto importante si aparecen en sistemas expuestos.

En este apartado se relaciona cada vulnerabilidad observada con una serie de medidas

preventivas. La idea es pasar del fallo concreto a la práctica de desarrollo que habría permitido evitarlo. Así, el análisis realizado con OWASP ZAP y las comprobaciones manuales no queda solo como una fase de detección, sino que sirve como base para definir mejoras aplicables durante el desarrollo.

### 7.2.1. Prevención de inyección SQL

La inyección SQL detectada en el formulario de inicio de sesión se produjo porque la consulta a la base de datos se construía concatenando directamente los valores introducidos por el usuario. Este tipo de implementación permite que una entrada especialmente preparada modifique la lógica de la consulta y altere el comportamiento previsto de la aplicación.

La medida principal para evitar este problema consiste en utilizar consultas parametrizadas. Con este enfoque, los valores introducidos por el usuario no se incorporan directamente a la sentencia SQL, sino que se tratan como parámetros. De esta forma, aunque el usuario introduzca caracteres especiales o fragmentos de código SQL, la base de datos los interpreta como datos y no como parte de la consulta.

Un ejemplo de consulta insegura sería el siguiente:

```
query = "SELECT * FROM users WHERE username = '" + username +
        "' AND password = '" + password + "'"
```

En cambio, una forma más segura sería utilizar parámetros:

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
cursor.execute(query, (username, password))
```

Además del uso de consultas parametrizadas, también es recomendable aplicar validaciones básicas sobre los datos recibidos. Por ejemplo, limitar la longitud de los campos, comprobar que el nombre de usuario cumple el formato esperado y evitar mensajes de error que revelen detalles internos de la base de datos. Estas medidas no sustituyen a las consultas parametrizadas, pero ayudan a reducir la superficie de ataque.

También debe tenerse en cuenta que la autenticación no debería basarse únicamente en comparar contraseñas en texto plano. En una aplicación real, las contraseñas deberían almacenarse utilizando funciones de hash seguras y mecanismos de salado. Aunque este aspecto no era el objetivo principal de la vulnerabilidad analizada, forma parte de una implementación segura del inicio de sesión.

En resumen, para prevenir la inyección SQL se deberían aplicar las siguientes medidas:

- Utilizar siempre consultas parametrizadas o mecanismos equivalentes del framework.
- Evitar la concatenación directa de datos introducidos por el usuario en sentencias SQL.
- Validar el formato y la longitud de los campos recibidos.
- No mostrar mensajes de error internos de la base de datos al usuario final.
- Almacenar las contraseñas mediante funciones de hash seguras en aplicaciones reales.

### 7.2.2. Prevención de XSS reflejado

El XSS reflejado comprobado en la funcionalidad de búsqueda se produjo porque el valor recibido en el parámetro `q` se devolvía dentro de la respuesta HTML sin tratarse correctamente. Como consecuencia, el navegador podía interpretar parte de la entrada del usuario como código JavaScript.

La medida principal para evitar este problema consiste en escapar siempre los datos antes de mostrarlos en una página HTML. Esto significa que los caracteres con significado especial para el navegador deben convertirse en texto seguro, de forma que se muestren como contenido y no se ejecuten como código.

En una aplicación desarrollada con plantillas, lo recomendable es aprovechar el sistema de escape automático del motor de plantillas. Por ejemplo, en Flask se utiliza normalmente Jinja2, que escapa por defecto las variables insertadas en las plantillas HTML. El problema aparece cuando se construyen respuestas manualmente o se marca contenido como seguro sin haberlo validado previamente.

Un ejemplo inseguro sería generar directamente una respuesta HTML con el valor introducido por el usuario:

```
return "<p>Has buscado: " + q + "</p>"
```

Una forma más segura sería utilizar una plantilla y dejar que el motor de plantillas trate el valor como texto:

```
return render_template("search.html", query=q)
```

Además del escape de salida, es recomendable validar los datos de entrada. En el caso de una búsqueda, por ejemplo, se puede limitar la longitud del término introducido y rechazar valores que no tengan sentido para esa funcionalidad. Aun así, la validación de entrada no sustituye al escape de salida, ya que incluso una entrada aparentemente válida puede acabar siendo peligrosa si se inserta en un contexto incorrecto.

Otra medida útil es definir una cabecera `Content-Security-Policy`. Esta cabecera no elimina por sí sola la vulnerabilidad, pero puede reducir el impacto de ciertos ataques XSS al limitar qué scripts puede ejecutar el navegador. Por tanto, debe entenderse como una capa adicional de protección, no como una solución única.

En resumen, para prevenir vulnerabilidades de XSS reflejado se deberían aplicar las siguientes medidas:

- Escapar siempre los datos antes de incluirlos en una respuesta HTML.
- Utilizar plantillas con escape automático en lugar de construir HTML mediante concatenación.
- Evitar marcar contenido como seguro si procede de entradas del usuario.
- Validar la longitud y el formato de los parámetros recibidos.
- Configurar una política `Content-Security-Policy` como capa adicional de defensa.

### 7.2.3. Control de acceso y autorización

El fallo de control de acceso se observó en el panel de administración. La ruta `/admin` mostraba información de usuarios registrados sin comprobar si quien accedía había iniciado sesión ni si tenía rol de administrador. Esto permitió acceder al panel desde una ventana de incógnito, simplemente conociendo la ruta.

La primera medida para evitar este tipo de problema es comprobar siempre la autenticación antes de permitir el acceso a una zona privada. Es decir, antes de mostrar cualquier contenido restringido, la aplicación debe verificar que existe una sesión válida asociada al usuario.

Además de comprobar la autenticación, también es necesario comprobar la autorización. No todos los usuarios autenticados deben tener acceso a las mismas funcionalidades. En este caso, el panel de administración debería estar disponible únicamente para usuarios con rol `admin`. Por tanto, no basta con saber que el usuario ha iniciado sesión; también hay que comprobar qué permisos tiene.

Un ejemplo de comprobación básica sería el siguiente:

```
if "user_id" not in session:
    return redirect("/login")

if session.get("role") != "admin":
    return "Acceso denegado", 403
```

También es importante que estas comprobaciones se realicen en el servidor. Ocultar un enlace en el menú de navegación no es suficiente, ya que un usuario puede escribir directamente la URL en el navegador. La autorización debe aplicarse en cada ruta protegida, independientemente de que el enlace sea visible o no en la interfaz.

Otra práctica recomendable es aplicar el principio de mínimo privilegio. Cada usuario debería tener únicamente los permisos necesarios para realizar sus tareas. De esta forma, aunque una cuenta normal sea comprometida, el impacto queda limitado y no se concede acceso innecesario a funcionalidades administrativas.

En resumen, para evitar fallos de control de acceso se deberían aplicar las siguientes medidas:

- Comprobar que existe una sesión válida antes de acceder a rutas privadas.
- Verificar el rol o los permisos del usuario antes de mostrar funcionalidades restringidas.
- Realizar las comprobaciones de acceso siempre en el servidor.
- No confiar únicamente en ocultar enlaces o botones en la interfaz.
- Aplicar el principio de mínimo privilegio a los distintos tipos de usuario.

### 7.2.4. Configuración segura de cabeceras HTTP

Durante el análisis con OWASP ZAP se detectó la ausencia de varias cabeceras HTTP de seguridad. En concreto, la herramienta generó alertas relacionadas con la falta de una

política `Content-Security-Policy`, la ausencia de la cabecera `X-Content-Type-Options` y la falta de protección frente a clickjacking.

Estas cabeceras no corrigen por sí solas vulnerabilidades como la inyección SQL o el control de acceso roto, pero sí ayudan a reducir ciertos riesgos en el navegador. Por este motivo, deben entenderse como una capa adicional dentro de la seguridad de la aplicación.

Una primera medida consiste en definir una política `Content-Security-Policy`. Esta cabecera permite indicar al navegador qué fuentes de contenido están permitidas, por ejemplo, desde dónde se pueden cargar scripts, hojas de estilo o imágenes. En una aplicación sencilla como la desarrollada en este trabajo, se podría empezar con una política restrictiva y ajustarla después según las necesidades reales de la interfaz.

Otra cabecera importante es `X-Content-Type-Options`. Al establecer el valor `nosniff`, se evita que el navegador intente interpretar un recurso con un tipo de contenido distinto al declarado por el servidor. Esta medida ayuda a reducir problemas derivados de una interpretación incorrecta de archivos o respuestas.

También es recomendable añadir una protección frente a clickjacking. Para ello se puede utilizar la cabecera `X-Frame-Options` o una directiva equivalente dentro de la política CSP. Con esto se impide que la aplicación sea cargada dentro de un marco o iframe no autorizado.

En una aplicación Flask, estas cabeceras pueden añadirse de forma centralizada después de cada respuesta. De esta forma, no es necesario configurarlas ruta por ruta, sino que se aplican de manera uniforme a toda la aplicación.

En resumen, para mejorar la configuración de cabeceras HTTP se deberían aplicar las siguientes medidas:

- Definir una política `Content-Security-Policy` adecuada a la aplicación.
- Añadir la cabecera `X-Content-Type-Options` con el valor `nosniff`.
- Incluir protección frente a clickjacking mediante `X-Frame-Options` o una directiva equivalente en la política CSP.
- Configurar las cabeceras de forma centralizada para todas las respuestas.
- Revisar de nuevo la aplicación con OWASP ZAP para comprobar que las alertas desaparecen o se reducen.

## 7.3. Corrección de las vulnerabilidades detectadas

### 7.3.1. Corrección de la inyección SQL

La primera corrección afecta al formulario de inicio de sesión. En la versión vulnerable, la consulta SQL se construía concatenando directamente los valores introducidos por el usuario. Este comportamiento fue el que permitió modificar la lógica de autenticación mediante una entrada preparada.

De forma simplificada, el código vulnerable seguía una estructura como la siguiente:

```
query = "SELECT * FROM users WHERE username = '" + username +
        "' AND password = '" + password + "'"
cursor.execute(query)
```

La corrección consiste en sustituir esta construcción por una consulta parametrizada. Con este cambio, los valores recibidos desde el formulario se pasan como datos y no se interpretan como parte de la sentencia SQL.

Una versión corregida sería la siguiente:

```
query = "SELECT * FROM users WHERE username = ? AND password = ?"
cursor.execute(query, (username, password))
```

Con esta modificación, aunque el usuario introduzca caracteres especiales o fragmentos de SQL, la base de datos los tratará como valores normales. Por tanto, la entrada del usuario ya no podrá alterar la estructura de la consulta.

Además, en una aplicación real también debería modificarse la forma de almacenar y comprobar las contraseñas. En lugar de guardar contraseñas en texto plano, sería necesario almacenarlas mediante una función de hash segura y comparar el hash durante el inicio de sesión. Esta mejora no corrige directamente la inyección SQL, pero sí forma parte de una autenticación más segura.

Por tanto, la corrección principal consiste en eliminar la concatenación de cadenas en las consultas SQL y utilizar siempre parámetros. Es un cambio pequeño a nivel de código, pero evita que el formulario de login pueda utilizarse para manipular la consulta de autenticación.

### 7.3.2. Corrección del XSS reflejado

La segunda corrección afecta a la funcionalidad de búsqueda. En la versión vulnerable, el valor recibido en el parámetro `q` se insertaba directamente en la respuesta HTML. Esto permitía que el navegador interpretara parte de la entrada del usuario como código JavaScript, dando lugar al XSS reflejado comprobado en el capítulo anterior.

De forma simplificada, el comportamiento vulnerable podía representarse así:

```
q = request.args.get("q", "")
return "<p>Has buscado: " + q + "</p>"
```

La corrección pasa por no construir la respuesta HTML uniendo directamente texto con datos introducidos por el usuario. En su lugar, se debe utilizar una plantilla y dejar que el motor de plantillas aplique el escape correspondiente.

Una versión corregida sería la siguiente:

```
q = request.args.get("q", "")
return render_template("search.html", query=q)
```

En la plantilla, el valor se mostraría como una variable normal:

```
<p>Has buscado: {{ query }}</p>
```

Con esta modificación, Jinja2 trata el valor como texto y escapa los caracteres especiales antes de incluirlos en la respuesta HTML. De esta forma, si el usuario introduce una cadena con etiquetas o código JavaScript, el navegador no la ejecutará como código, sino que la mostrará como texto.

Además, sería recomendable limitar la longitud del parámetro de búsqueda y rechazar valores vacíos o excesivamente largos. Esta validación no sustituye al escape de salida, pero ayuda a reducir entradas inesperadas y mejora el comportamiento general de la aplicación.

Por tanto, la corrección principal consiste en no concatenar directamente la entrada del usuario dentro del HTML y utilizar plantillas con escape automático. Con este cambio se elimina la causa del XSS reflejado que se vio durante las pruebas.

### 7.3.3. Corrección del control de acceso

La tercera corrección afecta al panel de administración. En la versión vulnerable, la ruta `/admin` mostraba el listado de usuarios sin comprobar antes si había una sesión iniciada ni si el usuario tenía rol de administrador. Por eso era posible acceder al panel directamente desde una ventana de incógnito.

De forma simplificada, el problema estaba en una ruta como la siguiente:

```
@app.route("/admin")
def admin():
    conn = get_db_connection()
    users = conn.execute("SELECT * FROM users").fetchall()
    conn.close()
    return render_template("admin.html", users=users)
```

En este caso, la aplicación consulta la base de datos y muestra la página sin hacer ninguna comprobación previa. Para corregirlo, primero habría que comprobar que el usuario ha iniciado sesión y, después, verificar que su rol es `admin`.

Una versión corregida podría ser la siguiente:

```
@app.route("/admin")
def admin():
    if "user_id" not in session:
        return redirect("/login")

    if session.get("role") != "admin":
        return "Acceso denegado", 403

    conn = get_db_connection()
    users = conn.execute("SELECT * FROM users").fetchall()
    conn.close()
    return render_template("admin.html", users=users)
```

Con este cambio, un usuario no autenticado ya no podría acceder directamente al panel de administración. En ese caso, sería redirigido al formulario de inicio de sesión. Además, si un usuario autenticado no tiene rol de administrador, la aplicación devolvería una respuesta de acceso denegado.

Esta corrección también muestra una idea importante: ocultar el enlace al panel en la barra de navegación no sería suficiente. Aunque el enlace no aparezca en la interfaz, cualquier usuario podría escribir la ruta manualmente en el navegador. Por eso la comprobación debe hacerse siempre en el servidor, justo antes de mostrar el contenido protegido.

En una aplicación más grande, esta lógica se podría mover a un decorador o a una función común para no repetir las mismas comprobaciones en varias rutas. En este caso, al tratarse de una aplicación pequeña, basta con mostrar la comprobación directamente en la ruta afectada.

Con esta modificación se corrige el fallo de control de acceso observado durante las pruebas, ya que el panel de administración queda limitado a usuarios autenticados con el rol adecuado.

### 7.3.4. Corrección de las cabeceras HTTP de seguridad

La cuarta corrección afecta a la configuración general de las respuestas HTTP. Durante el análisis con OWASP ZAP se comprobó que la aplicación no añadía varias cabeceras de seguridad, como `Content-Security-Policy`, `X-Content-Type-Options` o `X-Frame-Options`. Este problema no estaba en una ruta concreta, sino en la forma general en la que Flask devolvía las respuestas.

En una aplicación Flask, estas cabeceras pueden añadirse de forma centralizada utilizando `after_request`. Esta función permite modificar cada respuesta antes de enviarla al navegador, por lo que evita tener que repetir la misma configuración en cada ruta.

Una posible corrección sería la siguiente:

```
@app.after_request
def add_security_headers(response):
    response.headers["Content-Security-Policy"] = (
        "default-src 'self'"
    )
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Referrer-Policy"] = (
        "strict-origin-when-cross-origin"
    )
    return response
```

Con este cambio, todas las respuestas de la aplicación incluirían una configuración mínima de seguridad. La política CSP limita la carga de recursos al propio origen de la aplicación, `X-Content-Type-Options` evita que el navegador intente interpretar recursos con un tipo distinto al declarado, y `X-Frame-Options` impide que la página se cargue dentro de un marco no autorizado.

La cabecera `Referrer-Policy` no apareció como una de las alertas principales del análisis, pero se incluye como una mejora adicional razonable. Su objetivo es limitar la

información que el navegador envía en la cabecera `Referer` cuando el usuario navega desde la aplicación hacia otros sitios.

En una aplicación real, la política CSP tendría que ajustarse con más detalle. Por ejemplo, si la interfaz cargara estilos, imágenes o scripts desde dominios externos, habría que permitirlos de forma explícita. En este trabajo, como la aplicación es sencilla y funciona en local, una política basada en `default-src 'self'` resulta suficiente como propuesta inicial.

Después de aplicar esta corrección, lo adecuado sería repetir el análisis con OWASP ZAP para comprobar si desaparecen o se reducen las alertas relacionadas con cabeceras HTTP. De esta forma, no solo se propone el cambio en el código, sino que también se mantiene el mismo criterio de validación usado durante las pruebas.

## 7.4. Propuesta de metodología de desarrollo seguro

A partir de las vulnerabilidades analizadas y de las correcciones planteadas, se puede proponer una metodología sencilla de desarrollo seguro para aplicaciones web pequeñas o medianas. La idea no es definir un proceso complejo, sino recoger una forma de trabajo que ayude a tener en cuenta la seguridad desde el principio y no solo cuando la aplicación ya está terminada.

La metodología propuesta se basa en cuatro momentos principales: diseño, implementación, pruebas y revisión. Estos momentos coinciden con las fases habituales de desarrollo, pero incorporando en cada una de ellas una serie de comprobaciones de seguridad. De esta forma, la seguridad no queda como una tarea separada, sino integrada dentro del propio proceso de desarrollo.

### 7.4.1. Seguridad desde el diseño

La primera fase de la metodología consiste en tener en cuenta la seguridad desde el diseño de la aplicación. Antes de escribir el código, conviene identificar qué funcionalidades pueden manejar datos sensibles, qué rutas deberían estar protegidas y qué puntos de entrada recibirán información del usuario.

En el caso de la aplicación desarrollada en este trabajo, esta fase habría permitido detectar desde el principio varios puntos críticos: el formulario de inicio de sesión, la funcionalidad de búsqueda, el panel de administración y la configuración general de las respuestas HTTP. Todos ellos acabaron siendo relevantes durante el análisis con OWASP ZAP.

Una práctica útil en esta fase es relacionar cada funcionalidad con los riesgos que puede introducir. Por ejemplo, un formulario de login debe revisarse pensando en inyección SQL y autenticación segura; una búsqueda debe revisarse pensando en XSS; y una zona de administración debe diseñarse con controles claros de autenticación y autorización.

También es recomendable definir desde el diseño qué usuarios existen y qué permisos tiene cada uno. En la aplicación vulnerable, el panel de administración mostraba información restringida sin comprobar el rol del usuario. Si esta restricción se hubiera definido claramente en el diseño, habría sido más difícil dejar la ruta sin protección durante la implementación.

Por tanto, en esta primera fase se deberían realizar al menos las siguientes tareas:

- Identificar las funcionalidades que reciben datos del usuario.
- Definir qué rutas son públicas y cuáles deben requerir autenticación.
- Establecer los roles de usuario y los permisos asociados a cada uno.
- Prever qué cabeceras HTTP de seguridad debería incluir la aplicación.
- Relacionar cada punto crítico con categorías de riesgo conocidas, como OWASP Top 10.

Con estas decisiones tomadas antes de programar, se reduce la probabilidad de introducir fallos básicos durante la implementación. Además, el diseño deja de centrarse solo en que la aplicación funcione y empieza a considerar también cómo puede fallar desde el punto de vista de la seguridad.

### 7.4.2. Implementación segura

La segunda fase de la metodología se centra en la implementación. En esta parte ya se escribe el código de la aplicación, por lo que las decisiones tomadas en el diseño deben traducirse en comprobaciones y mecanismos concretos.

Uno de los puntos más importantes es tratar siempre los datos introducidos por el usuario como datos no confiables. Esto afecta a formularios, parámetros de URL, campos de búsqueda y cualquier otra entrada recibida desde el navegador. En la aplicación analizada, tanto la inyección SQL como el XSS reflejado aparecieron precisamente por no tratar correctamente estos datos.

En el caso del acceso a la base de datos, la práctica principal debe ser utilizar consultas parametrizadas. No debería construirse una consulta SQL uniendo cadenas con valores introducidos por el usuario. Aunque esta forma pueda parecer más rápida en aplicaciones pequeñas, acaba introduciendo un riesgo importante.

También es importante separar la lógica de presentación de la lógica de negocio. En el caso del XSS reflejado, el problema aparecía porque la respuesta HTML se generaba directamente con una cadena. Utilizar plantillas ayuda a mantener el código más ordenado y permite aprovechar el escape automático que ofrece el motor de plantillas.

Otro aspecto clave es aplicar las comprobaciones de acceso dentro del propio servidor. Las rutas privadas deben comprobar siempre si el usuario ha iniciado sesión y si tiene los permisos adecuados. Esta lógica no debe depender de que un enlace esté visible u oculto en la interfaz, ya que cualquier usuario puede intentar acceder escribiendo la URL manualmente.

Además, la configuración de seguridad debería centralizarse siempre que sea posible. Por ejemplo, las cabeceras HTTP de seguridad pueden añadirse una sola vez para todas las respuestas de la aplicación, evitando depender de configuraciones repetidas en cada ruta.

En esta fase, las prácticas principales serían las siguientes:

- Usar consultas parametrizadas para todas las operaciones con la base de datos.

- Evitar construir respuestas HTML concatenando directamente datos del usuario.
- Utilizar plantillas con escape automático para mostrar información en la interfaz.
- Comprobar autenticación y autorización en todas las rutas privadas.
- Centralizar la configuración de cabeceras HTTP de seguridad.
- Evitar mostrar errores internos de la aplicación al usuario final.

De esta forma, la implementación no se limita a que las funcionalidades funcionen correctamente, sino que también incorpora medidas básicas para evitar vulnerabilidades habituales.

### 7.4.3. Pruebas de seguridad durante el desarrollo

La tercera fase de la metodología consiste en realizar pruebas de seguridad durante el desarrollo, y no solo al final del proyecto. En aplicaciones web, muchos fallos pueden detectarse antes de que el sistema esté terminado si se revisan los puntos de entrada principales a medida que se van implementando.

En este trabajo, OWASP ZAP se utilizó una vez que la aplicación vulnerable ya estaba construida. Sin embargo, en un proyecto real sería recomendable usar este tipo de herramienta de forma periódica. Por ejemplo, después de implementar el login, la búsqueda o el panel de administración, se podrían ejecutar pruebas concretas sobre esas funcionalidades antes de continuar con el resto del desarrollo.

Las pruebas automáticas son útiles porque ayudan a detectar problemas que pueden pasar desapercibidos durante la programación. En el caso analizado, OWASP ZAP permitió identificar la inyección SQL y varias alertas relacionadas con cabeceras HTTP de seguridad. Esto demuestra que una herramienta de análisis dinámico puede servir como apoyo para localizar errores de implementación y configuración.

Aun así, estas herramientas no deben utilizarse como única forma de validación. Algunas vulnerabilidades dependen de la lógica interna de la aplicación y necesitan una revisión manual. El fallo de control de acceso del panel de administración es un buen ejemplo, ya que se confirmó accediendo directamente a la ruta `/admin` desde una ventana de incógnito.

Por tanto, la fase de pruebas debería combinar análisis automático y comprobaciones manuales. Las pruebas automáticas permiten cubrir más superficie de la aplicación, mientras que la revisión manual ayuda a interpretar los resultados y a confirmar si el comportamiento observado supone realmente un problema de seguridad.

En esta fase se proponen las siguientes tareas:

- Ejecutar pruebas con OWASP ZAP sobre las funcionalidades principales.
- Revisar formularios y parámetros que reciban datos del usuario.
- Comprobar manualmente las rutas privadas y los controles de acceso.
- Revisar las cabeceras HTTP devueltas por la aplicación.

- Analizar las alertas de la herramienta antes de aceptarlas como definitivas.
- Repetir las pruebas después de aplicar correcciones en el código.

Con este enfoque, las pruebas de seguridad dejan de ser una actividad puntual al final del proyecto y pasan a formar parte del desarrollo normal de la aplicación. Esto permite detectar antes los problemas y corregirlos cuando todavía son más fáciles de solucionar.

### 7.4.4. Revisión y mejora continua

La cuarta fase de la metodología consiste en revisar lo que se ha hecho y utilizar los resultados para mejorar la aplicación. La seguridad no debería entenderse como algo que se comprueba una sola vez, sino como un proceso que se va ajustando a medida que la aplicación cambia.

En el caso de este trabajo, las pruebas permitieron detectar fallos concretos y, a partir de ellos, plantear correcciones en el código. Este mismo enfoque podría aplicarse en un proyecto real: analizar la aplicación, corregir los problemas encontrados y volver a probar para comprobar que las vulnerabilidades se han solucionado.

Esta revisión también sirve para aprender de los fallos detectados. Por ejemplo, la inyección SQL muestra la importancia de usar consultas parametrizadas desde el principio; el XSS reflejado recuerda la necesidad de escapar la salida HTML; y el fallo de control de acceso demuestra que las comprobaciones de permisos deben hacerse siempre en el servidor.

Además, conviene mantener una relación entre las vulnerabilidades encontradas y las buenas prácticas que las corrigen. De esta forma, los resultados de una prueba no quedan solo como incidencias aisladas, sino que ayudan a mejorar la forma de desarrollar nuevas funcionalidades en el futuro.

En esta fase se proponen las siguientes tareas:

- Revisar las vulnerabilidades detectadas y analizar su causa.
- Aplicar las correcciones necesarias en el código.
- Repetir las pruebas para comprobar que los problemas se han corregido.
- Actualizar las buenas prácticas del proyecto a partir de los fallos encontrados.
- Documentar las decisiones de seguridad más importantes.

Con esta última fase, la metodología no termina en la detección de vulnerabilidades. El objetivo es que cada revisión sirva para mejorar la aplicación y para reducir la probabilidad de repetir los mismos errores en desarrollos posteriores.

## 7.5. Relación con OWASP ASVS

Las buenas prácticas y correcciones propuestas en este capítulo también pueden relacionarse con OWASP ASVS. Esta relación es útil porque permite comprobar que las

medidas planteadas no son solo soluciones aisladas para la aplicación desarrollada, sino que encajan con controles de seguridad reconocidos para aplicaciones web.

En este trabajo no se pretende realizar una evaluación completa de cumplimiento de ASVS. La aplicación creada es un entorno de pruebas sencillo y vulnerable de forma intencionada, por lo que no tendría sentido tratarla como una aplicación preparada para producción. Aun así, ASVS sirve como referencia para entender qué tipo de controles deberían aplicarse si se quisiera desarrollar una versión segura de la aplicación.

La Tabla 7.1 resume la relación entre las vulnerabilidades analizadas, las correcciones propuestas y el tipo de control asociado en OWASP ASVS.

| **Vulnerabilidad** | **Corrección propuesta** | **Relación con ASVS** |
|---|---|---|
| Inyección SQL en el login | Uso de consultas parametrizadas y eliminación de la concatenación directa de datos del usuario en sentencias SQL. | Controles relacionados con validación de entradas y protección frente a inyecciones. |
| XSS reflejado en la búsqueda | Uso de plantillas con escape automático y evitar construir respuestas HTML concatenando datos del usuario. | Controles relacionados con codificación de salida y tratamiento seguro de datos no confiables. |
| Acceso indebido al panel de administración | Comprobación de sesión y rol antes de mostrar rutas privadas como `/admin`. | Controles relacionados con autenticación, autorización y control de acceso en el servidor. |
| Cabeceras HTTP de seguridad ausentes | Configuración centralizada de cabeceras como CSP, `X-Content-Type-Options` y `X-Frame-Options`. | Controles relacionados con configuración segura de respuestas HTTP y protección adicional en el navegador. |

Tabla 7.1: Relación entre las correcciones propuestas y OWASP ASVS

Como se observa en la tabla, ASVS ayuda a pasar de una corrección concreta a una categoría de control más general. Por ejemplo, corregir la consulta vulnerable del login no es solo arreglar una línea de código, sino aplicar una práctica relacionada con la protección frente a inyecciones. Del mismo modo, proteger la ruta `/admin` no consiste únicamente en añadir una condición concreta, sino en aplicar un control de autorización en el servidor.

Esta relación también permite reforzar la metodología propuesta en la sección anterior. Durante el diseño se pueden revisar los controles de ASVS para identificar requisitos de seguridad; durante la implementación se pueden usar como guía para aplicar buenas prácticas; y durante las pruebas pueden servir como referencia para comprobar si la aplicación cumple unos mínimos razonables.

Por tanto, OWASP ASVS funciona en este trabajo como un apoyo para transformar los resultados del análisis en medidas de desarrollo seguro. No sustituye a las pruebas con OWASP ZAP ni a la revisión manual, pero ayuda a ordenar las correcciones y a relacionarlas con controles técnicos reconocidos.

## 7.6. Conclusión del capítulo

En este capítulo se han propuesto buenas prácticas y correcciones a partir de las vulnerabilidades detectadas en la aplicación desarrollada. El objetivo no ha sido plantear

recomendaciones genéricas, sino relacionar cada medida con un problema concreto observado durante las pruebas.

En primer lugar, se han revisado las buenas prácticas asociadas a la inyección SQL, el XSS reflejado, el control de acceso y la configuración de cabeceras HTTP. Estas prácticas permiten entender qué decisiones de diseño e implementación habrían ayudado a evitar los fallos introducidos en la aplicación.

Después, se han planteado modificaciones concretas en el código para corregir las vulnerabilidades detectadas. En el caso de la inyección SQL, la corrección principal consiste en utilizar consultas parametrizadas. Para el XSS reflejado, la solución pasa por utilizar plantillas con escape automático y evitar construir HTML concatenando datos del usuario. En el panel de administración, la corrección consiste en comprobar la sesión y el rol antes de mostrar el contenido. Por último, las cabeceras HTTP de seguridad pueden añadirse de forma centralizada en las respuestas de Flask.

También se ha propuesto una metodología sencilla de desarrollo seguro basada en cuatro fases: diseño, implementación, pruebas y revisión. Esta metodología pretende integrar la seguridad dentro del proceso normal de desarrollo, en lugar de tratarla como una comprobación final una vez terminada la aplicación.

Finalmente, se ha relacionado la propuesta con OWASP ASVS. Esta relación permite situar las correcciones dentro de un marco técnico reconocido y refuerza la idea de que los fallos observados no son casos aislados, sino ejemplos concretos de controles de seguridad que deberían tenerse en cuenta en cualquier aplicación web.

En conjunto, el capítulo muestra que las vulnerabilidades detectadas no solo pueden corregirse después de encontrarlas, sino que muchas de ellas podrían haberse evitado si la seguridad se hubiera tenido en cuenta desde el diseño y durante la implementación. Esta idea conecta directamente con el objetivo principal del trabajo: utilizar el análisis de vulnerabilidades como base para proponer buenas prácticas de desarrollo seguro.

# Capítulo 8

# Conclusiones

## 8.1. Revisión de costes y planificación

En el capítulo 3 se realizó una estimación inicial del esfuerzo y del coste que tendría el proyecto en un entorno profesional. Aunque este TFG se ha desarrollado en un contexto académico y no ha supuesto un coste económico real, esa estimación sirve para comparar lo previsto al inicio con el trabajo que finalmente se ha realizado.

La planificación inicial planteaba un esfuerzo total de 300 horas, repartidas entre el diseño de la aplicación, la implementación, la configuración del entorno, las pruebas con OWASP ZAP, el análisis de resultados y la definición de buenas prácticas. Una vez finalizado el trabajo, se puede considerar que esta estimación ha sido bastante ajustada, ya que el alcance previsto se ha mantenido y las tareas principales se han completado.

Aun así, el reparto real del tiempo no ha sido exactamente igual al planteado al principio. La implementación de la aplicación vulnerable se mantuvo dentro de lo previsto, porque se trataba de una aplicación sencilla y ejecutada en local. En cambio, la parte de pruebas, revisión de resultados, obtención de capturas y redacción final acabó requiriendo más tiempo del esperado. Esto se debe a que no bastaba con ejecutar OWASP ZAP y copiar sus alertas, sino que había que interpretarlas, comprobar manualmente algunas vulnerabilidades y explicar los resultados de forma clara en la memoria.

La comparación entre la planificación inicial y el resultado real puede resumirse así:

| **Aspecto** | **Resultado final** |
|---|---|
| Esfuerzo total | Se estimaron 300 horas. El esfuerzo final ha sido similar, aunque con más tiempo dedicado a pruebas, revisión de resultados y redacción. |
| Coste estimado | El coste profesional estimado fue de 12350 euros. No se han producido desviaciones económicas relevantes. |
| Recursos utilizados | Se han usado herramientas gratuitas y ejecución en local: Python, Flask, SQLite y OWASP ZAP. |
| Alcance del proyecto | El alcance previsto se ha cumplido: aplicación vulnerable, análisis con OWASP ZAP, correcciones de código y propuesta de buenas prácticas. |

Tabla 8.1: Comparación entre la planificación inicial y el resultado real del proyecto

En cuanto al coste económico, el proyecto se ha mantenido dentro de lo previsto. La aplicación se ha ejecutado en local y se han utilizado herramientas gratuitas y de código abierto, como Python, Flask, SQLite y OWASP ZAP. Por tanto, no ha sido necesario contratar servicios externos, adquirir licencias ni desplegar infraestructura adicional.

En el capítulo 3 se estimó un coste total aproximado de 12350 euros en un entorno profesional. Tras finalizar el trabajo, no se han identificado desviaciones económicas importantes respecto a esa estimación. La principal diferencia ha estado en la distribución del tiempo dedicado a cada fase, no en los recursos necesarios para realizar el proyecto.

Por tanto, se puede concluir que la planificación inicial ha sido adecuada para el alcance real del TFG. El esfuerzo estimado representa de forma razonable el trabajo necesario para construir la aplicación vulnerable, analizarla con OWASP ZAP, revisar los resultados obtenidos y elaborar una propuesta de buenas prácticas y correcciones orientadas al desarrollo seguro.

## 8.2. Conclusiones

En este Trabajo de Fin de Grado se ha analizado la seguridad de aplicaciones web a partir de un caso práctico desarrollado en un entorno controlado. Para ello, se ha diseñado e implementado una aplicación web vulnerable con Flask y SQLite, incorporando de forma deliberada varios fallos habituales: inyección SQL, XSS reflejado, control de acceso incorrecto y ausencia de cabeceras HTTP de seguridad.

El objetivo no era desarrollar una aplicación completa para producción, sino construir un entorno sencillo que permitiera observar cómo aparecen estas vulnerabilidades y cómo pueden detectarse mediante una herramienta de análisis dinámico como OWASP ZAP. Este enfoque ha permitido relacionar conceptos teóricos de seguridad con ejemplos prácticos y comprobables.

Durante el trabajo se ha podido comprobar que OWASP ZAP resulta útil para identificar determinados problemas de seguridad, especialmente aquellos relacionados con formularios, parámetros de entrada y configuraciones inseguras de las respuestas HTTP. En el caso de la aplicación desarrollada, la herramienta permitió detectar la inyección SQL en el formulario de inicio de sesión y varias alertas relacionadas con cabeceras de seguridad ausentes.

Sin embargo, también se ha visto que el análisis automático no es suficiente por sí solo. Algunas vulnerabilidades necesitan una revisión manual para confirmar su impacto real. Esto se ha observado especialmente en el XSS reflejado y en el fallo de control de acceso del panel de administración. En estos casos, la comprobación directa desde el navegador fue necesaria para entender el comportamiento de la aplicación y confirmar que el problema era real.

Otro resultado importante del trabajo es la relación entre las vulnerabilidades detectadas y los marcos de referencia estudiados. OWASP Top 10 ha servido para clasificar los riesgos de forma general, mientras que OWASP ASVS ha permitido asociar cada fallo con controles de seguridad más concretos. Esta relación ayuda a entender que las vulnerabilidades no son problemas aislados, sino ejemplos de malas prácticas que pueden prevenirse durante el desarrollo.

A partir de los resultados obtenidos, se han propuesto buenas prácticas y correcciones concretas para cada vulnerabilidad. Entre ellas destacan el uso de consultas parametriza-

das para evitar la inyección SQL, el uso de plantillas con escape automático para prevenir XSS, la comprobación de sesión y rol en rutas privadas, y la configuración centralizada de cabeceras HTTP de seguridad en Flask.

Además, se ha planteado una metodología sencilla de desarrollo seguro basada en cuatro fases: diseño, implementación, pruebas y revisión. Esta metodología resume una idea central del trabajo: la seguridad debe incorporarse desde el inicio del desarrollo y no tratarse únicamente como una comprobación final una vez que la aplicación ya está construida.

En conjunto, se puede concluir que los objetivos planteados al inicio del TFG se han cumplido. Se ha revisado el estado del arte, se ha diseñado e implementado una aplicación vulnerable, se han ejecutado pruebas con OWASP ZAP, se han analizado los resultados obtenidos y se ha elaborado una propuesta de buenas prácticas orientada al desarrollo seguro de aplicaciones web.

Como conclusión final, el trabajo muestra que incluso una aplicación sencilla puede presentar vulnerabilidades relevantes si no se aplican controles básicos de seguridad. También demuestra que las herramientas automáticas son útiles, pero deben ir acompañadas de criterio técnico, revisión manual y buenas prácticas de desarrollo.

## 8.3. Trabajo futuro

Aunque el trabajo ha permitido cumplir los objetivos planteados, existen varias líneas de mejora que podrían abordarse como continuación del proyecto. Estas mejoras permitirían ampliar el análisis realizado y acercar la aplicación a un escenario más completo.

Una primera línea de trabajo futuro sería implementar realmente las correcciones propuestas en el capítulo 7. En este TFG se han explicado las modificaciones que deberían realizarse sobre el código vulnerable, pero una continuación natural sería aplicarlas en una segunda versión de la aplicación. De esta forma, se podría comparar la versión vulnerable con una versión corregida.

Después de aplicar esas correcciones, sería conveniente repetir el análisis con OWASP ZAP. Esto permitiría comprobar si las alertas detectadas inicialmente desaparecen o se reducen, y serviría para validar de forma práctica la eficacia de las medidas propuestas. Además, esta repetición del análisis permitiría mantener el mismo criterio utilizado durante las pruebas del capítulo 6.

Otra posible mejora sería ampliar el conjunto de vulnerabilidades estudiadas. En este trabajo se han analizado fallos representativos como la inyección SQL, el XSS reflejado, el control de acceso roto y la ausencia de cabeceras HTTP de seguridad. Sin embargo, podrían añadirse otros problemas habituales, como errores en la gestión de sesiones, exposición de información sensible, subida insegura de archivos o configuraciones incorrectas más avanzadas.

También sería interesante mejorar el sistema de autenticación de la aplicación. Por ejemplo, se podrían almacenar las contraseñas mediante funciones de hash seguras, añadir caducidad de sesión, controlar intentos repetidos de inicio de sesión o separar mejor los permisos de usuarios normales y administradores. Estas mejoras permitirían estudiar aspectos de seguridad más cercanos a una aplicación real.

Otra línea de trabajo sería comparar OWASP ZAP con otras herramientas de análisis

de seguridad. El uso de una única herramienta permite centrar el trabajo, pero también limita la comparación de resultados. Utilizar otras herramientas DAST o complementar el análisis con herramientas SAST permitiría estudiar qué vulnerabilidades detecta cada enfoque y qué limitaciones presenta cada uno.

Por último, se podría desplegar la aplicación en un entorno controlado más parecido a producción, por ejemplo mediante contenedores Docker o un servidor de pruebas aislado. Esto permitiría analizar también aspectos relacionados con la configuración del entorno, el despliegue, la gestión de dependencias y la exposición de servicios.

En resumen, el trabajo futuro podría orientarse a construir una versión corregida de la aplicación, repetir las pruebas de seguridad, ampliar el catálogo de vulnerabilidades y comparar distintas herramientas de análisis. Estas líneas permitirían continuar el enfoque práctico seguido en este TFG y profundizar en la integración de la seguridad dentro del ciclo de vida del desarrollo web.

# Apéndice A

# Apéndice

## A.1. Configuración del entorno de pruebas

El análisis se realizó en un equipo con las siguientes características:

- Sistema operativo: macOS ARM
- Versión de OWASP ZAP: 2.16.1
- Tipo de escaneo realizado: Spider tradicional y escaneo activo

## A.2. Ejemplo de consulta vulnerable a SQL Injection

Fragmento simplificado del código utilizado:

```
query = "SELECT * FROM users WHERE id = " + request.GET["id"]
```

# Bibliografía


[1] Verizon Communications Inc. 2024 Data Breach Investigations Report, 2024. Consultado en enero de 2026.

[2] National Institute of Standards and Technology (NIST). Secure Software Development Framework (SSDF) Version 1.1, 2022. Consultado en enero de 2026.

[3] OWASP Foundation. OWASP Application Security Verification Standard (ASVS) v5.0.0. https://owasp.org/www-project-application-security-verification-standard/, 2025. Consultado en enero de 2026.

[4] International Organization for Standardization. ISO/IEC 27001:2022 – Information Security, Cybersecurity and Privacy Protection – Information Security Management Systems – Requirements, 2022. Consultado en enero de 2026.

[5] International Organization for Standardization. ISO/IEC 27002:2022 – Information Security, Cybersecurity and Privacy Protection – Information Security Controls, 2022. Consultado en enero de 2026.

[6] Gobierno de España. Real Decreto 311/2022, de 3 de mayo, por el que se regula el Esquema Nacional de Seguridad. https://www.boe.es/eli/es/rd/2022/05/03/311, 2022. Consultado en enero de 2026.

[7] OWASP Foundation. OWASP Software Assurance Maturity Model (SAMM) v2.0. https://owasp.org/www-project-samm/, 2024. Consultado en enero de 2026.

[8] OWASP Foundation. OWASP Top 10: 2025 – The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/, 2025. Consultado en enero de 2026.

[9] OWASP Foundation. OWASP Zed Attack Proxy (ZAP). https://www.zaproxy.org/, 2025. Consultado en enero de 2026.

[10] Boletín Oficial del Estado. Resolución de 13 de abril de 2026, por la que se registra y publica el Acta de la Comisión paritaria del XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública. https://www.boe.es/buscar/doc.php?id=BOE-A-2026-9024, 2026. Consultado en mayo de 2026.

[11] Hays. Guía del Mercado Laboral 2026. https://guiasalarial.hays.es, 2026. Consultado en mayo de 2026.